

MPMC-*Wortschatz*

Jedes Wort, jede Wendung und jedes Grammatikmuster aus der deutschen Serie, das über A2 hinausgeht — jeweils mit englischer Bedeutung *und* einem Beispielsatz aus dem Text, das Stichwort hervorgehoben. **Neu in dieser Ausgabe:** jeder Beispielsatz trägt seine englische Übersetzung direkt darunter mit hervorgehobenem Kontext.

591 Vokabeleinträge · 84 Lehnwörter · Beispielsätze wörtlich aus **plan/blog/mpmc/de/**, englische Zeile aus **../en/** · T0-T4 = Teil des ersten Vorkommens

Legende. *der* maskulin · *die* feminin · *das* neutrum · "-e Plural mit Umlaut · kein Pl. kein Plural · Stammformen bei starken Verben: Präteritum · Partizip II · (+ Dat.) Dativ-Verb · Die englische Zeile unter jedem Beispiel ist der Paralleltext aus der englischen Fassung.

Kernwortschatz the words the whole series runs on — learn these first

23

Diese Wörter kommen in jedem Teil vor. Wer sie kann, versteht die Struktur jedes Absatzes.

der MPMC-Ring

- e T0

multiple-producer multiple-consumer ring buffer

- **MPMC-Ring** — ein beschränkter Ringpuffer, in den mehrere Producer pushen und aus dem mehrere Consumer poppen, lock-free und alloktationsfrei. - lock-free — irgendein Thread kommt immer voran; kein Thread hält ein Lock, auf das andere warten.

- **MPMC ring** — a bounded circular buffer that multiple producers push and multiple consumers pop, lock-free and allocation-free. - lock-free — some thread always makes progress; no thread holds a lock others wait on.

der Producer

- T0

producer thread

*Diese Serie entwirft die Queue, die das nicht tut: einen Ring — beschränkt, alloktationsfrei, lock-free —, den sich N **Producer** und M Consumer teilen und bei dem nur noch die Übergabe selbst etwas kostet.*

This series designs the queue that doesn't: a bounded, allocation-free, lock-free ring shared by N **producers** and M consumers, where the only cost is the hand-off itself.

der Consumer

- T0

consumer thread

*Diese Serie entwirft die Queue, die das nicht tut: einen Ring — beschränkt, alloktationsfrei, lock-free —, den sich N Producer und M **Consumer** teilen und bei dem nur noch die Übergabe selbst etwas kostet.*

This series designs the queue that doesn't: a bounded, allocation-free, lock-free ring shared by N producers and M **consumers**, where the only cost is the hand-off itself.

der Slot

-s T0

slot, buffer cell

*Ein Boolean pro **Slot** kann das nicht reparieren — ein Boolean hat kein Gedächtnis, und Slots werden über Runden hinweg wiederverwendet.*

A per-slot boolean can't fix it — a boolean has no memory, and slots are reused across laps.

das Ticket

-s T0

ticket (sequencing token)

*Das Lock erledigt, wie sich herausstellt, zwei Jobs — Reihen vergeben und Daten übergeben —, und die beiden zu trennen ist das ganze Design: ein Array, zwei **Ticket**-Zähler und eine Maske.*

The lock turns out to be doing two jobs, allocating turns and handing off data, and separating them is the whole design: an array, two **ticket** counters, and a mask.

die Sequenz

-en T0

sequence counter (per-slot sequence)

*Er zählt nur hoch: pos (Producer darf schreiben), $pos + 1$ (Consumer darf lesen), $pos + capacity$ (der Producer der nächsten Runde darf schreiben). - *claim* vs. *publish* — ein Ticket auf dem Zähler ziehen vs. den payload über **`seq`** sichtbar machen.*

It only increases: pos (producer may write), $pos + 1$ (consumer may read), $pos + capacity$ (next lap's producer may write). - *claim* vs *publish* — taking a ticket on the counter vs making the payload visible through **`seq`**.

der Staffelstab

“-e T2

baton (passed from producer to consumer)

*## Die Sequenz: ein **Staffelstab** mit einer Nummer darauf — Der Fix stammt von Dmitry Vyukov, und er besteht aus einem Integer pro Slot: **seq**.*

The sequence: a **baton** with a number on it — The fix is Dmitry Vyukov's, and it's one integer per slot: **seq**.

der Zähler

- T0

counter (sequence or ticket counter)

*Das Lock erledigt, wie sich herausstellt, zwei Jobs — Reihen vergeben und Daten übergeben —, und die beiden zu trennen ist das ganze Design: ein Array, zwei Ticket-**Zähler** und eine Maske.*

The lock turns out to be doing two jobs, allocating turns and handing off data, and separating them is the whole design: an array, two ticket **counters**, and a mask.

das Atomic

-s T0

atomic variable

*Verschiedene Operationen, verschiedene **Atomics**. - Übergabe (hand-off) — ein Thread ist mit ein paar Bytes fertig und signalisiert es, ein anderer holt sie ab: eine Sendeseite und eine Empfangsseite.*

Different operations, different **atomics**. - hand-off — one thread finishing with some bytes and signaling, another picking them up: a sending side and a receiving side.

der Claim

-s T0

claim (ticket acquisition)

[Teil 2 — Ein Zähler kann nicht „geschrieben“ sagen.](02_the_sequence.md)
*Der naheliegende Ring zerreißt in dem Moment, in dem ein Consumer neben einem Producer läuft, weil **tail** beim ***Claim*** weiterrückt, vor dem Write.*
[Part 2 — One counter can't say "written".](02_the_sequence.md) The obvious ring tears the instant a consumer runs beside a producer, because the tail moves at ***claim*** time, before the write.

die Veröffentlichung

-en T3

publishing, publication

*> Der Claim kann die Daten nicht veröffentlichen, denn zum Zeitpunkt des Claims > existieren die Daten noch nicht. Die **Veröffentlichung** gehört dem letzten Store nach > dem Write — seq.*
*> The claim can't publish the data, because at claim time the data doesn't exist > yet. **Publication** belongs to the last store after the write — seq.*

die Übergabe

-n T0

hand-off (cross-thread data transfer)

*Diese Serie entwirft die Queue, die das nicht tut: einen Ring — beschränkt, alloktationsfrei, lock-free —, den sich N Producer und M Consumer teilen und bei dem nur noch die **Übergabe** selbst etwas kostet.*
This series designs the queue that doesn't: a bounded, allocation-free, lock-free ring shared by N producers and M consumers, where the only cost is the **hand-off** itself.

das Gate

-s T0

gate (memory ordering check / boundary)

*Ein Drei-Wege-**Gate**, die **publish**- und **vacate**-Regeln und der Vertrag darüber, wann **try_push** erneut versucht und wann es zurückkehrt.*
A three-way **gate**, the publish and vacate rules, and the contract for when **try_push** retries and when it returns.

der Boden

“-er T0

floor — in Release/Acquire: Release floors prior stores

***Boden** und Dach, ein Gate nach dem anderen, warum der CAS **ReLaxed** bleibt, und warum dieser Ring — anders als ein SeqLock — keinen Fence braucht.*
Floor and roof, one gate at a time, why the CAS stays **ReLaxed**, and why this ring — unlike a SeqLock — needs no fence.

das Dach

“-er T0

roof — in Release/Acquire: Acquire roofs subsequent loads

*Boden und **Dach**, ein Gate nach dem anderen, warum der CAS **ReLaxed** bleibt, und warum dieser Ring — anders als ein SeqLock — keinen Fence braucht.*
Floor and **roof**, one gate at a time, why the CAS stays **ReLaxed**, and why this ring — unlike a SeqLock — needs no fence.

das Happens-before

T0

happens-before relationship

Zwei pro Slot: A (publish, Producer → Consumer) und B (vacate, Consumer → Producer der nächsten Runde). - **compare_exchange** (CAS) — atomares Read-Modify-Write, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so beansprucht genau ein Thread ein Ticket. - **Relaxed / Acquire / Release** — **Relaxed** ist atomar ohne Ordering; **Release** auf einem Store ist ein Boden unter allem, was davor kommt; **Acquire** auf einem Load ist ein Dach über allem, was danach kommt; ein **Acquire-Load**, der einen **Release-Store** beobachtet, verbindet die beiden. - **happens-before** — die threadübergreifende Garantie, die ein **Release/Acquire**-Paar aufbaut; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - **False Sharing** — zwei unabhängige Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores invalidiert; behoben durch **CachePadded**. - **loom** — ein Model Checker, der einen kleinen Test unter jeder Verschränkung ausführt; blind für Übergaben, die über einen CAS laufen. - **Miri** — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB meldet — Data Races, uninitialisierte Reads, Leaks.

Two per slot: A (publish, producer → consumer) and B (vacate, consumer → next lap's producer). - **compare_exchange** (CAS) — atomic read-modify-write that succeeds only if the value is what you expected; how exactly one thread claims a ticket. - **Relaxed / Acquire / Release** — **Relaxed** is atomic with no ordering; **Release** on a store floors what comes before it; **Acquire** on a load roofs what comes after it; an **Acquire** load that observes a **Release** store joins the two. - **happens-before** — the cross-thread guarantee a **Release/Acquire** pair builds; without one, a plain write on one thread and a plain read on another are a data race. - **false sharing** — two independent values on one cache line, each write invalidating the other core's copy; fixed by **CachePadded**. - **loom** — a model checker that runs a small test under every interleaving; blind to hand-offs carried by a CAS. - **Miri** — an interpreter that runs Rust against the memory model and reports UB — data races, uninitialized reads, leaks.

der Compare-and-Swap

-s T0

compare-and-swap (atomic CAS)

Boden und Dach, ein Gate nach dem anderen, warum der **CAS Relaxed** bleibt, und warum dieser Ring — anders als ein **SeqLock** — keinen Fence braucht.

Floor and roof, one gate at a time, why the **CAS** stays **Relaxed**, and why this ring — unlike a **SeqLock** — needs no fence.

das False Sharing

T0

false sharing (cache line contention)

Zwei pro Slot: A (publish, Producer → Consumer) und B (vacate, Consumer → Producer der nächsten Runde). - **compare_exchange** (CAS) — atomares Read-Modify-Write, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so beansprucht genau ein Thread ein Ticket. - **Relaxed / Acquire / Release** — **Relaxed** ist atomar ohne Ordering; **Release** auf einem Store ist ein Boden unter allem, was davor kommt; **Acquire** auf einem Load ist ein Dach über allem, was danach kommt; ein **Acquire-Load**, der einen **Release-Store** beobachtet, verbindet die beiden. - **happens-before** — die threadübergreifende Garantie, die ein **Release/Acquire**-Paar aufbaut; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - **False Sharing** — zwei unabhängige Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores invalidiert; behoben durch **CachePadded**. - **loom** — ein Model Checker, der einen kleinen Test unter jeder Verschränkung ausführt; blind für Übergaben, die über einen CAS laufen. - **Miri** — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB meldet — Data Races, uninitialisierte Reads, Leaks.

Two per slot: A (publish, producer → consumer) and B (vacate, consumer → next lap's producer). - **compare_exchange** (CAS) — atomic read-modify-write that succeeds only if the value is what you expected; how exactly one thread claims a ticket. - **Relaxed / Acquire / Release** — **Relaxed** is atomic with no ordering; **Release** on a store floors what comes before it; **Acquire** on a load roofs what comes after it; an **Acquire** load that observes a **Release** store joins the two. - **happens-before** — the cross-thread guarantee a **Release/Acquire** pair builds; without one, a plain write on one thread and a plain read on another are a data race. - **false sharing** — two independent values on one cache line, each write invalidating the other core's copy; fixed by **CachePadded**. - **loom** — a model checker that runs a small test under every interleaving; blind to hand-offs carried by a CAS. - **Miri** — an interpreter that runs Rust against the memory model and reports UB — data races, uninitialized reads, leaks.

das Padding

-s T0

cache padding

Die Arbeitsteilung zwischen den drei Instrumenten, ein Drop, das Miris Leak-Checker absegnet, und der Benchmark: **head** und **tail** mit **Padding** auf getrennte cache lines zu legen, bringt die SPSC-Übergabe von ~51 ns auf ~9 ns.

The division of labor between the three instruments, a Drop that Miri's leak checker approves, and the benchmark: **padding** head and tail onto separate cache lines takes the SPSC hand-off from ~51 ns to ~9 ns.

das loom

T0

loom (permutation testing framework)

Kein Lock-free-Hintergrund wird vorausgesetzt; **compare_exchange**, **Acquire/Release**, **UnsafeCell**, **MaybeUninit**, **loom**, **Miri** und **CachePadded** werden dort eingeführt, wo das Design mit ihnen kollidiert.

No lock-free background is assumed; **compare_exchange**, **Acquire/Release**, **UnsafeCell**, **MaybeUninit**, **loom**, **Miri** and **CachePadded** are introduced where the design collides with them.

das **Miri**

T0

Miri (Rust interpreter for UB/memory safety)

*Kein Lock-free-Hintergrund wird vorausgesetzt; `compare_exchange`, `Acquire/Release`, `UnsafeCell`, `MaybeUninit`, `loom`, **Miri** und `CachePadded` werden dort eingeführt, wo das Design mit ihnen kollidiert.*

No lock-free background is assumed; `compare_exchange`, `Acquire/Release`, `UnsafeCell`, `MaybeUninit`, `loom`, **Miri** and `CachePadded` are introduced where the design collides with them.

allokationsfrei

T0

allocation-free (zero heap allocations)

*Diese Serie entwirft die Queue, die das nicht tut: einen Ring — beschränkt, **allokationsfrei**, lock-free —, den sich N Producer und M Consumer teilen und bei dem nur noch die Übergabe selbst etwas kostet.*

This series designs the queue that doesn't: a bounded, **allocation-free**, lock-free ring shared by N producers and M consumers, where the only cost is the hand-off itself.

lock-free

T0

lock-free (system-wide progress guarantee)

*Diese Serie entwirft die Queue, die das nicht tut: einen Ring — beschränkt, allokationsfrei, **lock-free** —, den sich N Producer und M Consumer teilen und bei dem nur noch die Übergabe selbst etwas kostet.*

This series designs the queue that doesn't: a bounded, allocation-free, **lock-free** ring shared by N producers and M consumers, where the only cost is the hand-off itself.

Substantive nouns

164

die Abdeckung

-en T3

coverage

*Das Argument hat drei Schritte: finden, wo Daten von einem Thread zum anderen wandern; entscheiden, welches Atomic jeden dieser Übergänge trägt; und dann `Release` und `Acquire` auf die Seite dieses Atomics legen, die die Daten tatsächlich **abdeckt**.*

The argument has three steps: find where data crosses threads, decide which atomic carries each crossing, then put `Release` and `Acquire` on the side of that atomic that actually covers the data.

die Absicht

-en T3

intention — mit Absicht = on purpose

*## Der CAS ist mit **Absicht** `Relaxed` — Bei (3) zucken die Leute zusammen.
The CAS is `Relaxed` **on purpose** — The one people flinch at is (3).*

die Adresse

-n T4

address (memory address)

*Data Race, die zwei Threads, der Typ an der **Adresse**: `MaybeUninit<usize>`. — Diese Präzision ist der Grund, warum Miri zuerst drankommt.*

Data race, the two threads, the type at the **address**: `MaybeUninit<usize>`. — That precision is why Miri comes first.

der Aggregator

-en T1

aggregator (metrics collector)

Ein Write-Ahead-Log bündelt ein Dutzend Writer-Threads in einen Group Commit. — Eine Metrics-Pipeline führt Samples von jedem Worker in einem einzigen Aggregator zusammen.

A write-ahead log fans a dozen writer threads into one group commit. — A metrics pipeline funnels samples from every worker into one aggregator.

die Allokation

-en T1

allocation (heap allocation)

*Ein Mutex um ein festes Array beseitigt die Allokation und behält das Lock. — Ein Mutex *pro Slot* verkleinert das Lock, ohne es loszuwerden, und bringt ein Problem mit, das das Design vorher nicht hatte: entscheiden, welchen Slot jeder Thread ansteuern soll.*

A Mutex around a fixed array fixes the allocation and keeps the lock. — A mutex *per slot* shrinks the lock without removing it, and adds a problem the design didn't have before: deciding which slot each thread should head for.

der Anfang

“-e T4

beginning, start

Auf dem All-Relaxed-Ring bleibt es beim ersten payload-Read stehen, der keins hat — die Karte am Anfang von Teil 3.

On the all-Relaxed ring it stops at the first payload read that doesn't — the card at the top of Part 3.

die Anforderung

-en T1

requirement

Drei Randbedingungen, nicht eine — Wäre Durchsatz die einzige Anforderung, wäre das ein Benchmark und kein Designproblem.

Three constraints, not one — If throughput were the only requirement this would be a benchmark, not a design problem.

der Anhaltspunkt

-e T3

hint, rough guide

tail.load — ein Anhaltspunkt. — Das Gate validiert ihn erneut, und ein veralteter Wert kostet einen zusätzlichen Schleifendurchlauf. Relaxed. 2.

tail.load — a hint. — The gate re-validates it, and a stale value costs one extra loop. Relaxed. 2.

die Antwort

-en T4

answer, response

*## loom: eine Explosion statt einer Antwort — loom führt einen kleinen Test unter *jeder* Verschränkung aus, die das Memory-Modell erlaubt, innerhalb einer Schranke.*

loom: an explosion instead of an answer — loom runs a small test under *every* interleaving the memory model permits, within a bound.

die Arbeitsteilung

-en T4

division of labor

Die Arbeitsteilung — Die Argumentation entscheidet, welches Ordering und warum, und ist blind für das, woran du nicht gedacht hast.

Division of labor — The reasoning decides which ordering and why, and is blind to what you didn't think of.

das Argument

-e T3

argument, reasoning

*Das **Argument** hat drei Schritte: finden, wo Daten von einem Thread zum anderen wandern; entscheiden, welches Atomic jeden dieser Übergänge trägt; und dann Release und Acquire auf die Seite dieses Atomics legen, die die Daten tatsächlich abdeckt.*

The **argument** has three steps: find where data crosses threads, decide which atomic carries each crossing, then put Release and Acquire on the side of that atomic that actually covers the data.

die Argumentation

-en T4

reasoning, deduction

*## Die Arbeitsteilung — Die **Argumentation** entscheidet, welches Ordering und warum, und ist blind für das, woran du nicht gedacht hast.*

Division of labor — The **reasoning** decides which ordering and why, and is blind to what you didn't think of.

die Atomarität

T3

atomicity (indivisibility)

***Atomarität** ist nicht Ordering. — `compare_exchange` garantiert, dass das Read-Modify-Write unteilbar ist — keine zwei Threads nehmen Ticket pos.*

Atomicity is not ordering. — `compare_exchange` guarantees the read-modify-write is indivisible — no two threads take ticket pos.

der Aufruf

-e T1

invocation, function call

*Ein Allocator-**Aufruf** bedeutet an einem guten Tag hunderte Nanosekunden und an einem schlechten ein prozessweites Lock; so oder so passt er nicht in eine Übergabe, die in unter einer Mikrosekunde fertig sein muss. - Lock-free. Kein einzelnes Speicherwort, das sich jeder Thread erst holen muss.*

An allocator **call** is hundreds of nanoseconds on a good day and a process-wide lock on a bad one; either way it doesn't fit inside a hand-off that has to complete in under a microsecond. - Lock-free. No single word every thread must acquire.

der Aufrufer

- T1

caller

*Die Queue parkt nie einen Thread. — Blockieren ist eine Policy, die der **Aufrufer** vielleicht will, und sie darf nicht fest ins Primitiv eingebaut sein.*

The queue never parks a thread. — Blocking is a policy the **caller** may want, and it must not be baked into the primitive.

das Ausprobieren

T3

trial and error

*Dieser Teil platziert jedes Ordering per Argument, nicht per **Ausprobieren**.*

This part places every ordering by argument, not by **trial**.

die Aussage

-n T0

statement, assertion

*Die Zahlen in Teil 4 stammen aus den **criterion**-Benchs der Crate, und jede **Aussage** über Korrektheit ist eine, die loom oder Miri tatsächlich geprüft hat.*

The numbers in Part 4 are from the crate's **criterion** benches, and every **claim** about correctness is one loom or Miri actually checked.

die Barriere

-n T3

barrier (hardware memory fence)

Der CAS sitzt zwischen zwei Gates, die die Arbeit erledigen. — Ein AcqRel darauf würde nichts einbringen und bei jedem Claim eine Barriere kosten — auf aarch64 ein casl, wo ein cas reicht.

The CAS sits between two gates that do the work. — An AcqRel on it would buy nothing and cost a **barrier** on every claim — on aarch64, a `casl` where a `cas` will do.

der Beleg

-e T4

evidence, proof

> loom kann einen Race nicht sehen, der über ein Compare-and-Swap läuft. — Sein Grün ist > nur ein Beleg für Übergaben, die über einen einfachen Store und Load gehen.

> loom cannot see a race carried by a compare-and-swap. — Its green is **evidence** only > for hand-offs that go through a plain store and load.

der Bereich

-e T2

range, scope, region

Er sagt nicht „frei“ oder „voll“. — Er sagt, für welches Ticket der Slot bereit ist, und diese Zahl geht immer nur nach oben.

It doesn't say "free" or "full". — It says which ticket the slot is ready for, and that number only ever goes up.

der Beweis

-e T4

proof

Teil 4 — Der Beweis: zwei Werkzeuge und ein lügendes Grün — Teil 3 hat jedes Ordering per Argument platziert.

Part 4 — **Proving** it: two tools, and a lying green — Part 3 placed every ordering by argument.

der Block

“-e T1

block

Die Queue parkt nie einen Thread. — Blockieren ist eine Policy, die der Aufrufer vielleicht will, und sie darf nicht fest ins Primitiv eingebaut sein.

The queue never parks a thread. — Blocking is a policy the caller may want, and it must not be baked into the primitive.

der Boden

“- T0

floor — in Release/Acquire: Release floors prior stores

Boden und Dach, ein Gate nach dem anderen, warum der CAS Relaxed bleibt, und warum dieser Ring — anders als ein SeqLock — keinen Fence braucht.

Floor and roof, one gate at a time, why the CAS stays Relaxed, and why this ring — unlike a SeqLock — needs no fence.

das Dach

“-er T0

roof — in Release/Acquire: Acquire roofs subsequent loads

Boden und Dach, ein Gate nach dem anderen, warum der CAS Relaxed bleibt, und warum dieser Ring — anders als ein SeqLock — keinen Fence braucht.

Floor and **roof**, one gate at a time, why the CAS stays Relaxed, and why this ring — unlike a SeqLock — needs no fence.

die Daten

pl. T0

data

*Das Lock erledigt, wie sich herausstellt, zwei Jobs — Reihen vergeben und **Daten** übergeben —, und die beiden zu trennen ist das ganze Design: ein Array, zwei Ticket-Zähler und eine Maske.*

The lock turns out to be doing two jobs, allocating turns and handing off **data**, and separating them is the whole design: an array, two ticket counters, and a mask.

das Detail

-s T1

detail

*Drei der vier opfern dasselbe. — Das Lock ist kein **Detail** dieser Designs; sie sind um das Lock herum gebaut.*

Three of the four surrender the same thing. — The lock isn't a **detail** of these designs; it's what they're built around.

die Differenz

-en T2

difference (e.g. signed difference)

*## Das Drei-Wege-Gate — Der Vergleich ist eine vorzeichenbehaftete **Differenz**, `seq.wrapping_sub(pos)` as `isize`, damit er aussagekräftig bleibt, falls die Zähler je überlaufen.*

The three-way gate — The comparison is a signed **difference**, `seq.wrapping_sub(pos)` as `isize`, so it stays meaningful if the counters ever wrap.

der Durchlauf

“-e T3

pass, iteration (of a loop)

*`tail.load` — ein Anhaltspunkt. — Das Gate validiert ihn erneut, und ein veralteter Wert kostet einen zusätzlichen **Schleifendurchlauf**. Relaxed. 2.*

`tail.load` — a hint. — The gate re-validates it, and a stale value costs one extra **loop**. Relaxed. 2.

der Durchsatz

T1

throughput

*## Drei Randbedingungen, nicht eine — Wäre **Durchsatz** die einzige Anforderung, wäre das ein Benchmark und kein Designproblem.*

Three constraints, not one — If **throughput** were the only requirement this would be a benchmark, not a design problem.

das Dutzend

-e T0

dozen

*# Einen lock-freien MPMC-Ring entwerfen — Am schreibenden Ende von allem, was schnell ist — eine Matching Engine, die Orders von einem **Dutzend** Feed-Threads entgegennimmt, ein Write-Ahead-Log, das viele Writer in einen einzigen Commit bündelt —, sitzt eine Queue, in die viele Threads pushen und aus der viele Threads poppen, millionenfach pro Sekunde.*

Designing a lock-free MPMC ring — At the write end of anything fast — a matching engine taking orders off a **dozen** feed threads, a write-ahead log fanning writers into one commit — there's a queue that many threads push and many threads pop, millions of times a second.

die Eigenschaft

-en T1

property, attribute

*Das Lock ist kein Detail dieser Designs; sie sind um das Lock herum gebaut. — Die Frage ist also nicht „welches Lock ist am billigsten“ — sondern was das Lock **eigentlich** *tut*, und ob sich diese Arbeit auch ohne es erledigen lässt.*
The lock isn't a detail of these designs; it's what they're built around. — So the question isn't "which lock is cheapest" — it's what the lock is actually *doing*, and whether that job can be done without it.

der Eingang

“-e T1

intake, entrance

*# Teil 1 — Das Problem, und warum die naheliegenden Queues nicht passen — Zwölf Feed-Handler, einer pro Core, jeder zieht Orders aus einer anderen Exchange-Session und schiebt sie an eine einzige Stelle: den **Eingang** der Matching Engine.*

Part 1 — The problem, and why the obvious queues don't fit — Twelve feed handlers, one per core, each pulling orders off a different exchange session and pushing them into one place: the matching engine's **intake**.

das Element

-e T0

element, item in queue

*Nicht wait-free: Ein einzelner Thread kann es erneut versuchen müssen. - Ticket — der Wert, den ein Thread von **tail** (Producer) oder **head** (Consumer) zieht; **ticket & (capacity - 1)** ist sein Slot. - Slot / Zelle — ein **Element** des Puffers: ein payload und seine Sequenznummer. - **seq** — der Zähler pro Slot, der sagt, für welches Ticket der Slot bereit ist.*

Not wait-free: an individual thread may retry. - ticket — the value a thread takes from **tail** (producer) or **head** (consumer); **ticket & (capacity - 1)** is its slot. - slot / cell — one **element** of the buffer: a payload and its sequence number. - **seq** — the per-slot counter that says which ticket the slot is ready for.

die Empfangsseite

-n T0

receiving side (of hand-off)

*Verschiedene Operationen, verschiedene Atomics. - Übergabe (hand-off) — ein Thread ist mit ein paar Bytes fertig und signalisiert es, ein anderer holt sie ab: eine Sendeseite und eine **Empfangsseite**.*

Different operations, different atomics. - hand-off — one thread finishing with some bytes and signaling, another picking them up: a sending side and a **receiving side**.

das Ende

-n T0

end (e.g. write end)

*# Einen lock-freien MPMC-Ring entwerfen — Am schreibenden **Ende** von allem, was schnell ist — eine Matching Engine, die Orders von einem Dutzend Feed-Threads entgegennimmt, ein Write-Ahead-Log, das viele Writer in einen einzigen Commit bündelt —, sitzt eine Queue, in die viele Threads pushen und aus der viele Threads poppen, millionenfach pro Sekunde.*

Designing a lock-free MPMC ring — At the write **end** of anything fast — a matching engine taking orders off a dozen feed threads, a write-ahead log fanning writers into one commit — there's a queue that many threads push and many threads pop, millions of times a second.

die Entscheidung

-en T0

decision

*Es ist eine Design-Untersuchung, kein Tutorial. — Jede **Entscheidung** ist erzwungen — durch einen Use Case, durch das Scheitern der einfacheren Variante oder durch ein Verifikationswerkzeug.*

It's a design investigation, not a tutorial. — Every **decision** is forced — by a use case, by the simpler version failing, or by a verification tool.

der Erfolg

-e T3

success (e.g. CAS success)

*Der Slot wurde schon in (2) geprüft; der payload wird in (5) veröffentlicht. — Auf dieser Op wechseln keine Daten den Thread. Relaxed, **Erfolg** wie Fehlschlag. 4.*

The slot was already checked by (2); the payload will be published by (5). — No data crosses on this op. Relaxed, **success** and failure. 4.

die Explosion

-en T4

explosion (state space explosion in model checking)

*## loom: eine **Explosion** statt einer Antwort — loom führt einen kleinen Test unter *jeder* Verschränkung aus, die das Memory-Modell erlaubt, innerhalb einer Schranke.*

loom: an **explosion** instead of an answer — loom runs a small test under *every* interleaving the memory model permits, within a bound.

der Fehler

- T0

bug, error, failure

*Nach Teil 2 aufzuhören lässt dich mit einem Ring zurück, der logisch korrekt ist; die Teile 3 und 4 sind der Ort, an dem er auf die Hardware und die Verifizierer trifft und an dem die **Fehler** wohnen, die Tests bestehen.*

Stopping after Part 2 leaves you with a ring that's logically correct; Parts 3 and 4 are where it meets the hardware and the verifiers, and where the **failures** that pass tests live.

der Fehlschlag

-e T3

failure (e.g. CAS failure)

*Der Slot wurde schon in (2) geprüft; der payload wird in (5) veröffentlicht. — Auf dieser Op wechseln keine Daten den Thread. Relaxed, Erfolg wie **Fehlschlag**. 4.*

The slot was already checked by (2); the payload will be published by (5). — No data crosses on this op. Relaxed, success and **failure**. 4.

die Folge

-n T1

sequence, run (e.g. of bits)

*Dieses & ist ein Modulo, aber nur, wenn capacity eine Zweierpotenz ist — dann ist $capacity - 1$ eine **Folge** niedriger 1-Bits, und die Maske behält genau diese.*

That & is a modulo, but only when capacity is a power of two — then $capacity - 1$ is a **run** of low 1 bits and the mask keeps exactly them.

die Form

-en T1

shape, pattern

*Diese **Form** taucht überall auf, wo schnelle Software ein schreibendes Ende hat. — Ein Write-Ahead-Log bündelt ein Dutzend Writer-Threads in einen Group Commit.*

The **shape** is everywhere fast software has a write end. — A write-ahead log fans a dozen writer threads into one group commit.

der Fortschritt

T1

progress (forward progress in lock-free)

*Die Garantie, die wir wollen: Zu jedem Zeitpunkt macht *irgendein* Thread **Fortschritt**, und ein Thread, den das OS mitten in einer Operation unterbricht — und der dabei nichts hält — kann den Rest nicht einfrieren. - Viele-zu-viele, und nicht-blockierend. N Producer, M Consumer, keine festen Rollen.*

The guarantee we want is that at any moment *some* thread is making **progress**, and a thread the OS preempts mid-operation — holding nothing — cannot freeze the rest. - Many-to-many, and non-blocking. N producers, M consumers, no fixed roles.

die Frage

-n T0

question

*Der Reihe nach. — Jeder Teil beginnt dort, wo der vorige aufgehört hat, und schließt mit der **Frage**, die der nächste beantwortet.*

In order. — Each part opens where the previous one stopped and closes on the **question** the next one answers.

die Garantie

-n T0

guarantee (hardware/ordering guarantee)

*Gebaut und gemessen auf **aarch64** (Apple M2), denn ein schwaches Memory-Modell ist der Ort, an dem sich die Fehler aus Teil 3 zeigen; auf x86 würde sich der Data Race hinter einer stärkeren Hardware-**Garantie** verstecken.*

Built and measured on **aarch64** (Apple M2), because a weak memory model is where the Part 3 failures show; on x86 the data race would hide behind a stronger hardware **guarantee**.

das Gedächtnis

-se T0

memory (ability to retain state across laps)

*Ein Boolean pro Slot kann das nicht reparieren — ein Boolean hat kein **Gedächtnis**, und Slots werden über Runden hinweg wiederverwendet.*

A per-slot boolean can't fix it — a boolean has no **memory**, and slots are reused across laps.

die Gegenseite

-n T3

opposite side, far side

*Beim SeqLock legten zwei der vier Kanten den payload auf die **Gegenseite**: Die payload-Stores des Writers kamen *nach* dem Öffnenden Bump, und die Kopie des Readers kam *vor* der schließenden Prüfung.*

In the SeqLock, two of the four edges put the payload on the **far side**: the writer's payload stores came *after* the opening bump, and the reader's copy came *before* the closing check.

das Gegenteil

-e T1

opposite

*Und in dem Moment, in dem der Halter des Locks vom OS verdrängt wird, wartet jeder andere Thread darauf, dass genau dieser eine Thread wieder eingeplant wird. Das ist das **Gegenteil** von lock-free.*

And the instant the holder is preempted, every other thread waits for that one thread to be scheduled again. That is the **opposite** of lock-free.

der Gewinn

-e T4

gain, speedup, benefit

Die Kontrolle, ein Thread, der beides macht, geht von 7,3 auf 7,0 — Rauschen — und genau das sagt dir, dass der Gewinn vom Padding kommt und nicht vom Lauf.

The control, one thread doing both, moves from 7.3 to 7.0 — noise — which is what tells you the **win** is the padding and not the run.

das Glossar

-e T0

glossary

Glossar — - MPMC-Ring — ein beschränkter Ringpuffer, in den mehrere Producer pushen und aus dem mehrere Consumer poppen, lock-free und allokatonsfrei. - lock-free — irgendein Thread kommt immer voran; kein Thread hält ein Lock, auf das andere warten.

Glossary — - MPMC ring — a bounded circular buffer that multiple producers push and multiple consumers pop, lock-free and allocation-free. - lock-free — some thread always makes progress; no thread holds a lock others wait on.

das Grün

T0

green (passing test status)

[Teil 4 — Der Beweis: zwei Werkzeuge und ein lügendes Grün.] (04_proving_it.md) Miri nennt den Race beim Namen; loom, mit dem Relaxed-Ring gefüttert, explodiert, statt zu antworten.

[Part 4 — Proving it: two tools, and a lying green.](04_proving_it.md) Miri names the race; loom, given the Relaxed ring, explodes instead of answering.

der Halter

- T1

holder (of a lock)

Und in dem Moment, in dem der Halter des Locks vom OS verdrängt wird, wartet jeder andere Thread darauf, dass genau dieser eine Thread wieder eingeplant wird.

And the instant the **holder** is preempted, every other thread waits for that one thread to be scheduled again.

das Herz

-en T0

heart, core (of the series)

Boden und Dach, ein Gate nach dem anderen, warum der CAS Relaxed bleibt, und warum dieser Ring — anders als ein SeqLock — keinen Fence braucht. Das Herz der Serie.

Floor and roof, one gate at a time, why the CAS stays Relaxed, and why this ring — unlike a SeqLock — needs no fence. The **heart** of the series.

der Hintergrund

“-e T0

background (domain knowledge)

Kein Lock-free-Hintergrund wird vorausgesetzt; compare_exchange, Acquire/Release, UnsafeCell, MaybeUninit, loom, Miri und CachePadded werden dort eingeführt, wo das Design mit ihnen kollidiert.

No lock-free **background** is assumed; compare_exchange, Acquire/Release, UnsafeCell, MaybeUninit, loom, Miri and CachePadded are introduced where the design collides with them.

die Hälfte

-n T3

half

Über das, was davor kommt, sagt es nichts. — Liest der Acquire-Load den Wert, den der Release-Store geschrieben hat, rasten die beiden Hälften ineinander: Was über dem Boden lag, ist jetzt garantiert unter dem Dach sichtbar.

It says nothing about what comes before. — When the Acquire load reads the value the Release store wrote, the two halves lock: what was above the floor is now guaranteed visible below the roof.

das Instrument

-e T0

instrument, tool (e.g. Miri, loom)

Die Arbeitsteilung zwischen den drei Instrumenten, ein Drop, das Miris Leak-Checker absegnet, und der Benchmark: head und tail mit Padding auf getrennte cache lines zu legen, bringt die SPSC-Übergabe von ~51 ns auf ~9 ns.

The division of labor between the three instruments, a Drop that Miri's leak checker approves, and the benchmark: padding head and tail onto separate cache lines takes the SPSC hand-off from ~51 ns to ~9 ns.

die Iteration

-en T4

iteration (in benchmark loop)

Jetzt die Zahl. — criterion auf dem M2, ein Element pro Iteration gepusht und gepoppt:

Now the number. — criterion on the M2, one item pushed and popped per iteration:

der Kandidat

-en T1

candidate (queue candidate)

Miss jeden Kandidaten unten an diesen dreien. — Jeder ist korrekt, und jeder gibt genau eine davon auf.

Hold every candidate below against those three. — Each one is correct, and each one gives up exactly one.

die Kante

-n T3

edge (in happens-before graph)

*Beim SeqLock legten zwei der vier Kanten den payload auf die Gegenseite: Die payload-Stores des Writers kamen *nach* dem öffnenden Bump, und die Kopie des Readers kam *vor* der schließenden Prüfung.*

In the SeqLock, two of the four edges put the payload on the far side: the writer's payload stores came *after* the opening bump, and the reader's copy came *before* the closing check.

die Kapazität

-en T4

capacity (buffer capacity)

Eine Contention-Achse pro Modell — zwei Producer und ein Consumer oder zwei Consumer und ein Producer, nie alles auf einmal — eine Kapazität von zwei, die Producer gejoint, bevor der Consumer den Ring leert, und jeder Retry im Testcode beschränkt.

One contention axis per model — two producers and a consumer, or two consumers and a producer, never everything at once — a capacity of two, producers joined before the consumer drains, and any retry in the test body bounded.

der **Knoten**

- T1

node (linked node)

- Beschränkt und allokatonsfrei. Ein fester Puffer, einmal dimensioniert. — Keine **Box** pro Nachricht, kein verketteter **Knoten**, der beim Push alloziert und beim Pop freigegeben wird.

- Bounded and allocation-free. A fixed buffer, sized once. — No **Box** per message, no linked **node** allocated on push and freed on pop.

die **Konsequenz**

-en T2

consequence

Eine **Konsequenz**, die man aussprechen sollte: `try_pop` gibt `None` zurück, wenn das Element **am Head** nicht veröffentlicht ist, selbst wenn spätere Slots es sind.

One **consequence** worth stating: `try_pop` returns `None` when the item **at the head** isn't published, even if later slots are.

der **Konstruktor**

-en T1

constructor function

Der **Konstruktor** prüft die Zweierpotenz also per `Assert` und speichert `mask = capacity - 1` ein einziges Mal.

So the **constructor** asserts the power of two and stores `mask = capacity - 1` once.

die **Kontrolle**

-n T4

control (baseline measurement)

Die **Kontrolle**, ein Thread, der beides macht, geht von 7,3 auf 7,0 — Rauschen — und genau das sagt dir, dass der Gewinn vom Padding kommt und nicht vom Lauf.

The **control**, one thread doing both, moves from 7.3 to 7.0 — noise — which is what tells you the win is the padding and not the run.

die **Kopie**

-n T4

copy (cache line copy)

Der Producer schreibt `tail`, und das invalidiert die **Kopie** der cache line auf dem Consumer-Core.

The producer writes `tail`, which invalidates the consumer core's **copy** of the line.

die **Korrektheit**

T0

correctness

Die Zahlen in Teil 4 stammen aus den `criterion`-Benchs der Crate, und jede Aussage über **Korrektheit** ist eine, die loom oder Miri tatsächlich geprüft hat.

The numbers in Part 4 are from the crate's `criterion` benches, and every claim about **correctness** is one loom or Miri actually checked.

die **Kosten**

Pl. T4

cost (in nanoseconds/cycles)

Das Siebenfache der Ein-Thread-**Kosten**, für eine Übergabe, die laut Teil 1 **keine** logische Contention hat: ein Schreiber pro Slot, ein Leser pro Slot und die beiden Zähler von verschiedenen Threads angefasst.

Seven times the single-thread **cost**, for a hand-off that Part 1 established has **no** logical contention: one writer per slot, one reader per slot, and the two counters touched by different threads.

der Lauf

--e T4

run (benchmark or execution run)

*Die Kontrolle, ein Thread, der beides macht, geht von 7,3 auf 7,0 — Rauschen — und genau das sagt dir, dass der Gewinn vom Padding kommt und nicht vom **Lauf**.*

The control, one thread doing both, moves from 7.3 to 7.0 — noise — which is what tells you the win is the padding and not the **run**.

der Leser

- T4

reader thread

*Das Siebenfache der Ein-Thread-Kosten, für eine Übergabe, die laut Teil 1 *keine* logische Contention hat: ein Schreiber pro Slot, ein **Leser** pro Slot und die beiden Zähler von verschiedenen Threads angefasst.*

Seven times the single-thread cost, for a hand-off that Part 1 established has *no* logical contention: one writer per slot, one **reader** per slot, and the two counters touched by different threads.

das Loch

--er T1

hole, flaw (in an idea)

*Dann zerreißt es, sobald zwei Threads es zum ersten Mal wirklich anfassen, aus einem Grund, der kein Bug im Code ist, sondern ein **Loch** in der Idee.*

Then it tears the first time two threads touch it for real, for a reason that isn't a bug in the code but a **hole** in the idea.

das Lock-Wort

--er T0

lock word (atomic variable storing lock state)

*Greif zu einem `Mutex<VecDeque>`, und jeder dieser Threads serialisiert sich auf einem einzigen **Lock-Wort**, das zwischen den Cores hin- und herspringt.*

Reach for a `Mutex<VecDeque>` and every one of those threads serializes on one **lock word** that bounces between cores.

die Logik

T2

logic (algorithmic logic)

*Die **Logik** stimmt. — Falsch ist das eine, was die Logik nicht kontrolliert: die Reihenfolge, in der zwei Cores Änderungen im Speicher sehen.*

The **logic** is right. — What's wrong is the one thing the logic doesn't control: the order in which two cores see memory change.

die Lücke

-n T2

gap, window of vulnerability

*## Zwei Producer machen es strukturell — Mit zwei Producern ist die **Lücke** kein Wettlauf mehr, sondern eine Tatsache über Zähler:*

*## Two producers make it structural — With two producers the **gap** stops being a race and becomes a fact about counters:*

die Maske

-n T0

mask (bitmask)

*Das Lock erledigt, wie sich herausstellt, zwei Jobs — Reihen vergeben und Daten übergeben —, und die beiden zu trennen ist das ganze Design: ein Array, zwei Ticket-Zähler und eine **Maske**.*

The lock turns out to be doing two jobs, allocating turns and handing off data, and separating them is the whole design: an array, two ticket counters, and a **mask**.

die Menge

-n T2

set, quantity

tail = 2 sagt: Zwei Tickets sind beansprucht. — Die **Menge** der tatsächlich geschriebenen Slots ist {1}.

tail = 2 says two tickets are claimed. — The **set** of slots actually written is {1}.

das MESI-Protokoll

T1

MESI cache coherence protocol

Aber jeder *push* und jeder *pop* nimmt dasselbe Lock, also springt das Lock-Wort bei jeder einzelnen Operation wie ein Pingpongball zwischen den Cores hin und her — das **MESI-Protokoll** invalidiert eine cache line in jedem anderen Core, millionenfach pro Sekunde, selbst wenn logisch gar keine Contention herrscht.

But every push and every pop takes the same lock, so the lock word ping-pongs between cores on every single operation — the **MESI protocol** invalidating one cache line in every other core, millions of times a second, even when nobody is logically contending.

das Modell

-e T0

model (state space model)

Gebaut und gemessen auf *aarch64* (Apple M2), denn ein schwaches Memory-**Modell** ist der Ort, an dem sich die Fehler aus Teil 3 zeigen; auf *x86* würde sich der Data Race hinter einer stärkeren Hardware-Garantie verstecken.

Built and measured on *aarch64* (Apple M2), because a weak memory **model** is where the Part 3 failures show; on *x86* the data race would hide behind a stronger hardware guarantee.

der Moment

-e T0

instant, moment

[Teil 2 — Ein Zähler kann nicht „geschrieben“ sagen.](02_the_sequence.md) Der naheliegende Ring zerreißt in dem **Moment**, in dem ein Consumer neben einem Producer läuft, weil *tail* beim **Claim** weiterrückt, vor dem Write.

[Part 2 — One counter can't say "written".](02_the_sequence.md) The obvious ring tears the **instant** a consumer runs beside a producer, because the tail moves at **claim** time, before the write.

der Nachhinein

T2

im Nachhinein = retroactively, in hindsight

Nur der Producer, der Ticket *pos* hält, kann *seq* von *pos* auf *pos + 1* bewegen, und wenn mein CAS auf *tail* von *pos* aus gelingt, dann bin ich dieser Producer. Ein erfolgreicher Claim beweist im **Nachhinein**, dass der Gate-Read aktuell war.

Only the producer holding ticket *pos* can move *seq* from *pos* to *pos + 1*, and if my CAS on *tail* succeeds from *pos*, that producer is me. A successful claim **retroactively** proves the gate read was current.

die Nachricht

-en T1

message

- Beschränkt und allokatonsfrei. Ein fester Puffer, einmal dimensioniert. — Keine Box pro **Nachricht**, kein verketteter Knoten, der beim Push alloziert und beim Pop freigegeben wird.

- Bounded and allocation-free. A fixed buffer, sized once. — No Box per **message**, no linked node allocated on push and freed on pop.

der Name

-n T0

name

[Teil 4 — Der Beweis: zwei Werkzeuge und ein lügendes Grün.]
(04_proving_it.md) Miri nennt den Race beim **Namen**; loom, mit dem **ReLaxed**-Ring gefüttert, explodiert, statt zu antworten.

[Part 4 — Proving it: two tools, and a lying green.](04_proving_it.md) Miri names the race; loom, given the ReLaxed ring, explodes instead of answering.

die Nanosekunde

-n T1

nanosecond (10⁻⁹ s)

Ein Allocator-Aufruf bedeutet an einem guten Tag hunderte **Nanosekunden** und an einem schlechten ein prozessweites Lock; so oder so passt er nicht in eine Übergabe, die in unter einer Mikrosekunde fertig sein muss. - Lock-free. Kein einzelnes Speicherwort, das sich jeder Thread erst holen muss.

An allocator call is hundreds of **nanoseconds** on a good day and a process-wide lock on a bad one; either way it doesn't fit inside a hand-off that has to complete in under a microsecond. - Lock-free. No single word every thread must acquire.

das Nichts

T1

nothingness — nichts hält

Die Garantie, die wir wollen: Zu jedem Zeitpunkt macht **irgendein** Thread Fortschritt, und ein Thread, den das OS mitten in einer Operation unterbricht — und der dabei **nichts** hält — kann den Rest nicht einfrieren. - Viele-zu-viele, und nicht-blockierend. N Producer, M Consumer, keine festen Rollen.

The guarantee we want is that at any moment **some** thread is making progress, and a thread the OS preempts mid-operation — holding **nothing** — cannot freeze the rest. - Many-to-many, and non-blocking. N producers, M consumers, no fixed roles.

die Nummer

-n T2

number (ticket number)

Die Sequenz: ein Staffeltab mit einer **Nummer** darauf — Der Fix stammt von Dmitry Vyukov, und er besteht aus einem Integer pro Slot: *seq*.

The sequence: a baton with a **number** on it — The fix is Dmitry Vyukov's, and it's one integer per slot: *seq*.

die Operation

-en T0

operation

Verschiedene **Operationen**, verschiedene Atomics. - Übergabe (hand-off) — ein Thread ist mit ein paar Bytes fertig und signalisiert es, ein anderer holt sie ab: eine Sendeseite und eine Empfangsseite.

Different **operations**, different atomics. - hand-off — one thread finishing with some bytes and signaling, another picking them up: a sending side and a receiving side.

das **Paar**

-e T1

pair (thread pair or Acquire/Release pair)

*Zwei Consumer: dasselbe. — Die einzige Stelle, an der zwei Threads dieselben Bytes anfassen, liegt zwischen dem Producer, der Slot *k* füllt, und dem Consumer, der Slot *k* später leert — ein Thread auf jeder Seite, und für jeden Slot ein anderes **Paar**.*

Two consumers, the same. — The only place two threads touch the same bytes is between the producer that fills slot *k* and the consumer that later empties slot *k* — one thread on each side, and a different **pair** for every slot.

der **Pfad**

-e T1

path (hot path or execution path)

*Mitten im heißesten **Pfad** des Systems sitzt eine Queue, und das Erste, wonach jeder greift, ist `Mutex<VecDeque<T>>`.*

In the middle of the hottest **path** in the system sits a queue, and the first thing anyone reaches for is `Mutex<VecDeque<T>>`.

der **Pingpongball**

--e T1

ping-pong ball (cache line bouncing)

*Aber jeder **push** und jeder **pop** nimmt dasselbe Lock, also springt das Lock-Wort bei jeder einzelnen Operation wie ein **Pingpongball** zwischen den Cores hin und her — das MESI-Protokoll invalidiert eine cache line in jedem anderen Core, millionenfach pro Sekunde, selbst wenn logisch gar keine Contention herrscht.*

But every push and every pop takes the same lock, so the lock word ping-pongs between cores on every single operation — the MESI protocol invalidating one cache line in every other core, millions of times a second, even when nobody is logically contending.

der **Preis**

-e T2

cost, price (trade-off paid)

*Consumer springen nicht vor. — Genau das hält den Ring FIFO, und das ist der **Preis** dafür.*

Consumers don't skip ahead. — That's what keeps the ring FIFO, and it's the **price** of it.

das **Primitiv**

-e T1

concurrency primitive

*Die Queue parkt nie einen Thread. — Blockieren ist eine Policy, die der Aufrufer vielleicht will, und sie darf nicht fest ins **Primitiv** eingebaut sein.*

The queue never parks a thread. — Blocking is a policy the caller may want, and it must not be baked into the **primitive**.

das **Problem**

-e T0

problem

*## Die Teile — [Teil 1 — Das **Problem**, und warum die naheliegenden Queues nicht passen.](01_the_problem.md) Drei Randbedingungen — beschränkt und allokatonsfrei, lock-free, Viele-zu-viele und nicht-blockierend — und die Tour durch die Queues, von denen jede eine davon verletzt.*

The parts — [Part 1 — The **problem**, and why the obvious queues don't fit.](01_the_problem.md) Three constraints — bounded and allocation-free, lock-free, many-to-many and non-blocking — and the tour of queues that each break one.

das Programm

-e T4

program

*## Miri: der Race, beim Namen genannt — Miri interpretiert das kompilierte **Programm** und verfolgt jedes Byte: wer es geschrieben hat, auf welchem Thread, mit welchem Ordering und ob der Read danach ein happens-before zum Write hat.*

*## Miri: the race, by name — Miri interprets the compiled **program** and tracks every byte: who wrote it, on which thread, with what ordering, and whether the read that follows has a happens-before to the write.*

die

Programmreihenfolge

T3

program order (sequence of statements in code)

*Beide Orderings sind einseitig, und es lohnt sich, genau festzuhalten, welche Seite. — Release auf einem Store ist ein Boden unter allem, was in der **Programmreihenfolge** davor kommt: Nichts darüber darf unter den Store sinken.*

The two orderings are one-sided, and it's worth being exact about which side. — Release on a store is a floor under everything before it in **program order**: nothing above may sink below the store.

die Präzision

T4

precision, exactness

*Data Race, die zwei Threads, der Typ an der Adresse: MaybeUninit<usize>. — Diese **Präzision** ist der Grund, warum Miri zuerst drankommt.*

Data race, the two threads, the type at the address: MaybeUninit<usize>. — That **precision** is why Miri comes first.

der Puffer

- T0

buffer

*Nicht wait-free: Ein einzelner Thread kann es erneut versuchen müssen. - Ticket — der Wert, den ein Thread von tail (Producer) oder head (Consumer) zieht; ticket & (capacity - 1) ist sein Slot. - Slot / Zelle — ein Element des **Puffers**: ein payload und seine Sequenznummer. - seq — der Zähler pro Slot, der sagt, für welches Ticket der Slot bereit ist.*

Not wait-free: an individual thread may retry. - ticket — the value a thread takes from tail (producer) or head (consumer); ticket & (capacity - 1) is its slot. - slot / cell — one element of the **buffer**: a payload and its sequence number. - seq — the per-slot counter that says which ticket the slot is ready for.

die Randbedingung

-en T0

constraint, boundary condition

*## Die Teile — [Teil 1 — Das Problem, und warum die naheliegenden Queues nicht passen.](01_the_problem.md) Drei **Randbedingungen** — beschränkt und allokatonsfrei, lock-free, Viele-zu-viele und nicht-blockierend — und die Tour durch die Queues, von denen jede eine davon verletzt.*

*## The parts — [Part 1 — The problem, and why the obvious queues don't fit.](01_the_problem.md) Three **constraints** — bounded and allocation-free, lock-free, many-to-many and non-blocking — and the tour of queues that each break one.*

die Rate

-n T1

rate, frequency

*Jedes Mal dasselbe Objekt: viele Threads pushen, viele Threads poppen, auf dem Hot Path, bei einer **Rate**, bei der der Overhead der Queue selbst das Budget ist.*

Same object each time: many threads push, many threads pop, on the hot path, at a **rate** where the queue's own overhead is the budget.

das Rauschen

T4

noise (measurement noise)

*Die Kontrolle, ein Thread, der beides macht, geht von 7,3 auf 7,0 — **Rauschen** — und genau das sagt dir, dass der Gewinn vom Padding kommt und nicht vom Lauf.*

The control, one thread doing both, moves from 7.3 to 7.0 — **noise** — which is what tells you the win is the padding and not the run.

die Regel

-n T0

rule

*Ein Drei-Wege-Gate, die **publish**- und **vacate**-**Regeln** und der Vertrag darüber, wann **try_push** erneut versucht und wann es zurückkehrt.*

A three-way gate, the **publish** and **vacate** **rules**, and the contract for when **try_push** retries and when it returns.

die Reihe

-n T0

turn, order — an der Reihe sein = to be one's turn

*Das Lock erledigt, wie sich herausstellt, zwei Jobs — **Reihen** vergeben und Daten übergeben —, und die beiden zu trennen ist das ganze Design: ein Array, zwei Ticket-Zähler und eine Maske.*

The lock **turns** out to be doing two jobs, allocating turns and handing off data, and separating them is the whole design: an array, two ticket counters, and a mask.

die Reihenfolge

-n T2

order, sequence of events

*Die Menge der tatsächlich geschriebenen Slots ist {1}. — Kein einzelner Zähler kann „Slot 1 ist geschrieben, Slot 0 nicht“ ausdrücken, denn Claims werden der Reihe nach vergeben, und Writes werden in der **Reihenfolge** fertig, die dem Scheduler gerade gefällt.*

The set of slots actually written is {1}. — No single counter can express "slot 1 is written, slot 0 isn't", because claims are handed out in **order** and writes complete in whatever order the scheduler likes.

der Rest

-e T1

the rest, remaining threads

*Die Garantie, die wir wollen: Zu jedem Zeitpunkt macht *irgendein* Thread Fortschritt, und ein Thread, den das OS mitten in einer Operation unterbricht — und der dabei nichts hält — kann den **Rest** nicht einfrieren. - Viele-zu-viele, und nicht-blockierend. N Producer, M Consumer, keine festen Rollen.*

The guarantee we want is that at any moment *some* thread is making progress, and a thread the OS preempts mid-operation — holding nothing — cannot freeze the **rest**. - Many-to-many, and non-blocking. N producers, M consumers, no fixed roles.

der Ring

-e T0

ring buffer

*Diese Serie entwirft die Queue, die das nicht tut: einen **Ring** — beschränkt, allokatonsfrei, lock-free —, den sich N Producer und M Consumer teilen und bei dem nur noch die Übergabe selbst etwas kostet.*

This series designs the queue that doesn't: a bounded, allocation-free, lock-free **ring** shared by N producers and M consumers, where the only cost is the hand-off itself.

der Ringpuffer

- T0

circular ring buffer

*- MPMC-Ring — ein beschränkter **Ringpuffer**, in den mehrere Producer pushen und aus dem mehrere Consumer poppen, lock-free und allokatonsfrei. - lock-free — irgendein Thread kommt immer voran; kein Thread hält ein Lock, auf das andere warten.*

- MPMC ring — a bounded **circular buffer** that multiple producers push and multiple consumers pop, lock-free and allocation-free. - lock-free — some thread always makes progress; no thread holds a lock others wait on.

die Rolle

-n T1

role (e.g. fixed thread roles)

*Die Garantie, die wir wollen: Zu jedem Zeitpunkt macht *irgendein* Thread Fortschritt, und ein Thread, den das OS mitten in einer Operation unterbricht — und der dabei nichts hält — kann den Rest nicht einfrieren. - Viele-zu-viele, und nicht-blockierend. N Producer, M Consumer, keine festen **Rollen**.*

The guarantee we want is that at any moment *some* thread is making progress, and a thread the OS preempts mid-operation — holding nothing — cannot freeze the rest. - Many-to-many, and non-blocking. N producers, M consumers, no fixed **roles**.

die Runde

-n T0

lap, cycle across buffer

*Ein Boolean pro Slot kann das nicht reparieren — ein Boolean hat kein Gedächtnis, und Slots werden über **Runden** hinweg wiederverwendet.*

A per-slot boolean can't fix it — a boolean has no memory, and slots are reused across **laps**.

die Rundengrenze

-n T4

wrap-around boundary of the ring

*Fünf gepusht und vier gepoppt über die **Rundengrenze** eines Zwei-Slot-Rings hinweg — genau fünf.*

Five pushed and four popped across a **wrap** of a two-slot ring — exactly five.

die Sackgasse

-n T2

dead end

*Dieselbe **Sackgasse**, in die der SeqLock in [seinem Teil 2] (../seqlock/de/02_the_bet.md) geraten ist: Ein Boolean hat kein Gedächtnis, und ein Slot, der über Runden hinweg wiederverwendet wird, braucht eines.*

Same **dead end** the SeqLock hit in [its Part 2](../seqlock/en/02_the_bet.md): a boolean has no memory, and a slot that's reused across laps needs one.

das Scheitern

T0

failure (of an idea / approach)

*Es ist eine Design-Untersuchung, kein Tutorial. — Jede Entscheidung ist erzwungen — durch einen Use Case, durch das **Scheitern** der einfacheren Variante oder durch ein Verifikationswerkzeug.*

It's a design investigation, not a tutorial. — Every decision is forced — by a use case, by the simpler version **failing**, or by a verification tool.

die Schleife

-n T2

loop (spin loop or retry loop)

*CAS verloren. Ein anderer Thread hat Ticket pos genommen. — Das nächste Ticket ist frei, niemand muss etwas tun, damit ich es bekomme, also läuft die **Schleife** mit dem neuen **tail** weiter.*

Lost the CAS. Another thread took ticket pos. — The next ticket is free, nobody has to do anything for me to get it, so the **loop** continues with the new **tail**.

die Schranke

-n T4

bound, limit

*## loom: eine Explosion statt einer Antwort — loom führt einen kleinen Test unter ***jeder*** Verschränkung aus, die das Memory-Modell erlaubt, innerhalb einer **Schranke**.*

loom: an explosion instead of an answer — loom runs a small test under ***every*** interleaving the memory model permits, within a **bound**.

der Schreiber

- T4

writer thread

*Das Siebenfache der Ein-Thread-Kosten, für eine Übergabe, die laut Teil 1 ***keine*** logische Contention hat: ein **Schreiber** pro Slot, ein Leser pro Slot und die beiden Zähler von verschiedenen Threads angefasst.*

Seven times the single-thread cost, for a hand-off that Part 1 established has ***no*** logical contention: one **writer** per slot, one reader per slot, and the two counters touched by different threads.

der Schreibtisch

-e T1

desk — am Schreibtisch = at your desk

*Ein Array, zwei Zähler, eine Maske. — Bau Push und Pop auf die naheliegende Art — Zähler erhöhen, Slot anfassen — und es besteht jeden Test, den du am **Schreibtisch** schreibst: ein Thread pusht, einer poppt, jede Zählung geht auf.*

An array, two counters, a mask. — Build the push and pop the obvious way — bump the counter, touch the slot — and it passes every test you write at your **desk**: one thread pushing, one popping, every count tallying.

der Schritt

-e T3

step (in reasoning or optimization)

*Das Argument hat drei **Schritte**: finden, wo Daten von einem Thread zum anderen wandern; entscheiden, welches Atomic jeden dieser Übergänge trägt; und dann **Release** und **Acquire** auf die Seite dieses Atomics legen, die die Daten tatsächlich abdeckt.*

The argument has three **steps**: find where data crosses threads, decide which atomic carries each crossing, then put **Release** and **Acquire** on the side of that atomic that actually covers the data.

die Sekunde

-n T0

second (unit of time)

Einen lock-freien MPMC-Ring entwerfen — Am schreibenden Ende von allem, was schnell ist — eine Matching Engine, die Orders von einem Dutzend Feed-Threads entgegennimmt, ein Write-Ahead-Log, das viele Writer in einen einzigen Commit bündelt —, sitzt eine Queue, in die viele Threads pushen und aus der viele Threads poppen, millionenfach pro Sekunde.

Designing a lock-free MPMC ring — At the write end of anything fast — a matching engine taking orders off a dozen feed threads, a write-ahead log fanning writers into one commit — there's a queue that many threads push and many threads pop, millions of times a **second**.

die Sendeseite

-n T0

sending side (of hand-off)

Verschiedene Operationen, verschiedene Atomics. - Übergabe (hand-off) — ein Thread ist mit ein paar Bytes fertig und signalisiert es, ein anderer holt sie ab: eine Sendeseite und eine Empfangsseite.

Different operations, different atomics. - hand-off — one thread finishing with some bytes and signaling, another picking them up: a **sending side** and a receiving side.

die Sequenz

-en T0

sequence counter (per-slot sequence)

Er zählt nur hoch: pos (Producer darf schreiben), pos + 1 (Consumer darf lesen), pos + capacity (der Producer der nächsten Runde darf schreiben). - claim vs. publish — ein Ticket auf dem Zähler ziehen vs. den payload über `seq` sichtbar machen.

It only increases: pos (producer may write), pos + 1 (consumer may read), pos + capacity (next lap's producer may write). - claim vs publish — taking a ticket on the counter vs making the payload visible through `seq`.

die Sequenznummer

-n T0

sequence number

Nicht wait-free: Ein einzelner Thread kann es erneut versuchen müssen. - Ticket — der Wert, den ein Thread von tail (Producer) oder head (Consumer) zieht; ticket & (capacity - 1) ist sein Slot. - Slot / Zelle — ein Element des Puffers: ein payload und seine Sequenznummer. - seq — der Zähler pro Slot, der sagt, für welches Ticket der Slot bereit ist.

Not wait-free: an individual thread may retry. - ticket — the value a thread takes from tail (producer) or head (consumer); ticket & (capacity - 1) is its slot. - slot / cell — one element of the buffer: a payload and its **sequence number**. - seq — the per-slot counter that says which ticket the slot is ready for.

die Serie

-n T0

blog series

Diese Serie entwirft die Queue, die das nicht tut: einen Ring — beschränkt, allokatonsfrei, lock-free —, den sich N Producer und M Consumer teilen und bei dem nur noch die Übergabe selbst etwas kostet.

This **series** designs the queue that doesn't: a bounded, allocation-free, lock-free ring shared by N producers and M consumers, where the only cost is the hand-off itself.

das **Signal**

-e T2

signal, coordination flag

*Der Consumer hat nichts falsch gemacht. — Er hat dem einzigen **Signal** vertraut, das er hatte, und das Signal wurde beim Claim gesetzt, nicht beim Write.*

The consumer did nothing wrong. — It trusted the only **signal** it had, and the signal was raised at claim time, not at write time.

die **Sorge**

-n T2

worry, concern

*Das weckt eine naheliegende **Sorge** — der Slot könnte sich zwischen Gate und Claim ändern.*

That raises an obvious **worry** — the slot could change between the gate and the claim.

der **Speicher**

- T2

memory

*Die Logik stimmt. — Falsch ist das eine, was die Logik nicht kontrolliert: die Reihenfolge, in der zwei Cores Änderungen im **Speicher** sehen.*

The logic is right. — What's wrong is the one thing the logic doesn't control: the order in which two cores see **memory** change.

das **Speicherwort**

“-er T1

memory word (single atomic address)

*Ein Allocator-Aufruf bedeutet an einem guten Tag hunderte Nanosekunden und an einem schlechten ein prozessweites Lock; so oder so passt er nicht in eine Übergabe, die in unter einer Mikrosekunde fertig sein muss. - Lock-free. Kein einzelnes **Speicherwort**, das sich jeder Thread erst holen muss.*

An allocator call is hundreds of nanoseconds on a good day and a process-wide lock on a bad one; either way it doesn't fit inside a hand-off that has to complete in under a microsecond. - Lock-free. No single **word** every thread must acquire.

das **Spiegelbild**

-er T1

mirror image, dual

*Gib den Producern eine Möglichkeit, die Reihen unter sich zu verteilen, ohne je einen Consumer zu berühren; gib den Consumern das **Spiegelbild** davon; mach die Übergabe des payloads zu einer Sache pro Slot, zwischen den zwei Threads, die sich diesen Slot teilen.*

Give producers a way to allocate turns among themselves that never touches a consumer; give consumers the **mirror** of it; let the payload hand-off be a per-slot matter between the two threads that share that slot.

die **SPSC-Übergabe**

-n T0

single-producer single-consumer hand-off

*Die Arbeitsteilung zwischen den drei Instrumenten, ein Drop, das Miris Leak-Checker absegnet, und der Benchmark: head und tail mit Padding auf getrennte cache lines zu legen, bringt die **SPSC-Übergabe** von ~51 ns auf ~9 ns.*

The division of labor between the three instruments, a Drop that Miri's leak checker approves, and the benchmark: padding head and tail onto separate cache lines takes the **SPSC hand-off** from ~51 ns to ~9 ns.

der **Staffelstab**

-e T2

baton (passed from producer to consumer)

Die Sequenz: ein **Staffelstab** mit einer Nummer darauf — Der Fix stammt von Dmitry Vyukov, und er besteht aus einem Integer pro Slot: **seq**.
The sequence: a **baton** with a number on it — The fix is Dmitry Vyukov's, and it's one integer per slot: **seq**.

die **Stelle**

-n T0

place, location, code position

Die Orderings werden hergeleitet, nicht geraten: die zwei **Stellen** finden, an denen Daten den Thread wechseln, und dann bemerken, dass der payload nur über **seq** veröffentlicht werden kann, nie über den Zähler — ein **Release** legt einen Boden unter alles, was **davor** kommt, und der Claim kommt vor dem Write.

The orderings are derived, not guessed: find the two **places** data crosses threads, then notice the payload can only be published through **seq**, never through the counter — a **Release** floors what comes **before** it, and the claim comes before the write.

die **Stufe**

-n T3

rung, level (e.g. SeqCst level)

Es gibt eine dritte **Stufe**, **SeqCst**, für den Fall, dass die Frage nicht lautet „sieht dieser Thread die Daten jenes Threads“, sondern „sind sich alle Threads über die Reihenfolge von Stores auf **verschiedene** Atomics einig“.

There's a third **rung**, **SeqCst**, for when the question isn't "does this thread see that thread's data" but "do all threads agree on the order of stores to **different** atomics".

das **Symptom**

-e T3

symptom (of a bug)

Auf dem M2 ist das **Symptom** ein Consumer, der gelegentlich den vorherigen Inhalt des Slots liest; in Rusts Memory-Modell ist es undefiniertes Verhalten, noch bevor es ein falscher Wert ist.

On the M2 the **symptom** is a consumer that occasionally reads the slot's previous contents; in Rust's memory model it's undefined behavior before it's a wrong value.

das **System**

-e T1

system

Mitten im heißesten Pfad des **Systems** sitzt eine Queue, und das Erste, wonach jeder greift, ist **Mutex<VecDeque<T>>**.

In the middle of the hottest path in the **system** sits a queue, and the first thing anyone reaches for is **Mutex<VecDeque<T>>**.

die **Tatsache**

-n T2

fact (immutable reality)

Zwei Producer machen es strukturell — Mit zwei Producern ist die Lücke kein Wettlauf mehr, sondern eine **Tatsache** über Zähler:

Two producers make it structural — With two producers the gap stops being a race and becomes a **fact** about counters:

der Teil

-e T0

part (of the series)

*Diese Serie entwirft die Queue, die das nicht tut: einen Ring — beschränkt, allokatonsfrei, lock-free —, den sich N Producer und M Consumer teilen und bei dem nur noch die Übergabe selbst etwas kostet. Vier **Teile**, zehn bis fünfzehn Minuten pro Teil.*

This series designs the queue that doesn't: a bounded, allocation-free, lock-free ring shared by N producers and M consumers, where the only cost is the hand-off itself. Four **parts**, ten to fifteen minutes each.

der Topf

“-e T1

pot — in einen Topf werfen = to conflate

*Das sind zwei verschiedene Aufgaben, und ein Lock wirft sie in einen **Topf**. — Schau hin, wo der echte Konflikt sitzt.*

Those are two different jobs, and a lock **conflates** them. — Look at where the genuine conflict lives.

der Typ

-en T4

type (data type)

*Data Race, die zwei Threads, der **Typ** an der Adresse: **MaybeUninit<usize>**. — Diese Präzision ist der Grund, warum Miri zuerst drankommt.*

Data race, the two threads, the **type** at the address: `MaybeUninit<usize>`. — That precision is why Miri comes first.

der Umfang

“-e T0

scope

*## **Umfang** — Ein generischer, beschränkter `MpmcRing<T>` — der Vyukov-Algorithmus, so wie man ihn in eine Concurrency-Crate legen würde.*

*## **Scope** — A generic bounded `MpmcRing<T>` — the Vyukov algorithm, as you'd put it in a concurrency crate.*

der Umweg

-e T2

detour — mit Umwegen = with extra steps

*Bleibt das Primitiv hier in einer Spin-Schleife, ist es zu einer blockierenden Queue mit **Umwegen** geworden, und zu einer schlechteren: Ein Producer, der mitten im Write vom OS verdrängt wird, hält jeden Consumer hinter sich in einer Spin-Schleife fest.*

If the primitive spins here, it has become a blocking queue with **extra steps**, and a worse one: a producer preempted mid-write holds every consumer behind it in a spin.

die Unterbrechung

-en T4

interruption, preemption

*Was Miri nicht tut, ist aufzählen. — Es führt die Verschränkungen aus, die es nun mal ausführt, mit ein paar eingestreuten **Unterbrechungen**.*

What Miri does not do is enumerate. — It runs the interleavings it happens to run, with some **preemption** thrown in.

die Untergrenze

-n T4

lower bound, floor

*Die Single-Producer/Single-Consumer-Übergabe geht von ~51 ns auf ~8,8 ns — 5,8×, und liegt jetzt keine zwei Nanosekunden über der Ein-Thread-**Untergrenze**.*

The single-producer/single-consumer hand-off goes from ~51 ns to ~8.8 ns — 5.8×, and now within two nanoseconds of the single-thread **floor**.

der Unterschied

-e T3

difference

Warum kein Fence — der **Unterschied** zum SeqLock — Der [SeqLock] ([../seqlock/de/03_memory_ordering.md](#)) brauchte zusätzlich zu seinen Orderings zwei eigenständige fences.

Why no fence — the **difference** from a SeqLock — The [SeqLock] ([../seqlock/en/03_memory_ordering.md](#)) needed two standalone fences on top of its orderings.

die Untersuchung

-en T0

investigation, analysis

*Es ist eine Design-**Untersuchung**, kein Tutorial. — Jede Entscheidung ist erzwungen — durch einen Use Case, durch das Scheitern der einfacheren Variante oder durch ein Verifikationswerkzeug.*

It's a design **investigation**, not a tutorial. — Every decision is forced — by a use case, by the simpler version failing, or by a verification tool.

das Urteil

-e T4

verdict, judgment

*Das ist ein **Urteil**, nur ein indirektes. — Die Gate-Schleife aus Teil 2 — `diff > 0`, Zähler neu laden, erneut versuchen — geht davon aus, dass das Neuladen irgendwann einen frischen Wert sieht.*

This is a **verdict**, just an indirect one. — The gate loop from Part 2 — `diff > 0`, reload the counter, retry — assumes the reload eventually sees a fresh value.

die Variante

-n T0

variant, alternative implementation

*Es ist eine Design-Untersuchung, kein Tutorial. — Jede Entscheidung ist erzwungen — durch einen Use Case, durch das Scheitern der einfacheren **Variante** oder durch ein Verifikationswerkzeug.*

It's a design investigation, not a tutorial. — Every decision is forced — by a use case, by the simpler **version** failing, or by a verification tool.

der Vergleich

-e T2

comparison

Das Drei-Wege-Gate — Der **Vergleich** ist eine vorzeichenbehaftete Differenz, `seq.wrapping_sub(pos) as isize`, damit er aussagekräftig bleibt, falls die Zähler je überlaufen.

The three-way gate — The **comparison** is a signed difference, `seq.wrapping_sub(pos) as isize`, so it stays meaningful if the counters ever wrap.

das Verhalten

T3

behavior (e.g. undefined behavior)

*Auf dem M2 ist das Symptom ein Consumer, der gelegentlich den vorherigen Inhalt des Slots liest; in Rusts Memory-Modell ist es undefiniertes **Verhalten**, noch bevor es ein falscher Wert ist.*

On the M2 the symptom is a consumer that occasionally reads the slot's previous contents; in Rust's memory model it's undefined **behavior** before it's a wrong value.

das

Verifikationswerkzeug

-e T0

verification tool

*Es ist eine Design-Untersuchung, kein Tutorial. — Jede Entscheidung ist erzwungen — durch einen Use Case, durch das Scheitern der einfacheren Variante oder durch ein **Verifikationswerkzeug**.*

It's a design investigation, not a tutorial. — Every decision is forced — by a use case, by the simpler version failing, or by a **verification tool**.

der Verifizierer

- T0

verifier (tool checking correctness)

*Nach Teil 2 aufzuhören lässt dich mit einem Ring zurück, der logisch korrekt ist; die Teile 3 und 4 sind der Ort, an dem er auf die Hardware und die **Verifizierer** trifft und an dem die Fehler wohnen, die Tests bestehen.*

Stopping after Part 2 leaves you with a ring that's logically correct; Parts 3 and 4 are where it meets the hardware and the **verifiers**, and where the failures that pass tests live.

die Verschränkung

-en T0

interleaving (concurrency schedule)

*Zwei pro Slot: A (publish, Producer → Consumer) und B (vacate, Consumer → Producer der nächsten Runde). - compare_exchange (CAS) — atomares Read-Modify-Write, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so beansprucht genau ein Thread ein Ticket. - Relaxed / Acquire / Release — Relaxed ist atomar ohne Ordering; Release auf einem Store ist ein Boden unter allem, was davor kommt; Acquire auf einem Load ist ein Dach über allem, was danach kommt; ein Acquire-Load, der einen Release-Store beobachtet, verbindet die beiden. - happens-before — die threadübergreifende Garantie, die ein Release/Acquire-Paar aufbaut; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - False Sharing — zwei unabhängige Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores invalidiert; behoben durch CachePadded. - loom — ein Model Checker, der einen kleinen Test unter jeder **Verschränkung** ausführt; blind für Übergaben, die über einen CAS laufen. - Miri — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB meldet — Data Races, uninitialisierte Reads, Leaks.*

Two per slot: A (publish, producer → consumer) and B (vacate, consumer → next lap's producer). - compare_exchange (CAS) — atomic read-modify-write that succeeds only if the value is what you expected; how exactly one thread claims a ticket. - Relaxed / Acquire / Release — Relaxed is atomic with no ordering; Release on a store floors what comes before it; Acquire on a load roofs what comes after it; an Acquire load that observes a Release store joins the two. - happens-before — the cross-thread guarantee a Release/Acquire pair builds; without one, a plain write on one thread and a plain read on another are a data race. - false sharing — two independent values on one cache line, each write invalidating the other core's copy; fixed by CachePadded. - loom — a model checker that runs a small test under every **interleaving**; blind to hand-offs carried by a CAS. - Miri — an interpreter that runs Rust against the memory model and reports UB — data races, uninitialized reads, leaks.

der Vertrag

--e T0

contract (API contract / invariants)

*Ein Drei-Wege-Gate, die **publish**- und **vacate**-Regeln und der **Vertrag** darüber, wann **try_push** erneut versucht und wann es zurückkehrt.*

A three-way gate, the **publish** and **vacate** rules, and the **contract** for when **try_push** retries and when it returns.

die Veröffentlichung

-en T3

publishing, publication

*> Der Claim kann die Daten nicht veröffentlichen, denn zum Zeitpunkt des Claims > existieren die Daten noch nicht. Die **Veröffentlichung** gehört dem letzten Store nach > dem Write — **seq**.*

> The claim can't publish the data, because at claim time the data doesn't exist > yet. **Publication** belongs to the last store after the write — **seq**.

die Wand

--e T3

wall (two-sided synchronization barrier)

*Kein Ordering auf diesen Ops konnte die Daten erreichen. — Ein Fence — eine zweiseitige **Wand**, die du selbst positionierst, kein an eine Op geklebtes Gate — war das einzige Werkzeug.*

No ordering on those ops could reach the data. — A fence — a two-sided **wall** you position yourself, not a gate glued to an op — was the only tool.

das Werkzeug

-e T0

tool, instrument

*[Teil 4 — Der Beweis: zwei **Werkzeuge** und ein lügendes Grün.] (04_proving_it.md) Miri nennt den Race beim Namen; loom, mit dem **Relaxed-Ring** gefüttert, explodiert, statt zu antworten.*

[Part 4 — Proving it: two **tools**, and a lying green.](04_proving_it.md) Miri names the race; loom, given the **Relaxed** ring, explodes instead of answering.

der Wert

-e T0

value (data or ticket value)

*Mit den Orderings drin werden beide grün — und das Grün von loom ist weniger **wert**, als es aussieht: Es modelliert einen Compare-and-Swap stärker als die Hardware und winkt eine kaputte Übergabe durch, die über einen läuft — gezeigt, indem man eine einzige Zeile ändert.*

With the orderings in, both go green — and loom's green is worth less than it looks: it models a compare-and-swap more strongly than the hardware, and passes a broken hand-off that runs through one, shown by changing one line.

der Wettlauf

--e T2

race (race condition / competition)

*## Zwei Producer machen es strukturell — Mit zwei Producern ist die Lücke kein **Wettlauf** mehr, sondern eine Tatsache über Zähler:*

Two producers make it structural — With two producers the gap stops being a **race** and becomes a fact about counters:

das Wort

--er T1

word (memory address / scalar)

*Ein einzelnes Lock lässt jeden Thread jedes Mal auf beide Aufgaben warten: Ein Producer blockiert einen Consumer, der einen völlig anderen Slot will, weil sie sich das Lock-**Wort** teilen.*

A single lock makes every thread wait for both jobs, every time: a producer blocks a consumer that wants a completely different slot, because they share the lock **word**.

die Zahl

-en T0

number, measured benchmark number

Die **Zahlen** in Teil 4 stammen aus den *criterion*-Benches der Crate, und jede Aussage über Korrektheit ist eine, die loom oder Miri tatsächlich geprüft hat.

The **numbers** in Part 4 are from the crate's *criterion* benches, and every claim about correctness is one loom or Miri actually checked.

die Zeile

-n T0

line of code

Mit den Orderings drin werden beide grün — und das Grün von loom ist weniger wert, als es aussieht: Es modelliert einen Compare-and-Swap stärker als die Hardware und winkt eine kaputte Übergabe durch, die über einen läuft — gezeigt, indem man eine einzige **Zeile** ändert.

With the orderings in, both go green — and loom's green is worth less than it looks: it models a compare-and-swap more strongly than the hardware, and passes a broken hand-off that runs through one, shown by changing one line.

die Zelle

-n T0

cell, buffer slot

Nicht wait-free: Ein einzelner Thread kann es erneut versuchen müssen. - Ticket — der Wert, den ein Thread von *tail* (Producer) oder *head* (Consumer) zieht; *ticket & (capacity - 1)* ist sein Slot. - Slot / **Zelle** — ein Element des Puffers: ein payload und seine Sequenznummer. - *seq* — der Zähler pro Slot, der sagt, für welches Ticket der Slot bereit ist.

Not wait-free: an individual thread may retry. - ticket — the value a thread takes from *tail* (producer) or *head* (consumer); *ticket & (capacity - 1)* is its slot. - slot / **cell** — one element of the buffer: a payload and its sequence number. - *seq* — the per-slot counter that says which ticket the slot is ready for.

der Zustand

“-e T2

state (slot state)

> *head* und *tail* verteilen, wer an der Reihe ist. — Den **Zustand** eines Slots können > sie nicht beschreiben.

> *head* and *tail* allocate turns. — They cannot describe the **state** of a slot.

die Zweierpotenz

-en T1

power of two

Dieses *&* ist ein Modulo, aber nur, wenn *capacity* eine **Zweierpotenz** ist — dann ist *capacity - 1* eine Folge niedriger 1-Bits, und die Maske behält genau diese.

That *&* is a modulo, but only when *capacity* is a **power of two** — then *capacity - 1* is a run of low 1 bits and the mask keeps exactly them.

der Zweig

-e T3

branch (e.g. Ok branch)

Bei (3) zucken die Leute zusammen. — Der payload-Zugriff sitzt im Ok-**Zweig** des CAS — sollte das, was entscheidet, ob du den Slot anfassen darfst, nicht auch das Anfassen ordnen?

The one people flinch at is (3). — The payload access sits inside the Ok **branch** of the CAS — surely the thing that decides whether you may touch the slot should also order the touch?

der Zähler

- T0

counter (sequence or ticket counter)

Das Lock erledigt, wie sich herausstellt, zwei Jobs — Reihen vergeben und Daten übergeben —, und die beiden zu trennen ist das ganze Design: ein Array, zwei Ticket-Zähler und eine Maske.

The lock turns out to be doing two jobs, allocating turns and handing off data, and separating them is the whole design: an array, two ticket **counters**, and a mask.

die Zählung

-en T1

count, tally

Ein Array, zwei Zähler, eine Maske. — Bau Push und Pop auf die naheliegende Art — Zähler erhöhen, Slot anfassen — und es besteht jeden Test, den du am Schreibtisch schreibst: ein Thread pusht, einer poppt, jede Zählung geht auf.

An array, two counters, a mask. — Build the push and pop the obvious way — bump the counter, touch the slot — and it passes every test you write at your desk: one thread pushing, one popping, every **count** tallying.

die Übergabe

-n T0

hand-off (cross-thread data transfer)

Diese Serie entwirft die Queue, die das nicht tut: einen Ring — beschränkt, alloktationsfrei, lock-free —, den sich N Producer und M Consumer teilen und bei dem nur noch die Übergabe selbst etwas kostet.

This series designs the queue that doesn't: a bounded, allocation-free, lock-free ring shared by N producers and M consumers, where the only cost is the **hand-off** itself.

der Übergang

-e T3

transition, crossing point

Das Argument hat drei Schritte: finden, wo Daten von einem Thread zum anderen wandern; entscheiden, welches Atomic jeden dieser Übergänge trägt; und dann Release und Acquire auf die Seite dieses Atomics legen, die die Daten tatsächlich abdeckt.

The argument has three steps: find where data crosses threads, decide which atomic carries each **crossing**, then put Release and Acquire on the side of that atomic that actually covers the data.

Verben verbs

192

abbilden (auf)

bildet ab · bildete ab
· abgebildet, trennb.

T1

to map (onto)

Keiner der beiden Zähler nimmt je ab, also laufen sie fast sofort über die Array-Länge hinaus, und ein Ticket wird auf einen physischen Slot abgebildet, indem man es zurückfaltet: `slot = ticket & (capacity - 1)`.

Neither counter ever decreases, so they run past the array length almost immediately, and a ticket **maps** to a physical slot by folding it back: `slot = ticket & (capacity - 1)`.

abdecken

deckt ab · deckte ab ·
abgedeckt, trennb.

T3

to cover (scope/safety)

*Das Argument hat drei Schritte: finden, wo Daten von einem Thread zum anderen wandern; entscheiden, welches Atomic jeden dieser Übergänge trägt; und dann Release und Acquire auf die Seite dieses Atomics legen, die die Daten tatsächlich **abdeckt**.*

The argument has three steps: find where data crosses threads, decide which atomic carries each crossing, then put Release and Acquire on the side of that atomic that actually **covers** the data.

abfangen

fängt ab · fing ab ·
abgefangen, trennb.

T4

to catch, intercept (bugs/leaks)

*Diese Präzision ist der Grund, warum Miri zuerst drankommt. — Es **fängt** auch die anderen Dinge, die eine Queue mit Speicher falsch machen kann — einen uninitialisierten Read, ein Leak, ein Double Free — und Spin-Schleifen stören es nicht.*

That precision is why Miri comes first. — It also **catches** the other things a queue can do wrong with memory — an uninitialized read, a leak, a double free — and it's fine with spin loops.

abholen

holt ab · holte ab ·
abgeholt, trennb.

T0

to pick up, collect

*Verschiedene Operationen, verschiedene Atomics. - Übergabe (hand-off) — ein Thread ist mit ein paar Bytes fertig und signalisiert es, ein anderer **holt sie ab**: eine Sendeseite und eine Empfangsseite.*

Different operations, different atomics. - hand-off — one thread finishing with some bytes and signaling, another **picking them up**: a sending side and a receiving side.

abnehmen

nimmt ab · nahm ab ·
abgenommen, trennb.

T1

to decrease (counter value)

*Keiner der beiden Zähler **nimmt je ab**, also laufen sie fast sofort über die Array-Länge hinaus, und ein Ticket wird auf einen physischen Slot abgebildet, indem man es zurückfaltet: $\text{slot} = \text{ticket} \& (\text{capacity} - 1)$.*

Neither counter ever **decreases**, so they run past the array length almost immediately, and a ticket maps to a physical slot by folding it back: $\text{slot} = \text{ticket} \& (\text{capacity} - 1)$.

absegnen

segnet ab · segnete ab ·
abgesegnet, trennb.

T0

to approve, bless

*Die Arbeitsteilung zwischen den drei Instrumenten, ein Drop, das Miris Leak-Checker **absegnet**, und der Benchmark: head und tail mit Padding auf getrennte cache lines zu legen, bringt die SPSC-Übergabe von ~51 ns auf ~9 ns.*

The division of labor between the three instruments, a Drop that Miri's leak checker **approves**, and the benchmark: padding head and tail onto separate cache lines takes the SPSC hand-off from ~51 ns to ~9 ns.

anfassen

fasst an · fasste an ·
angefasst, trennb.

T1

to touch (shared memory/bytes)

*Zwei Consumer: dasselbe. — Die einzige Stelle, an der zwei Threads dieselben Bytes **anfassen**, liegt zwischen dem Producer, der Slot *k* füllt, und dem Consumer, der Slot *k* später leert — ein Thread auf jeder Seite, und für jeden Slot ein anderes Paar.*

Two consumers, the same. — The only place two threads **touch** the same bytes is between the producer that fills slot *k* and the consumer that later empties slot *k* — one thread on each side, and a different pair for every slot.

ansteuern

steuert an · steuerte
an · angesteuert,
trennb.

T1

to head for, target (a slot)

*Ein Mutex *pro Slot* verkleinert das Lock, ohne es loszuwerden, und bringt ein Problem mit, das das Design vorher nicht hatte: entscheiden, welchen Slot jeder Thread **ansteuern** soll.*

A mutex *per slot* shrinks the lock without removing it, and adds a problem the design didn't have before: deciding which slot each thread should **head for**.

antworten

antwortet · antwortete
· geantwortet

T0

to answer, respond

*[Teil 4 — Der Beweis: zwei Werkzeuge und ein lügendes Grün.]
(04_proving_it.md) Miri nennt den Race beim Namen; loom, mit dem Relaxed-Ring gefüttert, explodiert, statt zu **antworten**.*

[Part 4 — Proving it: two tools, and a lying green.](04_proving_it.md) Miri names the race; loom, given the Relaxed ring, explodes instead of **answering**.

aufbauen

baut auf · baute auf ·
aufgebaut, trennb.

T0

to build up, establish (happens-before)

*Zwei pro Slot: A (publish, Producer → Consumer) und B (vacate, Consumer → Producer der nächsten Runde). - compare_exchange (CAS) — atomares Read-Modify-Write, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so beansprucht genau ein Thread ein Ticket. - Relaxed / Acquire / Release — Relaxed ist atomar ohne Ordering; Release auf einem Store ist ein Boden unter allem, was davor kommt; Acquire auf einem Load ist ein Dach über allem, was danach kommt; ein Acquire-Load, der einen Release-Store beobachtet, verbindet die beiden. - happens-before — die threadübergreifende Garantie, die ein Release/Acquire-Paar **aufbaut**; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - False Sharing — zwei unabhängige Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores invalidiert; behoben durch CachePadded. - loom — ein Model Checker, der einen kleinen Test unter jeder Verschränkung ausführt; blind für Übergaben, die über einen CAS laufen. - Miri — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB meldet — Data Races, uninitialisierte Reads, Leaks.*

Two per slot: A (publish, producer → consumer) and B (vacate, consumer → next lap's producer). - compare_exchange (CAS) — atomic read-modify-write that succeeds only if the value is what you expected; how exactly one thread claims a ticket. - Relaxed / Acquire / Release — Relaxed is atomic with no ordering; Release on a store floors what comes before it; Acquire on a load roofs what comes after it; an Acquire load that observes a Release store joins the two. - happens-before — the cross-thread guarantee a Release/Acquire pair **builds**; without one, a plain write on one thread and a plain read on another are a data race. - false sharing — two independent values on one cache line, each write invalidating the other core's copy; fixed by CachePadded. - loom — a model checker that runs a small test under every interleaving; blind to hand-offs carried by a CAS. - Miri — an interpreter that runs Rust against the memory model and reports UB — data races, uninitialized reads, leaks.

aufbrauchen

braucht auf · brauchte
auf · aufgebraucht,
trennb.

T4

to exhaust, use up

*Unter Relaxed sagt nichts, dass es das muss; ein veralteter Wert ist für immer legal, also ist der Pfad, auf dem die Schleife nie terminiert, ein echter Pfad, und loom folgt ihm, bis sein Branch-Budget **aufgebraucht** ist.*

Under Relaxed, nothing says it must; a stale value is legal forever, so the path where the loop never terminates is a real path, and loom follows it until its branch budget **runs out**.

aufgeben

gibt auf · gab auf ·
aufgegeben, trennb.

T1

to surrender, give up

*Und eine try_-API: try_push auf einem vollen Ring **gibt den Wert zurück**; try_pop auf einem leeren Ring liefert None.*

And a try_ API: try_push on a full ring hands the value back; try_pop on an empty ring returns None.

aufgehen

geht auf · ging auf ·
aufgegangen, trennb.

T1

to tally, work out

*Ein Array, zwei Zähler, eine Maske. — Bau Push und Pop auf die naheliegende Art — Zähler erhöhen, Slot anfassen — und es besteht jeden Test, den du am Schreibtisch schreibst: ein Thread pusht, einer poppt, jede Zählung **geht auf**.*

An array, two counters, a mask. — Build the push and pop the obvious way — bump the counter, touch the slot — and it passes every test you write at your desk: one thread pushing, one popping, every count **tallying**.

aufhören

hört auf · hörte auf ·
aufgehört, trennb.

T0

to stop, finish

*Der Reihe nach. — Jeder Teil beginnt dort, wo der vorige **aufgehört** hat, und schließt mit der Frage, die der nächste beantwortet.*

In order. — Each part opens where the previous one **stopped** and closes on the question the next one answers.

aufrufen

ruft auf · rief auf ·
aufgerufen, trennb.

T1

to call, invoke

*Die Queue parkt nie einen Thread. — Blockieren ist eine Policy, die der **Aufrufer** vielleicht will, und sie darf nicht fest ins Primitiv eingebaut sein.*

The queue never parks a thread. — Blocking is a policy the **caller** may want, and it must not be baked into the primitive.

aufsteigen

steigt auf · stieg auf ·
aufgestiegen, trennb.

T3

to rise (above an Acquire boundary)

*Über das, was danach kommt, sagt es nichts. — Acquire auf einem Load ist ein Dach über allem, was danach kommt: Nichts darunter darf über den Load **aufsteigen**.*

It says nothing about what comes after. — Acquire on a load is a roof over everything after it: nothing below may **rise** above the load.

auftauchen

taucht auf · tauchte
auf · aufgetaucht,
trennb.

T1

to surface, appear

*Diese Form **taucht überall auf**, wo schnelle Software ein schreibendes Ende hat. — Ein Write-Ahead-Log bündelt ein Dutzend Writer-Threads in einen Group Commit.*

The shape is **everywhere** fast software has a write end. — A write-ahead log fans a dozen writer threads into one group commit.

aufzählen

zählt auf · zählte auf ·
aufgezählt, trennb.

T4

to enumerate (interleavings)

*Was Miri nicht tut, ist **aufzählen**. — Es führt die Verschränkungen aus, die es nun mal ausführt, mit ein paar eingestreuten Unterbrechungen.*

What Miri does not do is **enumerate**. — It runs the interleavings it happens to run, with some preemption thrown in.

ausdrücken

drückt aus · drückte
aus · ausgedrückt,
trennb.

T2

to express, represent

*Die Menge der tatsächlich geschriebenen Slots ist {1}. — Kein einzelner Zähler kann „Slot 1 ist geschrieben, Slot 0 nicht“ **ausdrücken**, denn Claims werden der Reihe nach vergeben, und Writes werden in der Reihenfolge fertig, die dem Scheduler gerade gefällt.*

The set of slots actually written is {1}. — No single counter can **express** "slot 1 is written, slot 0 isn't", because claims are handed out in order and writes complete in whatever order the scheduler likes.

auseinanderhalten

hält auseinander ·
hielt auseinander ·
auseinandergehalten,
trennb.

T2

to distinguish, tell apart

written = false hieß „konsumiert“ *oder* „beansprucht, aber noch nicht geschrieben“, und P2 kann die beiden nicht **auseinanderhalten**.

written = false meant "consumed" *or* "claimed but not yet written", and P2 can't **tell which**.

ausführen

führt aus · führte aus
· ausgeführt, trennb.

T0

to execute, run

*Zwei pro Slot: A (publish, Producer → Consumer) und B (vacate, Consumer → Producer der nächsten Runde). - compare_exchange (CAS) — atomares Read-Modify-Write, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so beansprucht genau ein Thread ein Ticket. - Relaxed / Acquire / Release — Relaxed ist atomar ohne Ordering; Release auf einem Store ist ein Boden unter allem, was davor kommt; Acquire auf einem Load ist ein Dach über allem, was danach kommt; ein Acquire-Load, der einen Release-Store beobachtet, verbindet die beiden. - happens-before — die threadübergreifende Garantie, die ein Release/Acquire-Paar aufbaut; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - False Sharing — zwei unabhängige Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores invalidiert; behoben durch CachePadded. - loom — ein Model Checker, der einen kleinen Test unter jeder Verschränkung **ausführt**; blind für Übergaben, die über einen CAS laufen. - Miri — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB meldet — Data Races, uninitialisierte Reads, Leaks.*

Two per slot: A (publish, producer → consumer) and B (vacate, consumer → next lap's producer). - compare_exchange (CAS) — atomic read-modify-write that succeeds only if the value is what you expected; how exactly one thread claims a ticket. - Relaxed / Acquire / Release — Relaxed is atomic with no ordering; Release on a store floors what comes before it; Acquire on a load roofs what comes after it; an Acquire load that observes a Release store joins the two. - happens-before — the cross-thread guarantee a Release/Acquire pair builds; without one, a plain write on one thread and a plain read on another are a data race. - false sharing — two independent values on one cache line, each write invalidating the other core's copy; fixed by CachePadded. - loom — a model checker that **runs** a small test under every interleaving; blind to hand-offs carried by a CAS. - Miri — an interpreter that runs Rust against the memory model and reports UB — data races, uninitialized reads, leaks.

ausliefern

liefert aus · lieferte
aus · ausgeliefert,
trennb.

T1

to ship (code into production)

*## Die naheliegenden Queues, je eine Randbedingung verletzt — Mutex<VecDeque<T>> ist das, was der meiste Code **ausliefert**, und es funktioniert.*

*## The obvious queues, one constraint each — Mutex<VecDeque<T>> is what most code **ships**, and it works.*

aussehen

sieht aus · sah aus ·
ausgesehen, trennb.

T0

to look (like), appear

*Mit den Orderings drin werden beide grün — und das Grün von loom ist weniger wert, als es **aussieht**: Es modelliert einen Compare-and-Swap stärker als die Hardware und winkt eine kaputte Übergabe durch, die über einen läuft — gezeigt, indem man eine einzige Zeile ändert.*

With the orderings in, both go green — and loom's green is worth less than it

looks: it models a compare-and-swap more strongly than the hardware, and passes a broken hand-off that runs through one, shown by changing one line.

aussprechen

spricht aus · sprach
aus · ausgesprochen,
trennb.

T2

to state clearly, articulate

*Eine Konsequenz, die man **aussprechen** sollte: `try_pop` gibt `None` zurück, wenn das Element **am Head** nicht veröffentlicht ist, selbst wenn spätere Slots es sind.*

One consequence worth **stating**: `try_pop` returns `None` when the item **at the head** isn't published, even if later slots are.

beanspruchen

beansprucht ·
beanspruchte ·
beansprucht

T0

to claim (ticket / turn)

*Zwei pro Slot: A (`publish`, `Producer` → `Consumer`) und B (`vacate`, `Consumer` → `Producer` der nächsten Runde). - `compare_exchange` (CAS) — atomares Read-Modify-Write, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so **beansprucht** genau ein Thread ein Ticket. - `Relaxed` / `Acquire` / `Release` — `Relaxed` ist atomar ohne Ordering; `Release` auf einem Store ist ein Boden unter allem, was davor kommt; `Acquire` auf einem Load ist ein Dach über allem, was danach kommt; ein `Acquire`-Load, der einen `Release`-Store beobachtet, verbindet die beiden. - `happens-before` — die threadübergreifende Garantie, die ein `Release`/`Acquire`-Paar aufbaut; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - `False Sharing` — zwei unabhängige Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores invalidiert; behoben durch `CachePadded`. - `loom` — ein Model Checker, der einen kleinen Test unter jeder Verschränkung ausführt; blind für Übergaben, die über einen CAS laufen. - `Miri` — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB meldet — Data Races, uninitialisierte Reads, Leaks.*

Two per slot: A (`publish`, `producer` → `consumer`) and B (`vacate`, `consumer` → next lap's `producer`). - `compare_exchange` (CAS) — atomic read-modify-write that succeeds only if the value is what you expected; how exactly one thread **claims** a ticket. - `Relaxed` / `Acquire` / `Release` — `Relaxed` is atomic with no ordering; `Release` on a store floors what comes before it; `Acquire` on a load roofs what comes after it; an `Acquire` load that observes a `Release` store joins the two. - `happens-before` — the cross-thread guarantee a `Release`/`Acquire` pair builds; without one, a plain write on one thread and a plain read on another are a data race. - `false sharing` — two independent values on one cache line, each write invalidating the other core's copy; fixed by `CachePadded`. - `loom` — a model checker that runs a small test under every interleaving; blind to hand-offs carried by a CAS. - `Miri` — an interpreter that runs Rust against the memory model and reports UB — data races, uninitialized reads, leaks.

bedeuten

bedeutet · bedeutete ·
bedeutet

T1

to mean, signify

*Ein Allocator-Aufruf **bedeutet** an einem guten Tag hunderte Nanosekunden und an einem schlechten ein prozessweites Lock; so oder so passt er nicht in eine Übergabe, die in unter einer Mikrosekunde fertig sein muss. - Lock-free. Kein einzelnes Speicherwort, das sich jeder Thread erst holen muss.*

An allocator call **is** hundreds of nanoseconds on a good day and a process-wide lock on a bad one; either way it doesn't fit inside a hand-off that has to complete in under a microsecond. - Lock-free. No single word every thread must acquire.

beginnen

beginnt · begann ·
begonnen

T0

to begin, start

*Der Reihe nach. — Jeder Teil **beginnt** dort, wo der vorige aufgehört hat, und schließt mit der Frage, die der nächste beantwortet.*

In order. — Each part **opens** where the previous one stopped and closes on the question the next one answers.

behalten

behält · behielt ·
behalten

T1

to retain, keep

*Dieses & ist ein Modulo, aber nur, wenn **capacity** eine Zweierpotenz ist — dann ist **capacity** - 1 eine Folge niedriger 1-Bits, und die Maske **behält** genau diese.*

That & is a modulo, but only when **capacity** is a power of two — then **capacity** - 1 is a run of low 1 bits and the mask **keeps** exactly them.

bekommen

bekommt · bekam ·
bekommen

T1

to get, receive

*Jeder braucht einen eigenen Slot, und es ist egal, wer welchen **bekommt**. — Zwei Consumer: dasselbe.*

Each needs a distinct slot, and it doesn't matter who **gets** which. — Two consumers, the same.

bemerken

bemerkt · bemerkte ·
bemerkt

T0

to notice, observe

*Die Orderings werden hergeleitet, nicht geraten: die zwei Stellen finden, an denen Daten den Thread wechseln, und dann **bemerken**, dass der payload nur über **seq** veröffentlicht werden kann, nie über den Zähler — ein **Release** legt einen Boden unter alles, was **davor** kommt, und der Claim kommt vor dem Write.*

The orderings are derived, not guessed: find the two places data crosses threads, then **notice** the payload can only be published through **seq**, never through the counter — a **Release** floors what comes **before** it, and the claim comes before the write.

benennen

benennt · benannte ·
benannt

T1

to name, identify

*Nimm das Lock weg und **benenne**, wofür es stellvertretend stand: — > (i) Entscheiden, welcher Thread welchen Slot besitzt. (ii) Den payload von dem > Producer, der einen Slot füllt, zu dem Consumer bringen, der ihn leert.*

Strip the lock away and **state** what it was standing in for: — > (i) Decide which thread owns which slot. (ii) Move the payload from the producer > that fills a slot to the consumer that empties it.

beobachten

beobachtet ·
beobachtete ·
beobachtet

T0

to observe (load observing a store)

Zwei pro Slot: A (*publish*, Producer → Consumer) und B (*vacate*, Consumer → Producer der nächsten Runde). - *compare_exchange* (CAS) — atomares Read-Modify-Write, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so beansprucht genau ein Thread ein Ticket. - *Relaxed / Acquire / Release* — *Relaxed* ist atomar ohne Ordering; *Release* auf einem Store ist ein Boden unter allem, was davor kommt; *Acquire* auf einem Load ist ein Dach über allem, was danach kommt; ein *Acquire*-Load, der einen *Release*-Store **beobachtet**, verbindet die beiden. - *happens-before* — die threadübergreifende Garantie, die ein *Release*/*Acquire*-Paar aufbaut; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - *False Sharing* — zwei unabhängige Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores invalidiert; behoben durch *CachePadded*. - *loom* — ein Model Checker, der einen kleinen Test unter jeder Verschränkung ausführt; blind für Übergaben, die über einen CAS laufen. - *Miri* — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB meldet — Data Races, uninitialisierte Reads, Leaks.

Two per slot: A (*publish*, producer → consumer) and B (*vacate*, consumer → next lap's producer). - *compare_exchange* (CAS) — atomic read-modify-write that succeeds only if the value is what you expected; how exactly one thread claims a ticket. - *Relaxed / Acquire / Release* — *Relaxed* is atomic with no ordering; *Release* on a store floors what comes before it; *Acquire* on a load roofs what comes after it; an *Acquire* load that **observes** a *Release* store joins the two. - *happens-before* — the cross-thread guarantee a *Release*/*Acquire* pair builds; without one, a plain write on one thread and a plain read on another are a data race. - *false sharing* — two independent values on one cache line, each write invalidating the other core's copy; fixed by *CachePadded*. - *loom* — a model checker that runs a small test under every interleaving; blind to hand-offs carried by a CAS. - *Miri* — an interpreter that runs Rust against the memory model and reports UB — data races, uninitialized reads, leaks.

beschreiben

beschreibt · beschrieb
· beschrieben

T2

to describe

> *head* und *tail* verteilen, wer an der Reihe ist. — Den Zustand eines Slots können > sie nicht **beschreiben**.

> *head* and *tail* allocate turns. — They cannot **describe** the state of a slot.

beschränken

beschränkt ·
beschränkte ·
beschränkt

T4

to bound, restrict

Eine Contention-Achse pro Modell — zwei Producer und ein Consumer oder zwei Consumer und ein Producer, nie alles auf einmal — eine Kapazität von zwei, die Producer gejoint, bevor der Consumer den Ring leert, und jeder Retry im Testcode **beschränkt**.

One contention axis per model — two producers and a consumer, or two consumers and a producer, never everything at once — a capacity of two, producers joined before the consumer drains, and any retry in the test body **bounded**.

beseitigen

beseitigt · beseitigte
· beseitigt

T1

to eliminate, remove

*Ein **Mutex** um ein festes Array **beseitigt** die Allokation und behält das Lock. — Ein **Mutex** *per Slot* verkleinert das Lock, ohne es loszuwerden, und bringt ein Problem mit, das das Design vorher nicht hatte: entscheiden, welchen Slot jeder Thread ansteuern soll.*

A **Mutex** around a fixed array **fixes** the allocation and keeps the lock. — A **mutex** *per slot* shrinks the lock without removing it, and adds a problem the design didn't have before: deciding which slot each thread should head for.

besitzen

besitzt · besaß ·
besessen

T1

to own (a slot or ticket)

*Nimm das Lock weg und benenne, wofür es stellvertretend stand: — > (i) Entscheiden, welcher Thread welchen Slot **besitzt**. (ii) Den payload von dem > Producer, der einen Slot füllt, zu dem Consumer bringen, der ihn leert.*

Strip the lock away and state what it was standing in for: — > (i) Decide which thread **owns** which slot. (ii) Move the payload from the producer > that fills a slot to the consumer that empties it.

bestehen

besteht · bestand ·
bestanden

T1

to pass (tests) / consist of

*Ein Array, zwei Zähler, eine Maske. — Bau Push und Pop auf die naheliegende Art — Zähler erhöhen, Slot anfassen — und es **besteht** jeden Test, den du am Schreibtisch schreibst: ein Thread pusht, einer poppt, jede Zählung geht auf.*

An array, two counters, a mask. — Build the push and pop the obvious way — bump the counter, touch the slot — and it **passes** every test you write at your desk: one thread pushing, one popping, every count tallying.

bewegen

bewegt · bewegte ·
bewegt

T2

to advance, move (counter/state)

*## Beanspruchen ist nicht Veröffentlichen — Das **compare_exchange** des Producers **bewegt** **tail** von 0 auf 1 in dem Moment, in dem das Ticket *beansprucht* wird — und dann wird der Producer vom OS unterbrochen, noch vor dem Write:*

Claiming is not publishing — The producer's **compare_exchange** **moves** **tail** from 0 to 1 the moment the ticket is *claimed* — and then the producer gets preempted, before the write:

beweisen

beweist · bewies ·
bewiesen

T2

to prove

*Nur der Producer, der Ticket **pos** hält, kann **seq** von **pos** auf **pos + 1** bewegen, und wenn mein CAS auf **tail** von **pos** aus gelingt, dann bin ich dieser Producer. Ein erfolgreicher Claim **beweist** im Nachhinein, dass der Gate-Read aktuell war.*

Only the producer holding ticket **pos** can move **seq** from **pos** to **pos + 1**, and if my CAS on **tail** succeeds from **pos**, that producer is me. A successful claim retroactively **proves** the gate read was current.

bleiben

bleibt · blieb ·
geblieben

T0

to remain, stay

*Boden und Dach, ein Gate nach dem anderen, warum der CAS **Re**laxed **bleibt**, und warum dieser Ring — anders als ein SeqLock — keinen Fence braucht.*

Floor and roof, one gate at a time, why the CAS **stays** **Re**laxed, and why this ring — unlike a SeqLock — needs no fence.

blockieren

blockiert · blockierte
· blockiert

T1

to block, obstruct

*Die Queue parkt nie einen Thread. — **Blockieren** ist eine Policy, die der Aufrufer vielleicht will, und sie darf nicht fest ins Primitiv eingebaut sein.*

The queue never parks a thread. — **Blocking** is a policy the caller may want, and it must not be baked into the primitive.

brauchen

braucht · brauchte ·
gebraucht

T0

to need, require

*Boden und Dach, ein Gate nach dem anderen, warum der CAS ReLaxed bleibt, und warum dieser Ring — anders als ein SeqLock — keinen Fence **braucht**.*

Floor and roof, one gate at a time, why the CAS stays ReLaxed, and why this ring — unlike a SeqLock — **needs** no fence.

bringen

bringt · brachte ·
gebracht

T0

to bring, transfer

*Die Arbeitsteilung zwischen den drei Instrumenten, ein Drop, das Miri's Leak-Checker absegnet, und der Benchmark: head und tail mit Padding auf getrennte cache lines zu legen, **bringt** die SPSC-Übergabe von ~51 ns auf ~9 ns.*

The division of labor between the three instruments, a Drop that Miri's leak checker approves, and the benchmark: padding head and tail onto separate cache lines **takes** the SPSC hand-off from ~51 ns to ~9 ns.

bündeln

bündelt · bündelte ·
gebündelt

T0

to fan into, batch together

*# Einen lock-freien MPMC-Ring entwerfen — Am schreibenden Ende von allem, was schnell ist — eine Matching Engine, die Orders von einem Dutzend Feed-Threads entgegennimmt, ein Write-Ahead-Log, das viele Writer in einen einzigen Commit **bündelt** —, sitzt eine Queue, in die viele Threads pushen und aus der viele Threads poppen, millionenfach pro Sekunde.*

Designing a lock-free MPMC ring — At the write end of anything fast — a matching engine taking orders off a dozen feed threads, a write-ahead log fanning writers into one commit — there's a queue that many threads push and many threads pop, millions of times a second.

droppen

droppt · droppte ·
gedroppt

T4

to drop (run destructor)

*Noch etwas, worauf Miri achtet. — `Box<Cell<T>>` **droppt** die Zellen, aber eine Zelle hält ein `MaybeUninit<T>`, und das droppt nichts — es kann nicht wissen, ob es einen Wert hält.*

One more thing Miri watches for. — `Box<Cell<T>>` **drops** the cells, but a cell holds `MaybeUninit<T>`, which drops nothing — it can't know whether it holds a value.

durchlaufen

durchläuft · durchlief
· durchlaufen

T3

to pass all tests, run through

*Jedes Ordering oben wurde per Argument platziert. — Die All-ReLaxed-Version auch — das Argument war, dass die Tests **durchliefen**.*

Every ordering above was placed by an argument. — So was the all-ReLaxed version — the argument being that the tests **passed**.

durchrutschen

rutscht durch ·
rutschte durch ·
durchgerutscht, trennb.

T4

to slip past (undetected race)

*Es führt die Verschränkungen aus, die es nun mal ausführt, mit ein paar eingestreuten Unterbrechungen. Ein Race, der einen bestimmten Schedule braucht, kann **durchrutschen**.*

It runs the interleavings it happens to run, with some preemption thrown in. A race that needs a specific schedule can **slip past**.

einbauen

baut ein · baute ein ·
eingebaut, trennb.

T1

to bake in, embed

*Die Queue parkt nie einen Thread. — Blockieren ist eine Policy, die der Aufrufer vielleicht will, und sie darf nicht fest ins Primitiv **eingebaut** sein.*

The queue never parks a thread. — Blocking is a policy the caller may want, and it must not be **baked into** the primitive.

einbringen

bringt ein · brachte
ein · eingebracht,
trennb.

T3

to buy, yield, gain

*Der CAS sitzt zwischen zwei Gates, die die Arbeit erledigen. — Ein AcqRel darauf würde nichts **einbringen** und bei jedem Claim eine Barriere kosten — auf aarch64 ein `casal`, wo ein `cas` reicht.*

The CAS sits between two gates that do the work. — An AcqRel on it would **buy** nothing and cost a barrier on every claim — on aarch64, a `casal` where a `cas` will do.

einfrieren

friert ein · fror ein ·
eingefroren, trennb.

T1

to freeze (stall the whole system)

*Die Garantie, die wir wollen: Zu jedem Zeitpunkt macht *irgendein* Thread Fortschritt, und ein Thread, den das OS mitten in einer Operation unterbricht — und der dabei nichts hält — kann den Rest nicht **einfrieren**. - Viele-zu-viele, und nicht-blockierend. N Producer, M Consumer, keine festen Rollen.*

The guarantee we want is that at any moment *some* thread is making progress, and a thread the OS preempts mid-operation — holding nothing — cannot **freeze** the rest. - Many-to-many, and non-blocking. N producers, M consumers, no fixed roles.

einhalten

hält ein · hielt ein ·
eingehalten, trennb.

T1

to satisfy, hold (constraints)

*Wäre Durchsatz die einzige Anforderung, wäre das ein Benchmark und kein Designproblem. Es ist ein Designproblem, weil die Queue drei Dinge gleichzeitig **einhalten** muss:*

If throughput were the only requirement this would be a benchmark, not a design problem. It's a design problem because the queue has to **hold** three things at once:

einplanen

plant ein · plante ein
· eingeplant, trennb.

T1

to schedule (by OS)

*Und in dem Moment, in dem der Halter des Locks vom OS verdrängt wird, wartet jeder andere Thread darauf, dass genau dieser eine Thread wieder **eingeplant** wird.*

And the instant the holder is preempted, every other thread waits for that one thread to be **scheduled** again.

einsetzen

setzt ein · setzte ein
· eingesetzt, trennb.

T4

to put in, insert

***Setz** die Orderings aus Teil 3 ein, und die Schedules, in denen der Wert ewig veraltet bleibt, sind nicht mehr legal.*

Put the Part 3 orderings in and the stale-forever schedules stop being legal.

einstreuen

streut ein · streute
ein · eingestreut,
trennb.

T4

to sprinkle in, intersperse

*Was Miri nicht tut, ist aufzählen. — Es führt die Verschränkungen aus, die es nun mal ausführt, mit ein paar **eingestreuten** Unterbrechungen.*

What Miri does not do is enumerate. — It runs the interleavings it happens to run, with some preemption **thrown in**.

einteilen

teilt ein · teilte ein
· eingeteilt, trennb.

T4

to divide, allocate (labor/turns)

*## Die **Arbeitsteilung** — Die Argumentation entscheidet, welches Ordering und warum, und ist blind für das, woran du nicht gedacht hast.*

*## **Division of labor** — The reasoning decides which ordering and why, and is blind to what you didn't think of.*

einwechseln

wechselt ein ·
wechselte ein ·
eingewechselt, trennb.

T4

to swap in (mock atomics)

*Der payload-Zugriff läuft über `loom::cell::UnsafeCell`, damit loom ihn sieht, hinter einem `cfg(loom)`-Shim, der looms Atomics **einwechselt**.*

Payload access goes through `loom::cell::UnsafeCell` so loom can see it, behind a `cfg(loom)` shim that **swaps in** loom's atomics.

entgegennehmen

nimmt entgegen · nahm
entgegen ·
entgegengenommen,
trennb.

T0

to take in, receive orders

*# Einen lock-freien MPMC-Ring entwerfen — Am schreibenden Ende von allem, was schnell ist — eine Matching Engine, die Orders von einem Dutzend Feed-Threads **entgegennimmt**, ein Write-Ahead-Log, das viele Writer in einen einzigen Commit bündelt —, sitzt eine Queue, in die viele Threads pushen und aus der viele Threads poppen, millionenfach pro Sekunde.*

Designing a lock-free MPMC ring — At the write end of anything fast — a matching engine **taking** orders off a dozen feed threads, a write-ahead log fanning writers into one commit — there's a queue that many threads push and many threads pop, millions of times a second.

entscheiden

entscheidet · entschied
· entschieden

T1

to decide

*Ein Mutex **pro Slot** verkleinert das Lock, ohne es loszuwerden, und bringt ein Problem mit, das das Design vorher nicht hatte: **entscheiden**, welchen Slot jeder Thread ansteuern soll.*

A mutex **per slot** shrinks the lock without removing it, and adds a problem the design didn't have before: deciding which slot each thread should head for.

entwerfen

entwirft · entwarf ·
entworfen

T0

to design, architect

*Diese Serie **entwirft** die Queue, die das nicht tut: einen Ring — beschränkt, alloktationsfrei, lock-free —, den sich N Producer und M Consumer teilen und bei dem nur noch die Übergabe selbst etwas kostet.*

This series **designs** the queue that doesn't: a bounded, allocation-free, lock-free ring shared by N producers and M consumers, where the only cost is the hand-off itself.

erforschen

erforscht · erforschte
· erforscht

T4

to explore (schedules)

*Es modelliert auch die Orderings: Ein Relaxed-Load darf einen Wert liefern, den der speichernde Thread vor einer Weile geschrieben hat, und loom **erkundet** den Schedule, in dem er das tut.*

It also models the orderings: a Relaxed load may return a value the store thread wrote a while ago, and loom will **explore** the schedule where it does.

ergeben (sich)

ergibt sich · ergab
sich · ergeben

T1

to fall out of, result from

*Ein einzelnes Lock lässt jeden Thread jedes Mal auf beide Aufgaben warten: Ein Producer blockiert einen Consumer, der einen völlig anderen Slot will, weil sie sich das Lock-Wort teilen. Das ganze Design **ergibt sich** daraus, genau das zu verweigern.*

A single lock makes every thread wait for both jobs, every time: a producer blocks a consumer that wants a completely different slot, because they share the lock word. The whole design **falls out of** refusing that.

erhöhen

erhöht · erhöhte ·
erhöht

T1

to increment, bump

*Ein Array, zwei Zähler, eine Maske. — Bau Push und Pop auf die naheliegende Art — Zähler **erhöhen**, Slot anfassen — und es besteht jeden Test, den du am Schreibtisch schreibst: ein Thread pusht, einer poppt, jede Zählung geht auf.*

An array, two counters, a mask. — Build the push and pop the obvious way — **bump** the counter, touch the slot — and it passes every test you write at your desk: one thread pushing, one popping, every count tallying.

erkunden

erkundet · erkundete ·
erkundet

T4

to explore (state permutations)

*Es modelliert auch die Orderings: Ein Relaxed-Load darf einen Wert liefern, den der speichernde Thread vor einer Weile geschrieben hat, und loom **erkundet** den Schedule, in dem er das tut.*

It also models the orderings: a Relaxed load may return a value the store thread wrote a while ago, and loom will **explore** the schedule where it does.

erledigen

erledigt · erledigte ·
erledigt

T0

to do, accomplish (jobs/tasks)

*Das Lock **erledigt**, wie sich herausstellt, zwei Jobs — Reihen vergeben und Daten übergeben —, und die beiden zu trennen ist das ganze Design: ein Array, zwei Ticket-Zähler und eine Maske.*

The lock turns out to be **doing** two jobs, allocating turns and handing off data, and separating them is the whole design: an array, two ticket counters, and a mask.

erreichen

erreicht · erreichte ·
erreicht

T3

to reach, achieve

*Vier Gates, viermal die Antwort „gleiche Seite“. — Ein Ordering auf der Op **erreicht** alles, was es **erreichen** muss.*

Four gates, four same-side answers. — An ordering on the op reaches everything it needs to.

erzwingen

erzwingt · erzwang ·
erzwungen

T0

to force (by constraints)

*Es ist eine Design-Untersuchung, kein Tutorial. — Jede Entscheidung ist **erzwungen** — durch einen Use Case, durch das Scheitern der einfacheren Variante oder durch ein Verifikationswerkzeug.*

It's a design investigation, not a tutorial. — Every decision is **forced** — by a use case, by the simpler version failing, or by a verification tool.

fehlschlagen

schlägt fehl · schlug
fehl · fehlgeschlagen,
trennb.

T4

to fail

*Auf dem All-Re~~l~~axed-Ring ist loom nicht **fehlgeschlagen**. — Es hat aufgegeben:*

On the all-Re~~l~~axed ring, loom didn't **fail**. — It gave up:

festhalten

hält fest · hielt fest
· festgehalten, trennb.

T2

to hold fast, pin down / trap in spin

*Bleibt das Primitiv hier in einer Spin-Schleife, ist es zu einer blockierenden Queue mit Umwegen geworden, und zu einer schlechteren: Ein Producer, der mitten im Write vom OS verdrängt wird, hält jeden Consumer hinter sich in einer Spin-Schleife **fest**.*

If the primitive spins here, it has become a blocking queue with extra steps, and a worse one: a producer preempted mid-write **holds** every consumer behind it in a spin.

finden

findet · fand ·
gefunden

T0

to find

*Die Orderings werden hergeleitet, nicht geraten: die zwei Stellen **finden**, an denen Daten den Thread wechseln, und dann bemerken, dass der payload nur über seq veröffentlicht werden kann, nie über den Zähler — ein Release legt einen Boden unter alles, was *davor* kommt, und der Claim kommt vor dem Write.*

The orderings are derived, not guessed: **find** the two places data crosses threads, then notice the payload can only be published through seq, never through the counter — a Release floors what comes *before* it, and the claim comes before the write.

folgen

folgt · folgte ·
gefolgt

T3

to follow

*head.load ist ein Anhaltspunkt, Re~~l~~axed. — Das seq-Gate ist das Dach über dem payload-Read, der **folgt** — die Empfangsseite von Übergabe A — Acquire.*

head.load is a hint, Re~~l~~axed. — The seq gate is the roof over the payload read that **follows** — hand-off A's receiving side — Acquire.

fortsetzen

setzt fort · setzte
fort · fortgesetzt,
trennb.

T2

to continue (loop)

*CAS verloren. Ein anderer Thread hat Ticket pos genommen. — Das nächste Ticket ist frei, niemand muss etwas tun, damit ich es bekomme, also läuft die Schleife mit dem neuen tail **weiter**.*

Lost the CAS. Another thread took ticket pos. — The next ticket is free, nobody has to do anything for me to get it, so the loop **continues** with the new tail.

freigeben

gibt frei · gab frei ·
freigegeben, trennb.

T1

to free, release

- *Beschränkt und allokatonsfrei. Ein fester Puffer, einmal dimensioniert. — Keine Box pro Nachricht, kein verketteter Knoten, der beim Push alloziert und beim Pop **freigegeben** wird.*

- Bounded and allocation-free. A fixed buffer, sized once. — No Box per message, no linked node allocated on push and **freed** on pop.

funktionieren

funktioniert ·
funktionierte ·
funktioniert

T1

to work, function

Die naheliegenden Queues, je eine Randbedingung verletzt — *Mutex<VecDeque<T>> ist das, was der meiste Code ausliefert, und es **funktioniert**.*

The obvious queues, one constraint each — Mutex<VecDeque<T>> is what most code ships, and it **works**.

führen

führt · führte ·
geführt

T1

to funnel, lead

*Ein Write-Ahead-Log bündelt ein Dutzend Writer-Threads in einen Group Commit. — Eine Metrics-Pipeline **führt** Samples von jedem Worker in einem einzigen Aggregator zusammen.*

A write-ahead log fans a dozen writer threads into one group commit. — A metrics pipeline **funnels** samples from every worker into one aggregator.

füllen

füllt · füllte ·
gefüllt

T1

to fill (a slot)

*Nimm das Lock weg und benenne, wofür es stellvertretend stand: — > (i) Entscheiden, welcher Thread welchen Slot besitzt. (ii) Den payload von dem > Producer, der einen Slot **füllt**, zu dem Consumer bringen, der ihn leert.*

Strip the lock away and state what it was standing in for: — > (i) Decide which thread owns which slot. (ii) Move the payload from the producer > that **fills** a slot to the consumer that empties it.

füttern

füttert · fütterte ·
gefüttert

T0

to feed (model checker with code)

*[Teil 4 — Der Beweis: zwei Werkzeuge und ein lügendes Grün.] (04_proving_it.md) Miri nennt den Race beim Namen; loom, mit dem Relaxed-Ring **gefüttert**, explodiert, statt zu antworten.*

[Part 4 — Proving it: two tools, and a lying green.](04_proving_it.md) Miri names the race; loom, **given** the Relaxed ring, explodes instead of answering.

geben

gibt · gab · gegeben

T1

to give, surrender

*Und eine try_-API: try_push auf einem vollen Ring **gibt** den Wert zurück; try_pop auf einem leeren Ring liefert None.*

And a try_ API: try_push on a full ring hands the value back; try_pop on an empty ring returns None.

gefallen

gefällt · gefiel ·
gefallen

T2

to please (as scheduler likes)

*Die Menge der tatsächlich geschriebenen Slots ist {1}. — Kein einzelner Zähler kann „Slot 1 ist geschrieben, Slot 0 nicht“ ausdrücken, denn Claims werden der Reihe nach vergeben, und Writes werden in der Reihenfolge fertig, die dem Scheduler gerade **gefällt**.*

The set of slots actually written is {1}. — No single counter can express "slot 1 is written, slot 0 isn't", because claims are handed out in order and writes complete in whatever order the scheduler **likes**.

gehören

gehört · gehörte · gehört

T3

to belong to

Die Logik sagt: Der Consumer liest erst, nachdem der Producer veröffentlicht hat. — Die Hardware hat von der Logik nie gehört.

The logic says the consumer only reads after the producer published. — The hardware never heard the logic.

gelingen

gelingt · gelang · gelungen

T0

to succeed (CAS success)

Zwei pro Slot: A (publish, Producer → Consumer) und B (vacate, Consumer → Producer der nächsten Runde). - compare_exchange (CAS) — atomares Read-Modify-Write, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so beansprucht genau ein Thread ein Ticket. - Relaxed / Acquire / Release — Relaxed ist atomar ohne Ordering; Release auf einem Store ist ein Boden unter allem, was davor kommt; Acquire auf einem Load ist ein Dach über allem, was danach kommt; ein Acquire-Load, der einen Release-Store beobachtet, verbindet die beiden. - happens-before — die threadübergreifende Garantie, die ein Release/Acquire-Paar aufbaut; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - False Sharing — zwei unabhängige Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores invalidiert; behoben durch CachePadded. - loom — ein Model Checker, der einen kleinen Test unter jeder Verschränkung ausführt; blind für Übergaben, die über einen CAS laufen. - Miri — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB meldet — Data Races, uninitialisierte Reads, Leaks.

Two per slot: A (publish, producer → consumer) and B (vacate, consumer → next lap's producer). - compare_exchange (CAS) — atomic read-modify-write that **succeeds** only if the value is what you expected; how exactly one thread claims a ticket. - Relaxed / Acquire / Release — Relaxed is atomic with no ordering; Release on a store floors what comes before it; Acquire on a load roofs what comes after it; an Acquire load that observes a Release store joins the two. - happens-before — the cross-thread guarantee a Release/Acquire pair builds; without one, a plain write on one thread and a plain read on another are a data race. - false sharing — two independent values on one cache line, each write invalidating the other core's copy; fixed by CachePadded. - loom — a model checker that runs a small test under every interleaving; blind to hand-offs carried by a CAS. - Miri — an interpreter that runs Rust against the memory model and reports UB — data races, uninitialized reads, leaks.

gewinnen

gewinnt · gewann · gewonnen

T2

to win (a contention race)

Das nächste Ticket ist frei, niemand muss etwas tun, damit ich es bekomme, also läuft die Schleife mit dem neuen tail weiter. Hier in der Schleife zu bleiben ist ein Wettlauf, den ich allein gewinnen kann.

The next ticket is free, nobody has to do anything for me to get it, so the loop continues with the new tail. Looping here is a race I can **win** alone.

greifen nach

greift · griff · gegriffen

T0

to reach for

Greif zu einem Mutex<VecDeque>, und jeder dieser Threads serialisiert sich auf einem einzigen Lock-Wort, das zwischen den Cores hin- und herspringt.

Reach for a Mutex<VecDeque> and every one of those threads serializes on one lock word that bounces between cores.

halten

hält · hielt · gehalten T0

to hold (locks or tickets)

- MPMC-Ring — ein beschränkter Ringpuffer, in den mehrere Producer pushen und aus dem mehrere Consumer poppen, lock-free und allokatonsfrei. - lock-free — irgendein Thread kommt immer voran; kein Thread **hält** ein Lock, auf das andere warten.

- MPMC ring — a bounded circular buffer that multiple producers push and multiple consumers pop, lock-free and allocation-free. - lock-free — some thread always makes progress; no thread **holds** a lock others wait on.

heißen

heißt · hieß · geheißen T2

to mean, be called

Dass *tail* sich bewegt, **heißt**: *Jemand besitzt Ticket 0*. — Es heißt nicht, dass im Slot von Ticket 0 irgendetwas drin ist.

tail moving **means** *someone owns ticket 0*. — It does not mean ticket 0's slot has anything in it.

herausstellen (sich)

stellt sich heraus ·
stellte sich heraus ·
herausgestellt, trennb. T0

to turn out to be

Das Lock erledigt, wie sich **herausstellt**, zwei Jobs — Reihen vergeben und Daten übergeben —, und die beiden zu trennen ist das ganze Design: ein Array, zwei Ticket-Zähler und eine Maske.

The lock **turns out** to be doing two jobs, allocating turns and handing off data, and separating them is the whole design: an array, two ticket counters, and a mask.

herleiten

leitet her · leitete
her · hergeleitet,
trennb. T0

to derive (orderings by logic)

Die Orderings werden **hergeleitet**, nicht geraten: die zwei Stellen finden, an denen Daten den Thread wechseln, und dann bemerken, dass der payload nur über *seq* veröffentlicht werden kann, nie über den Zähler — ein *Release* legt einen Boden unter alles, was **davor** kommt, und der Claim kommt vor dem Write.

The orderings are **derived**, not guessed: find the two places data crosses threads, then notice the payload can only be published through *seq*, never through the counter — a *Release* floors what comes **before** it, and the claim comes before the write.

herrschen

herrscht · herrschte ·
geherrscht T1

to prevail, exist (contention)

Aber jeder *push* und jeder *pop* nimmt dasselbe Lock, also springt das Lock-Wort bei jeder einzelnen Operation wie ein Pingpongball zwischen den Cores hin und her — das MESI-Protokoll invalidiert eine cache line in jedem anderen Core, millionenfach pro Sekunde, selbst wenn logisch gar keine Contention **herrscht**.

But every push and every pop takes the same lock, so the lock word ping-pongs between cores on every single operation — the MESI protocol invalidating one cache line in every other core, millions of times a second, even when nobody is logically **contending**.

hin- und herspringen

springt hin und her ·
sprang hin und her ·
hin- und hergesprungen,
trennb. T0

to bounce back and forth (cache line ping-pong)

Greif zu einem *Mutex<VecDeque>*, und jeder dieser Threads serialisiert sich auf einem einzigen Lock-Wort, das zwischen den Cores **hin- und herspringt**.

Reach for a *Mutex<VecDeque>* and every one of those threads serializes on one lock word that **bounces** between cores.

hindurchsinken

sinkt hindurch · sank
hindurch ·
hindurchgesunken,
trennb.

T3

to sink past (a store boundary)

*Ein Release auf diesem CAS hat nichts unter sich. — Der Write liegt immer noch über dem Boden und kann ungehindert **hindurchsinken**.*

A Release on that CAS has nothing under it. — The write is still above the floor, free to **sink past**.

hochzählen

zählt hoch · zählte
hoch · hochgezählt,
trennb.

T1

to increment, count up

*Ein festes Array von Slots. — Ein Zähler, **tail**, den Producer atomar lesen und **hochzählen**, um eine Reihe zu beanspruchen — der Wert, den du dabei genommen hast, ist dein *Ticket*.*

A fixed array of slots. — One counter, **tail**, that producers atomically **read-and-bump** to claim a turn — the value you took is your *ticket*.

holen

holt · holte · geholt T4

to pull, fetch (cache lines in pairs)

*Das ist False Sharing, und der Fix ist, jeden Zähler auf seine eigene cache line zu legen: CachePadded, 128 Bytes auf Apple Silicon, wo der Prefetcher cache lines paarweise **holt**.*

That's false sharing, and the fix is to put each counter on its own line: CachePadded, 128 bytes on Apple silicon, where the prefetcher **pulls** lines in pairs.

hängen

hängt · hing · gehangen T1

to stall, hang

*Ein paar Millionen Operationen pro Sekunde, von beiden Enden gleichzeitig, und auf keiner der beiden Seiten kann es sich ein Thread leisten stehenzubleiben — ein Feed-Thread, der **hängt**, verliert Marktdaten; ein Matcher, der hängt, weitet den Spread bei jedem Symbol, das hinter ihm in der Queue wartet.*

A few million operations a second, from both ends at once, and no thread on either side can afford to stop — a feed thread that **stalls** drops market data; a matcher that stalls widens the spread on every symbol queued behind it.

ineinanderrasten

rastet ineinander ·
rastete ineinander ·
ineinandergerastet,
trennb.

T3

to lock together, interlock

*Über das, was davor kommt, sagt es nichts. — Liest der Acquire-Load den Wert, den der Release-Store geschrieben hat, **rasten die beiden Hälften ineinander**: Was über dem Boden lag, ist jetzt garantiert unter dem Dach sichtbar.*

It says nothing about what comes before. — When the Acquire load reads the value the Release store wrote, the two halves **lock**: what was above the floor is now guaranteed visible below the roof.

invalidieren

invalidiert ·
invalidierte ·
invalidiert

T0

to invalidate (cache line via MESI)

Zwei pro Slot: A (publish, Producer → Consumer) und B (vacate, Consumer → Producer der nächsten Runde). - **compare_exchange** (CAS) — atomares Read-Modify-Write, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so beansprucht genau ein Thread ein Ticket. - **Relaxed / Acquire / Release** — **Relaxed** ist atomar ohne Ordering; **Release** auf einem Store ist ein Boden unter allem, was davor kommt; **Acquire** auf einem Load ist ein Dach über allem, was danach kommt; ein **Acquire-Load**, der einen **Release-Store** beobachtet, verbindet die beiden. - **happens-before** — die threadübergreifende Garantie, die ein **Release/Acquire-Paar** aufbaut; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - **False Sharing** — zwei unabhängige Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores **invalidiert**; behoben durch **CachePadded**. - **loom** — ein Model Checker, der einen kleinen Test unter jeder Verschränkung ausführt; blind für Übergaben, die über einen CAS laufen. - **Miri** — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB meldet — Data Races, uninitialisierte Reads, Leaks.

Two per slot: A (publish, producer → consumer) and B (vacate, consumer → next lap's producer). - **compare_exchange** (CAS) — atomic read-modify-write that succeeds only if the value is what you expected; how exactly one thread claims a ticket. - **Relaxed / Acquire / Release** — **Relaxed** is atomic with no ordering; **Release** on a store floors what comes before it; **Acquire** on a load roofs what comes after it; an **Acquire** load that observes a **Release** store joins the two. - **happens-before** — the cross-thread guarantee a **Release/Acquire** pair builds; without one, a plain write on one thread and a plain read on another are a data race. - **false sharing** — two independent values on one cache line, each write **invalidating** the other core's copy; fixed by **CachePadded**. - **loom** — a model checker that runs a small test under every interleaving; blind to hand-offs carried by a CAS. - **Miri** — an interpreter that runs Rust against the memory model and reports UB — data races, uninitialized reads, leaks.

jagen

jagt · jagte · gejagt T4

to chase (nanoseconds)

Auch noch das seq jedes Slots zu padden ist der naheliegende nächste Schritt — und lohnt sich nicht: Es vervierfacht den Speicher eines Rings mit kleinen payloads, um den letzten 1,8 ns **hinterherzujagen**.

Padding each slot's seq as well is the obvious next step and isn't worth it: it quadruples the memory of a ring of small payloads to **chase** the last 1.8 ns.

kehren

kehrt · kehrte ·
gekehrt

T2

to return (instead of spinning)

Das ist der Ring. — Er verteilt über zwei Zähler, wer an der Reihe ist, und übergibt Daten über eine Sequenz pro Slot; er ist beschränkt, allokatonsfrei, lock-free, und er **kehrt zurück**, statt zu warten.

That's the ring. — It allocates turns on two counters and hands data off through a per-slot sequence, it's bounded, allocation-free, lock-free, and it **returns** instead of waiting.

kleben (an)

klebt · klebte ·
geklebt

T3

to glue to

*Kein Ordering auf diesen Ops konnte die Daten erreichen. — Ein Fence — eine zweiseitige Wand, die du selbst positionierst, kein an eine Op **geklebtes** Gate — war das einzige Werkzeug.*

No ordering on those ops could reach the data. — A fence — a two-sided wall you position yourself, not a gate **glued** to an op — was the only tool.

kollidieren

kollidiert ·
kollidierte ·
kollidiert

T0

to collide with

*Kein Lock-free-Hintergrund wird vorausgesetzt; `compare_exchange`, `Acquire/Release`, `UnsafeCell`, `MaybeUninit`, `loom`, `Miri` und `CachePadded` werden dort eingeführt, wo das Design mit ihnen **kollidiert**.*

No lock-free background is assumed; `compare_exchange`, `Acquire/Release`, `UnsafeCell`, `MaybeUninit`, `loom`, `Miri` and `CachePadded` are introduced where the design **collides** with them.

kommen

kommt · kam · gekommen T0

to come, arrive

*- MPMC-Ring — ein beschränkter Ringpuffer, in den mehrere Producer pushen und aus dem mehrere Consumer poppen, lock-free und alloktationsfrei. - lock-free — irgendein Thread **kommt** immer voran; kein Thread hält ein Lock, auf das andere warten.*

- MPMC ring — a bounded circular buffer that multiple producers push and multiple consumers pop, lock-free and allocation-free. - lock-free — some thread always makes progress; no thread holds a lock others wait on.

kontrollieren

kontrolliert ·
kontrollierte ·
kontrolliert

T2

to control, govern

*Die Logik stimmt. — Falsch ist das eine, was die Logik nicht **kontrolliert**: die Reihenfolge, in der zwei Cores Änderungen im Speicher sehen.*

The logic is right. — What's wrong is the one thing the logic doesn't **control**: the order in which two cores see memory change.

landen

landet · landete ·
gelandet

T3

to land (write landing in memory)

*Beide Übergaben **landen** auf `seq`. — Übergabe A ist das `seq.store(pos + 1)` des Producers, gelesen vom Gate des Consumers.*

Both hand-offs **land** on `seq`. — Hand-off A is the producer's `seq.store(pos + 1)`, read by the consumer's gate.

lassen

lässt · ließ · gelassen T0

to let, allow

*Nach Teil 2 aufzuhören **lässt** dich mit einem Ring zurück, der logisch korrekt ist; die Teile 3 und 4 sind der Ort, an dem er auf die Hardware und die Verifizierer trifft und an dem die Fehler wohnen, die Tests bestehen.*

Stopping after Part 2 **leaves** you with a ring that's logically correct; Parts 3 and 4 are where it meets the hardware and the verifiers, and where the failures that pass tests live.

laufen

läuft · lief · gelaufen T0

to run (concurrently)

*[Teil 2 — Ein Zähler kann nicht „geschrieben“ sagen.](02_the_sequence.md) Der naheliegende Ring zerreißt in dem Moment, in dem ein Consumer neben einem Producer **läuft**, weil `tail` beim `*Claim*` weiterrückt, vor dem Write.*

[Part 2 — One counter can't say "written".](02_the_sequence.md) The obvious ring tears the instant a consumer **runs** beside a producer, because the tail moves at `*claim*` time, before the write.

leaken

leakt · leakte ·
geleakt

T4

to leak (memory)

*Box<[Cell<T>]> droppt die Zellen, aber eine Zelle hält ein MaybeUninit<T>, und das droppt nichts — es kann nicht wissen, ob es einen Wert hält. Elemente, die beim Drop noch im Ring liegen, **leaken**.*

Box<[Cell<T>]> drops the cells, but a cell holds MaybeUninit<T>, which drops nothing — it can't know whether it holds a value. Items still in the ring when it's dropped **leak**.

leben

lebt · lebte · gelebt

T2

to live, reside (state living in slot)

*Den Zustand eines Slots können > sie nicht beschreiben. — Diese Information muss im Slot **leben**.*

They cannot describe the state of a slot. — That > information has to **live** in the slot.

leeren

leert · leerte ·
geleert

T1

to empty, drain

*Nimm das Lock weg und benenne, wofür es stellvertretend stand: — > (i) Entscheiden, welcher Thread welchen Slot besitzt. (ii) Den payload von dem > Producer, der einen Slot füllt, zu dem Consumer bringen, der ihn **leert**.*

Strip the lock away and state what it was standing in for: — > (i) Decide which thread owns which slot. (ii) Move the payload from the producer > that fills a slot to the consumer that **empties** it.

legen

legt · legte · gelegt

T0

to place, lay

*Die Arbeitsteilung zwischen den drei Instrumenten, ein Drop, das Miris Leak-Checker absegnet, und der Benchmark: **head** und **tail** mit Padding auf getrennte cache lines zu **legen**, bringt die SPSC-Übergabe von ~51 ns auf ~9 ns.*

The division of labor between the three instruments, a Drop that Miri's leak checker approves, and the benchmark: padding **head** and **tail** onto separate cache lines takes the SPSC hand-off from ~51 ns to ~9 ns.

leisten (sich)

leistet sich · leistete
sich · sich geleistet

T1

to afford (cannot afford to stop)

*Ein paar Millionen Operationen pro Sekunde, von beiden Enden gleichzeitig, und auf keiner der beiden Seiten kann es sich ein Thread **leisten** stehenzubleiben — ein Feed-Thread, der hängt, verliert Marktdaten; ein Matcher, der hängt, weitet den Spread bei jedem Symbol, das hinter ihm in der Queue wartet.*

A few million operations a second, from both ends at once, and no thread on either side can **afford** to stop — a feed thread that stalls drops market data; a matcher that stalls widens the spread on every symbol queued behind it.

leiten

leitet · leitete ·
geleitet

T4

to route, direct

*Ein Grund mehr, warum das Design aus Teil 3 jede Übergabe durch **seq** **leitet** — ein Store und ein Load — und den CAS für das behält, was er kann: entscheiden, wer.*

Which is one more reason the Part 3 design **routes** every hand-off through **seq** — a store and a load — and keeps the CAS for what it can do, which is decide who.

lesen

liest · las · gelesen T0

to read (memory)

*Er zählt nur hoch: `pos` (Producer darf schreiben), `pos + 1` (Consumer darf **lesen**), `pos + capacity` (der Producer der nächsten Runde darf schreiben). - `claim` vs. `publish` — ein Ticket auf dem Zähler ziehen vs. den payload über `seq` sichtbar machen.*

It only increases: `pos` (producer may write), `pos + 1` (consumer may **read**), `pos + capacity` (next lap's producer may write). - `claim` vs `publish` — taking a ticket on the counter vs making the payload visible through `seq`.

liefern

liefert · lieferte · geliefert T1

to return, yield

*Und eine `try_`-API: `try_push` auf einem vollen Ring gibt den Wert zurück; `try_pop` auf einem leeren Ring **liefert** `None`.*

And a `try_` API: `try_push` on a full ring hands the value back; `try_pop` on an empty ring **returns** `None`.

liegen

liegt · lag · gelegen T1

to lie, reside

*Damit **liegt** der Durchsatz in der Hand des Allocators, und der aus `std` ist ohnehin schon dem Namen nach `single-consumer`.*

That hands throughput to the allocator, and `std`'s is `single-consumer` by name anyway.

lohn(n) (sich)

lohnt sich · lohnte sich · sich gelohnt T3

to be worth it

*## Boden und Dach — Beide Orderings sind einseitig, und es **lohnt sich**, genau festzuhalten, welche Seite.*

Floor and roof — The two orderings are one-sided, and it's **worth** being exact about which side.

lügen

lügt · log · gelogen T0

to lie (green passing status hiding bug)

*[Teil 4 — Der Beweis: zwei Werkzeuge und ein **lügendes** Grün.] (04_proving_it.md) Miri nennt den Race beim Namen; loom, mit dem Relaxed-Ring gefüttert, explodiert, statt zu antworten.*

[Part 4 — Proving it: two tools, and a **lying** green.](04_proving_it.md) Miri names the race; loom, given the Relaxed ring, explodes instead of answering.

machen

macht · machte · gemacht T1

to make (progress)

*Die Garantie, die wir wollen: Zu jedem Zeitpunkt **macht** *irgendein* Thread Fortschritt, und ein Thread, den das OS mitten in einer Operation unterbricht — und der dabei nichts hält — kann den Rest nicht einfrieren. - Viele-zu-viele, und nicht-blockierend. N Producer, M Consumer, keine festen Rollen.*

The guarantee we want is that at any moment *some* thread is making progress, and a thread the OS preempts mid-operation — holding nothing — cannot freeze the rest. - Many-to-many, and non-blocking. N producers, M consumers, no fixed roles.

melden

meldet · meldete ·
gemeldet

T0

to report (bugs / state)

Zwei pro Slot: A (`publish`, `Producer` → `Consumer`) und B (`vacate`, `Consumer` → `Producer` der nächsten Runde). - `compare_exchange` (CAS) — atomares Read-Modify-Write, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so beansprucht genau ein Thread ein Ticket. - `Relaxed` / `Acquire` / `Release` — `Relaxed` ist atomar ohne Ordering; `Release` auf einem Store ist ein Boden unter allem, was davor kommt; `Acquire` auf einem Load ist ein Dach über allem, was danach kommt; ein `Acquire`-Load, der einen `Release`-Store beobachtet, verbindet die beiden. - `happens-before` — die threadübergreifende Garantie, die ein `Release`/`Acquire`-Paar aufbaut; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - `False Sharing` — zwei unabhängige Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores invalidiert; behoben durch `CachePadded`. - `loom` — ein Model Checker, der einen kleinen Test unter jeder Verschränkung ausführt; blind für Übergaben, die über einen CAS laufen. - `Miri` — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB **meldet** — Data Races, uninitialisierte Reads, Leaks.

Two per slot: A (`publish`, `producer` → `consumer`) and B (`vacate`, `consumer` → next lap's producer). - `compare_exchange` (CAS) — atomic read-modify-write that succeeds only if the value is what you expected; how exactly one thread claims a ticket. - `Relaxed` / `Acquire` / `Release` — `Relaxed` is atomic with no ordering; `Release` on a store floors what comes before it; `Acquire` on a load roofs what comes after it; an `Acquire` load that observes a `Release` store joins the two. - `happens-before` — the cross-thread guarantee a `Release`/`Acquire` pair builds; without one, a plain write on one thread and a plain read on another are a data race. - `false sharing` — two independent values on one cache line, each write invalidating the other core's copy; fixed by `CachePadded`. - `loom` — a model checker that runs a small test under every interleaving; blind to hand-offs carried by a CAS. - `Miri` — an interpreter that runs Rust against the memory model and **reports** UB — data races, uninitialized reads, leaks.

messen

misst · maß · gemessen T0

to measure

Gebaut und **gemessen** auf `aarch64` (Apple M2), denn ein schwaches Memory-Modell ist der Ort, an dem sich die Fehler aus Teil 3 zeigen; auf `x86` würde sich der Data Race hinter einer stärkeren Hardware-Garantie verstecken.

Built and **measured** on `aarch64` (Apple M2), because a weak memory model is where the Part 3 failures show; on `x86` the data race would hide behind a stronger hardware guarantee.

modellieren

modelliert ·
modellierte ·
modelliert

T0

to model (concurrency semantics)

Mit den Orderings drin werden beide grün — und das Grün von `loom` ist weniger wert, als es aussieht: Es **modelliert** einen Compare-and-Swap stärker als die Hardware und winkt eine kaputte Übergabe durch, die über einen läuft — gezeigt, indem man eine einzige Zeile ändert.

With the orderings in, both go green — and `loom`'s green is worth less than it looks: it **models** a compare-and-swap more strongly than the hardware, and passes a broken hand-off that runs through one, shown by changing one line.

nachbohren

bohrt nach · bohrte nach · nachgebohrt, trennb.

T4

to probe further, investigate deeply

*Zeit, nicht länger zu vertrauen, sondern **nachzubohren**: Gib loom den *naiven* Ring aus Teil 2 — den, der auf `tail` beansprucht und erst danach schreibt, und von dem wir wissen, dass er einen Slot liest, bevor er geschrieben ist.*

Time to stop trusting and start **probing**: give loom the *naive* ring from Part 2, the one that claims on `tail` and writes afterward, and which we know reads a slot before it's written.

nehmen

nimmt · nahm · genommen

T1

to take (a ticket or lock)

*Ein festes Array von Slots. — Ein Zähler, `tail`, den Producer atomar lesen und hochzählen, um eine Reihe zu beanspruchen — der Wert, den du dabei **genommen** hast, ist dein *Ticket*.*

A fixed array of slots. — One counter, `tail`, that producers atomically read-and-bump to claim a turn — the value you took is your *ticket*.

nennen

nennt · nannte · genannt

T0

to name

*[Teil 4 — Der Beweis: zwei Werkzeuge und ein lügendes Grün.] (04_proving_it.md) Miri **nennt** den Race beim Namen; loom, mit dem **Relaxed-Ring** gefüttert, explodiert, statt zu antworten.*

[Part 4 — Proving it: two tools, and a lying green.](04_proving_it.md) Miri **names** the race; loom, given the **Relaxed** ring, explodes instead of answering.

opfern

opfert · opferte · geopfert

T1

to sacrifice, give up

*Drei der vier **opfern** dasselbe. — Das Lock ist kein Detail dieser Designs; sie sind um das Lock herum gebaut.*

Three of the four **surrender** the same thing. — The lock isn't a detail of these designs; it's what they're built around.

ordnen

ordnet · ordnete · geordnet

T3

to order (memory operations)

*Bei (3) zucken die Leute zusammen. — Der payload-Zugriff sitzt im Ok-Zweig des CAS — sollte das, was entscheidet, ob du den Slot anfassen darfst, nicht auch das Anfassen **ordnen**?*

The one people flinch at is (3). — The payload access sits inside the Ok branch of the CAS — surely the thing that decides whether you may touch the slot should also **order** the touch?

parken

parkt · parkte · geparkt

T1

to park (a thread)

*Die Queue **parkt** nie einen Thread. — Blockieren ist eine Policy, die der Aufrufer vielleicht will, und sie darf nicht fest ins Primitiv eingebaut sein.*

The queue never **parks** a thread. — Blocking is a policy the caller may want, and it must not be baked into the primitive.

passen

passt · passte ·
gepasst

T0

to fit

Die Teile — [Teil 1 — Das Problem, und warum die naheliegenden Queues nicht **passen**.](01_the_problem.md) Drei Randbedingungen — beschränkt und allokatonsfrei, lock-free, Viele-zu-viele und nicht-blockierend — und die Tour durch die Queues, von denen jede eine davon verletzt.

The parts — [Part 1 — The problem, and why the obvious queues don't **fit**.](01_the_problem.md) Three constraints — bounded and allocation-free, lock-free, many-to-many and non-blocking — and the tour of queues that each break one.

platzieren

platziert · platzierte
· platziert

T3

to place (orderings)

Das ist das ganze Werkzeug. — **Platziere** es, indem du an jeder atomaren Op fragst: **Auf welcher Seite liegen die Daten, und deckt das Gate dieser Op diese Seite ab?**

That's the whole tool. — **Place** it by asking, at each atomic op: **which side is the data on, and does this op's gate cover that side?**

poppen

poppt · poppte ·
gepoppt

T0

to pop (dequeue)

- MPMC-Ring — ein beschränkter Ringpuffer, in den mehrere Producer pushen und aus dem mehrere Consumer **poppen**, lock-free und allokatonsfrei. - lock-free — irgendein Thread kommt immer voran; kein Thread hält ein Lock, auf das andere warten.

- MPMC ring — a bounded circular buffer that multiple producers push and multiple consumers **pop**, lock-free and allocation-free. - lock-free — some thread always makes progress; no thread holds a lock others wait on.

prüfen

prüft · prüfte ·
geprüft

T0

to check, test, verify

Die Zahlen in Teil 4 stammen aus den *criterion*-Benchs der Crate, und jede Aussage über Korrektheit ist eine, die loom oder Miri tatsächlich **geprüft** hat.

The numbers in Part 4 are from the crate's *criterion* benches, and every claim about correctness is one loom or Miri actually **checked**.

pushen

pusht · pushte ·
gepusht

T0

to push (enqueue)

- MPMC-Ring — ein beschränkter Ringpuffer, in den mehrere Producer **pushen** und aus dem mehrere Consumer poppen, lock-free und allokatonsfrei. - lock-free — irgendein Thread kommt immer voran; kein Thread hält ein Lock, auf das andere warten.

- MPMC ring — a bounded circular buffer that multiple producers **push** and multiple consumers pop, lock-free and allocation-free. - lock-free — some thread always makes progress; no thread holds a lock others wait on.

reichen

reicht · reichte ·
gereicht

T3

to be sufficient / pass along

Der CAS sitzt zwischen zwei Gates, die die Arbeit erledigen. — Ein AcqRel darauf würde nichts einbringen und bei jedem Claim eine Barriere kosten — auf aarch64 ein *casal*, wo ein *cas* **reicht**.

The CAS sits between two gates that do the work. — An AcqRel on it would buy nothing and cost a barrier on every claim — on aarch64, a *casal* where a *cas* **will do**.

reparieren

repariert · reparierte
· repariert

T0

to fix, repair

*Ein Boolean pro Slot kann das nicht **reparieren** — ein Boolean hat kein Gedächtnis, und Slots werden über Runden hinweg wiederverwendet.*

A per-slot boolean can't **fix** it — a boolean has no memory, and slots are reused across laps.

ruhen (auf)

ruht · ruhte · geruht

T4

to rest upon, depend on

*Das Modell hat nur noch endlich viele Pfade — und ist grün. loom sagt dir auf seine Weise, dass der **Fortschritt** deiner Schleife auf einem Ordering **ruhte**, das du nicht geschrieben hattest.*

The model becomes finite, and green. loom is telling you, in its own way, that your loop's **progress** was **resting** on an ordering you hadn't written.

sagen

sagt · sagte · gesagt

T0

to say, indicate

*[Teil 2 — Ein Zähler kann nicht „geschrieben“ **sagen**.](02_the_sequence.md)
Der naheliegende Ring zerreißt in dem Moment, in dem ein Consumer neben einem Producer läuft, weil *tail* beim **Claim** weiterrückt, vor dem Write.*

[Part 2 — One counter can't **say** "written".](02_the_sequence.md) The obvious ring tears the instant a consumer runs beside a producer, because the tail moves at **claim** time, before the write.

schieben

schiebt · schob ·
geschoben

T1

to push, shove

*# Teil 1 — Das Problem, und warum die naheliegenden Queues nicht passen — Zwölf Feed-Handler, einer pro Core, jeder zieht Orders aus einer anderen Exchange-Session und **schiebt** sie an eine einzige Stelle: den Eingang der Matching Engine.*

Part 1 — The problem, and why the obvious queues don't fit — Twelve feed handlers, one per core, each pulling orders off a different exchange session and **pushing** them into one place: the matching engine's intake.

schließen

schließt · schloss ·
geschlossen

T0

to conclude / close

*Der Reihe nach. — Jeder Teil beginnt dort, wo der vorige aufgehört hat, und **schließt** mit der Frage, die der nächste beantwortet.*

In order. — Each part opens where the previous one stopped and **closes** on the question the next one answers.

schreiben

schreibt · schrieb ·
geschrieben

T0

to write

*[Teil 2 — Ein Zähler kann nicht „**geschrieben**“ sagen.](02_the_sequence.md)
Der naheliegende Ring zerreißt in dem Moment, in dem ein Consumer neben einem Producer läuft, weil *tail* beim **Claim** weiterrückt, vor dem Write.*

[Part 2 — One counter can't say "**written**".](02_the_sequence.md) The obvious ring tears the instant a consumer runs beside a producer, because the tail moves at **claim** time, before the write.

sehen

sieht · sah · gesehen

T2

to see

*Die Logik stimmt. — Falsch ist das eine, was die Logik nicht kontrolliert: die Reihenfolge, in der zwei Cores Änderungen im Speicher **sehen**.*

The logic is right. — What's wrong is the one thing the logic doesn't control: the order in which two cores **see** memory change.

serialisieren (sich)

serialisiert sich ·
serialisierte sich ·
sich serialisiert

T0

to serialize across threads

*Greif zu einem `Mutex<VecDeque>`, und jeder dieser Threads **serialisiert sich** auf einem einzigen Lock-Wort, das zwischen den Cores hin- und herspringt.*

Reach for a `Mutex<VecDeque>` and every one of those threads **serializes** on one lock word that bounces between cores.

signalisieren

signalisiert ·
signalisierte ·
signalisiert

T0

to signal

*Verschiedene Operationen, verschiedene Atomics. - Übergabe (hand-off) — ein Thread ist mit ein paar Bytes fertig und **signalisiert** es, ein anderer holt sie ab: eine Sendeseite und eine Empfangsseite.*

Different operations, different atomics. - hand-off — one thread finishing with some bytes and **signaling**, another picking them up: a sending side and a receiving side.

sinken

sinkt · sank · gesunken

T3

to sink (past a boundary)

*Beide Orderings sind einseitig, und es lohnt sich, genau festzuhalten, welche Seite. — **Release** auf einem Store ist ein Boden unter allem, was in der Programmreihenfolge davor kommt: Nichts darüber darf unter den Store **sinken**.*

The two orderings are one-sided, and it's worth being exact about which side. — **Re**lease on a store is a floor under everything before it in program order: nothing above may **sink** below the store.

sitzen

sitzt · saß · gesessen

T0

to sit, be located

*# Einen lock-freien MPMC-Ring entwerfen — Am schreibenden Ende von allem, was schnell ist — eine Matching Engine, die Orders von einem Dutzend Feed-Threads entgegennimmt, ein Write-Ahead-Log, das viele Writer in einen einzigen Commit bündelt —, **sitzt** eine Queue, in die viele Threads pushen und aus der viele Threads poppen, millionenfach pro Sekunde.*

Designing a lock-free MPMC ring — At the write end of anything fast — a matching engine taking orders off a dozen feed threads, a write-ahead log fanning writers into one commit — there's a queue that many threads push and many threads pop, millions of times a second.

speichern

speichert · speicherte ·
gespeichert

T1

to store (memory store)

*Der Konstruktor prüft die Zweierpotenz also per Assert und **speichert** `mask = capacity - 1` ein einziges Mal.*

So the constructor asserts the power of two and **stores** `mask = capacity - 1` once.

spiegeln

spiegelt · spiegelte ·
gespiegelt

T1

to mirror

*Gib den Producern eine Möglichkeit, die Reihen unter sich zu verteilen, ohne je einen Consumer zu berühren; gib den Consumern das **Spiegelbild** davon; mach die Übergabe des payloads zu einer Sache pro Slot, zwischen den zwei Threads, die sich diesen Slot teilen.*

Give producers a way to allocate turns among themselves that never touches a consumer; give consumers the **mirror** of it; let the payload hand-off be a per-slot matter between the two threads that share that slot.

starten

startet · startete ·
gestartet

T2

to start

- Slot i **startet** mit $seq = i$ — bereit für den Producer, der Ticket i hält. - Der Producer mit Ticket pos darf schreiben, wenn $seq == pos$.

- Slot i **starts** with $seq = i$ — ready for the producer holding ticket i . - The producer with ticket pos may write when $seq == pos$.

stehen

steht · stand ·
gestanden

T2

to stand, be positioned

- $diff == 0$ — der Slot ist bereit für mich. — Ticket beanspruchen. - $diff < 0$ — seq **steht** noch auf $pos - capacity + 1$: Das Element der vorigen Runde liegt im Slot, unkonsumiert.

- $diff == 0$ — the slot is ready for me. — Claim the ticket. - $diff < 0$ — seq is still $pos - capacity + 1$: the previous lap's item is in the slot, unconsumed.

stehenbleiben

bleibt stehen · blieb
stehen ·
stehengeblieben,
trennb.

T1

to stall, come to a halt

Ein paar Millionen Operationen pro Sekunde, von beiden Enden gleichzeitig, und auf keiner der beiden Seiten kann es sich ein Thread leisten

stehenzubleiben — ein Feed-Thread, der hängt, verliert Marktdaten; ein Matcher, der hängt, weitet den Spread bei jedem Symbol, das hinter ihm in der Queue wartet.

A few million operations a second, from both ends at once, and no thread on either side can afford to **stop** — a feed thread that stalls drops market data; a matcher that stalls widens the spread on every symbol queued behind it.

stellen

stellt · stellte ·
gestellt

T3

to pose (a question)

Der Ring **stellt** sie nie. — Jede Übergabe ist ein Atomic, eine Richtung.

The ring never **asks** it. — Each hand-off is one atomic, one direction.

stimmen

stimmt · stimmte ·
gestimmt

T2

to be correct, be right

Die Logik **stimmt**. — Falsch ist das eine, was die Logik nicht kontrolliert: die Reihenfolge, in der zwei Cores Änderungen im Speicher sehen.

The logic is **right**. — What's wrong is the one thing the logic doesn't control: the order in which two cores see memory change.

streiten (um)

streitet · stritt ·
gestritten

T1

to contend for, fight over

Schau hin, wo der echte Konflikt sitzt. — Zwei Producer, die beide pushen wollen, **streiten** nicht um **Daten** — sie streiten darum, wer **an der Reihe** ist.

Look at where the genuine **conflict** lives. — Two producers both wanting to push don't conflict over **data** — they conflict over **a turn**.

stören

stört · störte ·
gestört

T4

to disturb, bother

Diese Präzision ist der Grund, warum Miri zuerst drankommt. — Es fängt auch die anderen Dinge, die eine Queue mit Speicher falsch machen kann — einen uninitialisierten Read, ein Leak, ein Double Free — und Spin-Schleifen **stören** es nicht.

That precision is why Miri comes first. — It also catches the other things a queue can do wrong with memory — an uninitialized read, a leak, a double free — and it's **fine with** spin loops.

teilen (sich)

teilt sich · teilte sich · sich geteilt

T0

to share

*Diese Serie entwirft die Queue, die das nicht tut: einen Ring — beschränkt, alloktationsfrei, lock-free —, den sich N Producer und M Consumer **teilen** und bei dem nur noch die Übergabe selbst etwas kostet.*

This series designs the queue that doesn't: a bounded, allocation-free, lock-free ring **shared** by N producers and M consumers, where the only cost is the hand-off itself.

terminieren

terminiert · terminierte · terminiert

T4

to terminate (loops/algorithms)

*Unter **ReLaxed** sagt nichts, dass es das muss; ein veralteter Wert ist für immer legal, also ist der Pfad, auf dem die Schleife nie **terminiert**, ein echter Pfad, und loom folgt ihm, bis sein Branch-Budget aufgebraucht ist.*

Under **ReLaxed**, nothing says it must; a stale value is legal forever, so the path where the loop never **terminates** is a real path, and loom follows it until its branch budget runs out.

tragen

trägt · trug · getragen

T3

to carry (payload/synchronization)

*Das Argument hat drei Schritte: finden, wo Daten von einem Thread zum anderen wandern; entscheiden, welches Atomic jeden dieser Übergänge **trägt**; und dann **Release** und **Acquire** auf die Seite dieses Atomics legen, die die Daten tatsächlich abdeckt.*

The argument has three steps: find where data crosses threads, decide which atomic **carries** each crossing, then put **Release** and **Acquire** on the side of that atomic that actually covers the data.

trennen

trennt · trennte · getrennt

T0

to separate (turn allocation from payload transfer)

*Das Lock erledigt, wie sich herausstellt, zwei Jobs — Reihen vergeben und Daten übergeben —, und die beiden zu **trennen** ist das ganze Design: ein Array, zwei Ticket-Zähler und eine Maske.*

The lock turns out to be doing two jobs, allocating turns and handing off data, and **separating** them is the whole design: an array, two ticket counters, and a mask.

tun

tut · tat · getan

T0

to do

*Diese Serie entwirft die Queue, die das nicht **tut**: einen Ring — beschränkt, alloktationsfrei, lock-free —, den sich N Producer und M Consumer teilen und bei dem nur noch die Übergabe selbst etwas kostet.*

This series designs the queue that doesn't: a bounded, allocation-free, lock-free ring shared by N producers and M consumers, where the only cost is the hand-off itself.

umgehen

umgeht · umging · umgangen

T1

to bypass, dodge

*Ein unbeschränkter Channel — **std::sync::mpsc** oder irgendeine Queue aus verketteten Knoten — **umgeht** das eine Lock, indem er pro Nachricht einen Knoten alloziert.*

An unbounded channel — **std::sync::mpsc**, or any linked-node queue — **dodges** the single lock by allocating a node per message.

unterbrechen

unterbricht ·
unterbrach ·
unterbrochen

T1

to interrupt, preempt

*Die Garantie, die wir wollen: Zu jedem Zeitpunkt macht *irgendein* Thread Fortschritt, und ein Thread, den das OS mitten in einer Operation*

unterbricht — und der dabei nichts hält — kann den Rest nicht einfrieren. - Viele-zu-viele, und nicht-blockierend. N Producer, M Consumer, keine festen Rollen.

The guarantee we want is that at any moment *some* thread is making progress, and a thread the OS **preempts** mid-operation — holding nothing — cannot freeze the rest. - Many-to-many, and non-blocking. N producers, M consumers, no fixed roles.

validieren

validiert · validierte
· validiert

T3

to validate

*tail.load — ein Anhaltspunkt. — Das Gate **validiert** ihn erneut, und ein veralteter Wert kostet einen zusätzlichen Schleifendurchlauf. Relaxed. 2.*

tail.load — a hint. — The gate re-**validates** it, and a stale value costs one extra loop. Relaxed. 2.

verbinden

verbindet · verband ·
verbunden

T0

to join, connect (Acquire/Release)

*Zwei pro Slot: A (publish, Producer → Consumer) und B (vacate, Consumer → Producer der nächsten Runde). - compare_exchange (CAS) — atomares Read-Modify-Write, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so beansprucht genau ein Thread ein Ticket. - Relaxed / Acquire / Release — Relaxed ist atomar ohne Ordering; Release auf einem Store ist ein Boden unter allem, was davor kommt; Acquire auf einem Load ist ein Dach über allem, was danach kommt; ein Acquire-Load, der einen Release-Store beobachtet, **verbindet** die beiden. - happens-before — die threadübergreifende Garantie, die ein Release/Acquire-Paar aufbaut; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - False Sharing — zwei unabhängige Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores invalidiert; behoben durch CachePadded. - loom — ein Model Checker, der einen kleinen Test unter jeder Verschränkung ausführt; blind für Übergaben, die über einen CAS laufen. - Miri — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB meldet — Data Races, uninitialisierte Reads, Leaks.*

Two per slot: A (publish, producer → consumer) and B (vacate, consumer → next lap's producer). - compare_exchange (CAS) — atomic read-modify-write that succeeds only if the value is what you expected; how exactly one thread claims a ticket. - Relaxed / Acquire / Release — Relaxed is atomic with no ordering; Release on a store floors what comes before it; Acquire on a load roofs what comes after it; an Acquire load that observes a Release store **joins** the two. - happens-before — the cross-thread guarantee a Release/Acquire pair builds; without one, a plain write on one thread and a plain read on another are a data race. - false sharing — two independent values on one cache line, each write invalidating the other core's copy; fixed by CachePadded. - loom — a model checker that runs a small test under every interleaving; blind to hand-offs carried by a CAS. - Miri — an interpreter that runs Rust against the memory model and reports UB — data races, uninitialized reads, leaks.

verdrängen

verdrängt · verdrängte
· verdrängt

T1

to preempt, deschedule

*Und in dem Moment, in dem der Halter des Locks vom OS **verdrängt** wird, wartet jeder andere Thread darauf, dass genau dieser eine Thread wieder eingeplant wird.*

And the instant the holder is **preempted**, every other thread waits for that one thread to be scheduled again.

vergeben

vergift · vergab ·
vergeben

T0

to allocate, assign (turns)

*Das Lock erledigt, wie sich herausstellt, zwei Jobs — Reihen **vergeben** und Daten übergeben —, und die beiden zu trennen ist das ganze Design: ein Array, zwei Ticket-Zähler und eine Maske.*

The lock turns out to be doing two jobs, **allocating** turns and handing off data, and separating them is the whole design: an array, two ticket counters, and a mask.

vergleichen

vergleicht · verglich ·
verglichen

T2

to compare

*Das Gate des Consumers ist das Spiegelbild: gegen $pos + 1$ **vergleichen**; $diff < 0$ heißt leer (der Producer hat noch nicht veröffentlicht), $diff > 0$ heißt, ein anderer Consumer war schneller.*

The consumer's gate is the mirror: **compare** against $pos + 1$; $diff < 0$ means empty (the producer hasn't published), $diff > 0$ means another consumer beat me.

verkleinern

verkleinert ·
verkleinerte ·
verkleinert

T1

to shrink, reduce

*Ein Mutex *pro Slot* **verkleinert** das Lock, ohne es loszuwerden, und bringt ein Problem mit, das das Design vorher nicht hatte: entscheiden, welchen Slot jeder Thread ansteuern soll.*

A mutex *per slot* **shrinks** the lock without removing it, and adds a problem the design didn't have before: deciding which slot each thread should head for.

verlieren

verliert · verlor ·
verloren

T1

to lose (data or CAS)

*Ein paar Millionen Operationen pro Sekunde, von beiden Enden gleichzeitig, und auf keiner der beiden Seiten kann es sich ein Thread leisten stehenzubleiben — ein Feed-Thread, der hängt, **verliert** Marktdaten; ein Matcher, der hängt, weitet den Spread bei jedem Symbol, das hinter ihm in der Queue wartet.*

A few million operations a second, from both ends at once, and no thread on either side can afford to stop — a feed thread that stalls **drops** market data; a matcher that stalls widens the spread on every symbol queued behind it.

verpassen

verpasst · verpasste ·
verpasst

T4

to miss

*Zwei Zähler, die Reihen vergeben, eine Sequenz pro Slot, die den Staffelstab mit Ticketnummer drauf weiterreicht, vier Orderings, platziert je nachdem, wo die Daten liegen, drei Instrumente, von denen jedes fängt, was die anderen **verpassen**, und zwei cache lines, wo eine 40 Nanosekunden gekostet hat.*

Two counters that allocate turns, a sequence per slot that hands the baton around with a ticket number on it, four orderings placed by where the data sits, three instruments that each catch what the others **miss**, and two cache lines where one was costing 40 nanoseconds.

verstecken (sich)

versteckt sich ·
versteckte sich · sich
versteckt

T0

to hide (race hiding behind x86 guarantees)

*Gebaut und gemessen auf **aarch64** (Apple M2), denn ein schwaches Memory-Modell ist der Ort, an dem sich die Fehler aus Teil 3 zeigen; auf x86 würde sich der Data Race hinter einer stärkeren Hardware-Garantie **verstecken**.*

Built and measured on **aarch64** (Apple M2), because a weak memory model is where the Part 3 failures show; on x86 the data race would **hide** behind a stronger hardware guarantee.

vertrauen

vertraut · vertraute ·
vertraut

T2

to trust

*Der Consumer hat nichts falsch gemacht. — Er hat dem einzigen Signal **vertraut**, das er hatte, und das Signal wurde beim Claim gesetzt, nicht beim Write.*

The consumer did nothing wrong. — It **trusted** the only signal it had, and the signal was raised at claim time, not at write time.

vervielfachen

vervielfacht ·
vervielfachte ·
vervielfacht

T4

to multiply

*Auch noch das **seq** jedes Slots zu **padden** ist der naheliegende nächste Schritt — und lohnt sich nicht: Es **vervierfacht** den Speicher eines Rings mit kleinen payloads, um den letzten 1,8 ns hinterherzujagen.*

Padding each slot's seq as well is the obvious next step and isn't worth it: it **quadruples** the memory of a ring of small payloads to chase the last 1.8 ns.

vervierfachen

vervierfacht ·
vervierfachte ·
vervierfacht

T4

to quadruple

*Auch noch das **seq** jedes Slots zu **padden** ist der naheliegende nächste Schritt — und lohnt sich nicht: Es **vervierfacht** den Speicher eines Rings mit kleinen payloads, um den letzten 1,8 ns hinterherzujagen.*

Padding each slot's seq as well is the obvious next step and isn't worth it: it **quadruples** the memory of a ring of small payloads to chase the last 1.8 ns.

verweigern

verweigert ·
verweigerte ·
verweigert

T1

to refuse, reject

*Ein einzelnes Lock lässt jeden Thread jedes Mal auf beide Aufgaben warten: Ein Producer blockiert einen Consumer, der einen völlig anderen Slot will, weil sie sich das Lock-Wort teilen. Das ganze Design ergibt sich daraus, genau das zu **verweigern**.*

A single lock makes every thread wait for both jobs, every time: a producer blocks a consumer that wants a completely different slot, because they share the lock word. The whole design falls out of **refusing** that.

veröffentlichen

veröffentlicht ·
veröffentlichte ·
veröffentlicht

T2

to publish (payload visibility)

*## Beanspruchen ist nicht **Veröffentlichen** — Das **compare_exchange** des Producers bewegt **tail** von 0 auf 1 in dem Moment, in dem das Ticket ***beansprucht*** wird — und dann wird der Producer vom OS unterbrochen, noch vor dem Write:*

Claiming is not publishing — The producer's **compare_exchange** moves **tail** from 0 to 1 the moment the ticket is ***claimed*** — and then the producer gets preempted, before the write:

voraussetzen

setzt voraus · setzte voraus · vorausgesetzt, trennb.

T0

to assume, presuppose

*Kein Lock-free-Hintergrund wird **vorausgesetzt**; compare_exchange, Acquire/Release, UnsafeCell, MaybeUninit, loom, Miri und CachePadded werden dort eingeführt, wo das Design mit ihnen kollidiert.*

No lock-free background is **assumed**; compare_exchange, Acquire/Release, UnsafeCell, MaybeUninit, loom, Miri and CachePadded are introduced where the design collides with them.

warten

wartet · wartete · gewartet

T0

to wait

*- MPMC-Ring — ein beschränkter Ringpuffer, in den mehrere Producer pushen und aus dem mehrere Consumer poppen, lock-free und alloktationsfrei. - lock-free — irgendein Thread kommt immer voran; kein Thread hält ein Lock, auf das andere **warten**.*

- MPMC ring — a bounded circular buffer that multiple producers push and multiple consumers pop, lock-free and allocation-free. - lock-free — some thread always makes progress; no thread holds a lock others **wait** on.

wechseln

wechselt · wechselte · gewechselt

T0

to cross, switch (threads)

*Die Orderings werden hergeleitet, nicht geraten: die zwei Stellen finden, an denen Daten den Thread **wechseln**, und dann bemerken, dass der payload nur über seq veröffentlicht werden kann, nie über den Zähler — ein Release legt einen Boden unter alles, was *davor* kommt, und der Claim kommt vor dem Write.*

The orderings are derived, not guessed: find the two places data **crosses** threads, then notice the payload can only be published through seq, never through the counter — a Release floors what comes *before* it, and the claim comes before the write.

weiten

weitert · weitete · geweitet

T1

to widen (spread)

*Ein paar Millionen Operationen pro Sekunde, von beiden Enden gleichzeitig, und auf keiner der beiden Seiten kann es sich ein Thread leisten stehenzubleiben — ein Feed-Thread, der hängt, verliert Marktdaten; ein Matcher, der hängt, **weitert** den Spread bei jedem Symbol, das hinter ihm in der Queue wartet.*

A few million operations a second, from both ends at once, and no thread on either side can afford to stop — a feed thread that stalls drops market data; a matcher that stalls **widens** the spread on every symbol queued behind it.

weiterreichen

reicht weiter · reichte weiter · weitergereicht, trennb.

T2

to pass on (baton)

*Der Slot **reicht einen Staffelstab im Kreis weiter**, und auf dem Staffelstab steht die Ticketnummer.*

The slot **passes a baton around** a cycle, and the baton has the ticket number written on it.

weiterrücken

rückt weiter · rückte weiter · weitergerückt, trennb.

T0

to move forward, advance

*[Teil 2 — Ein Zähler kann nicht „geschrieben“ sagen.](02_the_sequence.md) Der naheliegende Ring zerreißt in dem Moment, in dem ein Consumer neben einem Producer läuft, weil tail beim *Claim* **weiterrückt**, vor dem Write.*

[Part 2 — One counter can't say "written".](02_the_sequence.md) The obvious ring tears the instant a consumer runs beside a producer, because the tail **moves** at *claim* time, before the write.

werfen

wirft · warf · geworfen T1

to throw

*Das sind zwei verschiedene Aufgaben, und ein Lock **wirft** sie in einen Topf. — Schau hin, wo der echte Konflikt sitzt.*

Those are two different jobs, and a lock **conflates** them. — Look at where the genuine conflict lives.

wiederverwenden

verwendet wieder ·
verwendete wieder ·
wiederverwendet,
trennb. T0

to reuse (slots across laps)

*Ein Boolean pro Slot kann das nicht reparieren — ein Boolean hat kein Gedächtnis, und Slots werden über Runden hinweg **wiederverwendet**.*

A per-slot boolean can't fix it — a boolean has no memory, and slots are **reused** across laps.

winken

winkt · winkte ·
gewinkt T0

to wave through (pass falsely)

*Mit den Orderings drin werden beide grün — und das Grün von loom ist weniger wert, als es aussieht: Es modelliert einen Compare-and-Swap stärker als die Hardware und **winkt eine kaputte Übergabe durch**, die über einen läuft — gezeigt, indem man eine einzige Zeile ändert.*

With the orderings in, both go green — and loom's green is worth less than it looks: it models a compare-and-swap more strongly than the hardware, and **passes** a broken hand-off that runs through one, shown by changing one line.

wissen

weiß · wusste · gewusst T2

to know

*P2 aus der Boolean-Geschichte findet auf Slot 0 jetzt $seq = 0$ — nicht 2 — und **weiß**, dass Ticket 0 noch nicht einmal geschrieben ist, geschweige denn konsumiert.*

P2 from the boolean story now finds $seq = 0$ on slot 0 — not 2 — and **knows** ticket 0 hasn't even been written, let alone consumed.

wohnen

wohnt · wohnte ·
gewohnt T0

to live, reside (where bugs live)

*Nach Teil 2 aufzuhören lässt dich mit einem Ring zurück, der logisch korrekt ist; die Teile 3 und 4 sind der Ort, an dem er auf die Hardware und die Verifizierer trifft und an dem die Fehler **wohnen**, die Tests bestehen.*

Stopping after Part 2 leaves you with a ring that's logically correct; Parts 3 and 4 are where it meets the hardware and the verifiers, and where the failures that pass tests **live**.

zahlen

zahlt · zahlte ·
gezahlt T4

to pay (in latency)

*Der Consumer schreibt head, und das invalidiert die des Producers. — Keiner der beiden Threads liest auf dem Hot Path je den Zähler des anderen, und trotzdem **zahlen** sie für die Writes des jeweils anderen.*

The consumer writes head, which invalidates the producer's. — Neither thread ever reads the other's counter on the hot path, and they **pay** for each other's writes anyway.

zeigen

zeigt · zeigte ·
gezeigt

T0

to show, reveal

*Gebaut und gemessen auf **aarch64** (Apple M2), denn ein schwaches Memory-Modell ist der Ort, an dem sich die Fehler aus Teil 3 **zeigen**; auf x86 würde sich der Data Race hinter einer stärkeren Hardware-Garantie verstecken.*

Built and measured on **aarch64** (Apple M2), because a weak memory model is where the Part 3 failures **show**; on x86 the data race would hide behind a stronger hardware guarantee.

zerreißen

zerreißt · zerriss ·
zerrissen

T0

to tear (data tearing / race)

*[Teil 2 — Ein Zähler kann nicht „geschrieben“ sagen.](02_the_sequence.md) Der naheliegende Ring **zerreißt** in dem Moment, in dem ein Consumer neben einem Producer läuft, weil **tail** beim **Claim** weiterrückt, vor dem Write.*

[Part 2 — One counter can't say "written".](02_the_sequence.md) The obvious ring **tears** the instant a consumer runs beside a producer, because the tail moves at **claim** time, before the write.

ziehen

zieht · zog · gezogen

T0

to pull, draw (a ticket)

*Er zählt nur hoch: **pos** (Producer darf schreiben), **pos + 1** (Consumer darf lesen), **pos + capacity** (der Producer der nächsten Runde darf schreiben). - *claim* vs. *publish* — ein Ticket auf dem Zähler **ziehen** vs. den payload über *seq* sichtbar machen.*

It only increases: **pos** (producer may write), **pos + 1** (consumer may read), **pos + capacity** (next lap's producer may write). - *claim* vs *publish* — taking a ticket on the counter vs making the payload visible through *seq*.

zucken

zuckt · zuckte ·
gezuckt

T3

to flinch

*Bei (3) **zucken** die Leute zusammen. — Der payload-Zugriff sitzt im *Ok*-Zweig des CAS — sollte das, was entscheidet, ob du den Slot anfassen darfst, nicht auch das Anfassen ordnen?*

The one people **flinch** at is (3). — The payload access sits inside the *Ok* branch of the CAS — surely the thing that decides whether you may touch the slot should also order the touch?

zurückfalten

faltet zurück · faltete
zurück ·
zurückgefaltet, trennb.

T1

to fold back (modulo masking)

*Keiner der beiden Zähler nimmt je ab, also laufen sie fast sofort über die Array-Länge hinaus, und ein Ticket wird auf einen physischen Slot abgebildet, indem man es **zurückfaltet**: **slot = ticket & (capacity - 1)**.*

Neither counter ever decreases, so they run past the array length almost immediately, and a ticket maps to a physical slot by **folding it back**: **slot = ticket & (capacity - 1)**.

zurückgeben

gibt zurück · gab
zurück · zurückgegeben,
trennb.

T1

to return, hand back

*Und eine **try_**-API: **try_push** auf einem vollen Ring **gibt den Wert zurück**; **try_pop** auf einem leeren Ring liefert **None**.*

And a **try_** API: **try_push** on a full ring **hands the value back**; **try_pop** on an empty ring returns **None**.

zurückkehren

kehrt zurück · kehrte zurück · zurückgekehrt, trennb.

T0

to return (from try API)

*Ein Drei-Wege-Gate, die **publish**- und **vacate**-Regeln und der Vertrag darüber, wann **try_push** erneut versucht und wann es **zurückkehrt**.*

A three-way gate, the publish and vacate rules, and the contract for when **try_push** retries and when it **returns**.

zusammenführen

führt zusammen · führte zusammen · zusammengeführt, trennb.

T1

to funnel together, aggregate

*Ein Write-Ahead-Log bündelt ein Dutzend Writer-Threads in einen Group Commit. — Eine Metrics-Pipeline führt Samples von jedem Worker in einem einzigen Aggregator **zusammen**.*

A write-ahead log fans a dozen writer threads into one group commit. — A metrics pipeline **funnels** samples from every worker into one aggregator.

zählen

zählt · zählte · gezählt

T0

to count

*Er **zählt** nur hoch: **pos** (Producer darf schreiben), **pos + 1** (Consumer darf lesen), **pos + capacity** (der Producer der nächsten Runde darf schreiben). - **claim** vs. **publish** — ein Ticket auf dem Zähler ziehen vs. den **payload** über **seq** sichtbar machen.*

It only increases: **pos** (producer may write), **pos + 1** (consumer may read), **pos + capacity** (next lap's producer may write). - **claim** vs **publish** — taking a ticket on the counter vs making the payload visible through **seq**.

Adjektive & Adverbien

adjectives and adverbs

88

abgeschlossen

T3

completed, finished (read/write)

*Danach wird dieser Thread den **payload** *schreiben*. — Dieser Write darf nicht beginnen, bevor der Read des vorherigen Consumers auf dieselben Bytes **abgeschlossen** ist — Übergabe B.*

After it, this thread will *write* the payload. — That write must not begin before the previous consumer's read of the same bytes has **finished** — hand-off B.

aktuell

T2

current, up-to-date

*Nur der Producer, der Ticket **pos** hält, kann **seq** von **pos** auf **pos + 1** bewegen, und wenn mein CAS auf **tail** von **pos** aus gelingt, dann bin ich dieser Producer. Ein erfolgreicher Claim beweist im Nachhinein, dass der Gate-Read **aktuell** war.*

Only the producer holding ticket **pos** can move **seq** from **pos** to **pos + 1**, and if my CAS on **tail** succeeds from **pos**, that producer is me. A successful claim retroactively proves the gate read was **current**.

allein

T2

alone — allein gewinnen = to win alone

*Das nächste Ticket ist frei, niemand muss etwas tun, damit ich es bekomme, also läuft die Schleife mit dem neuen **tail** weiter. Hier in der Schleife zu bleiben ist ein Wettlauf, den ich **allein** gewinnen kann.*

The next ticket is free, nobody has to do anything for me to get it, so the loop continues with the new **tail**. Looping here is a race I can win **alone**.

allokationsfrei

T0

allocation-free (zero heap allocations)

*Diese Serie entwirft die Queue, die das nicht tut: einen Ring — beschränkt, **allokationsfrei**, lock-free —, den sich N Producer und M Consumer teilen und bei dem nur noch die Übergabe selbst etwas kostet.*

This series designs the queue that doesn't: a bounded, **allocation-free**, lock-free ring shared by N producers and M consumers, where the only cost is the hand-off itself.

andere

T0

other, different

*Boden und Dach, ein Gate nach dem **anderen**, warum der CAS ReLaxed bleibt, und warum dieser Ring — anders als ein SeqLock — keinen Fence braucht.*

Floor and roof, one gate at a time, why the CAS stays ReLaxed, and why this ring — unlike a SeqLock — needs no fence.

atomar

T0

atomic (hardware/memory atomic)

*Zwei pro Slot: A (publish, Producer → Consumer) und B (vacate, Consumer → Producer der nächsten Runde). - compare_exchange (CAS) — atomares Read-Modify-Write, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so beansprucht genau ein Thread ein Ticket. - Relaxed / Acquire / Release — Relaxed ist **atomar** ohne Ordering; Release auf einem Store ist ein Boden unter allem, was davor kommt; Acquire auf einem Load ist ein Dach über allem, was danach kommt; ein Acquire-Load, der einen Release-Store beobachtet, verbindet die beiden. - happens-before — die threadübergreifende Garantie, die ein Release/Acquire-Paar aufbaut; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - False Sharing — zwei unabhängige Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores invalidiert; behoben durch CachePadded. - loom — ein Model Checker, der einen kleinen Test unter jeder Verschränkung ausführt; blind für Übergaben, die über einen CAS laufen. - Miri — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB meldet — Data Races, uninitialisierte Reads, Leaks.*

Two per slot: A (publish, producer → consumer) and B (vacate, consumer → next lap's producer). - compare_exchange (CAS) — **atomic** read-modify-write that succeeds only if the value is what you expected; how exactly one thread claims a ticket. - Relaxed / Acquire / Release — Relaxed is atomic with no ordering; Release on a store floors what comes before it; Acquire on a load roofs what comes after it; an Acquire load that observes a Release store joins the two. - happens-before — the cross-thread guarantee a Release/Acquire pair builds; without one, a plain write on one thread and a plain read on another are a data race. - false sharing — two independent values on one cache line, each write invalidating the other core's copy; fixed by CachePadded. - loom — a model checker that runs a small test under every interleaving; blind to hand-offs carried by a CAS. - Miri — an interpreter that runs Rust against the memory model and reports UB — data races, uninitialized reads, leaks.

aussagekräftig

T2

meaningful, informative (wrapping math)

*## Das Drei-Wege-Gate — Der Vergleich ist eine vorzeichenbehaftete Differenz, `seq.wrapping_sub(pos) as isize`, damit er **aussagekräftig** bleibt, falls die Zähler je überlaufen.*

*## The three-way gate — The comparison is a signed difference, `seq.wrapping_sub(pos) as isize`, so it stays **meaningful** if the counters ever wrap.*

bedeutend

T1

significant

*Ein Allocator-Aufruf **bedeutet** an einem guten Tag hunderte Nanosekunden und an einem schlechten ein prozessweites Lock; so oder so passt er nicht in eine Übergabe, die in unter einer Mikrosekunde fertig sein muss. - Lock-free. Kein einzelnes Speicherwort, das sich jeder Thread erst holen muss.*

An allocator call is hundreds of nanoseconds on a good day and a process-wide lock on a bad one; either way it doesn't fit inside a hand-off that has to complete in under a microsecond. - Lock-free. No single word every thread must acquire.

benachbart

T4

adjacent (neighboring atomic variables)

*Von verschiedenen Threads angefasst — und direkt nebeneinander. — `head` und `tail` sind zwei **benachbarte** `AtomicUsize` — sechzehn Bytes, eine cache line.*

*Touched by different threads, and sitting next to each other. — `head` and `tail` are two **adjacent** `AtomicUsize`s — sixteen bytes, one cache line.*

benutzbar

T2

usable (slot ready for access)

*Das Gate wird ***vor*** dem `compare_exchange` auf dem Zähler gelesen, und das muss so sein: Ein einmal genommenes Ticket kannst du nicht zurückgeben, also musst du vorher wissen, dass der Slot **benutzbar** ist.*

*The gate is read ***before*** the `compare_exchange` on the counter, and it has to be: once you've taken a ticket you can't give it back, so you must know the slot is **usable** first.*

bereit

T0

ready (for a ticket / operation)

*Nicht wait-free: Ein einzelner Thread kann es erneut versuchen müssen. - Ticket — der Wert, den ein Thread von `tail` (Producer) oder `head` (Consumer) zieht; `ticket & (capacity - 1)` ist sein Slot. - Slot / Zelle — ein Element des Puffers: ein payload und seine Sequenznummer. - `seq` — der Zähler pro Slot, der sagt, für welches Ticket der Slot **bereit** ist.*

*Not wait-free: an individual thread may retry. - ticket — the value a thread takes from `tail` (producer) or `head` (consumer); `ticket & (capacity - 1)` is its slot. - slot / cell — one element of the buffer: a payload and its sequence number. - `seq` — the per-slot counter that says which ticket the slot is **ready** for.*

beschränkt

T0

bounded (fixed ring capacity)

*Diese Serie entwirft die Queue, die das nicht tut: einen Ring — **beschränkt**, alloktationsfrei, lock-free —, den sich N Producer und M Consumer teilen und bei dem nur noch die Übergabe selbst etwas kostet.*

*This series designs the queue that doesn't: a **bounded**, allocation-free, lock-free ring shared by N producers and M consumers, where the only cost is the hand-off itself.*

billig

billiger · am
billigsten

T1

cheap — am billigsten = cheapest

*Das Lock ist kein Detail dieser Designs; sie sind um das Lock herum gebaut. — Die Frage ist also nicht „welches Lock ist am **billigsten**“ — sondern was das Lock eigentlich *tut*, und ob sich diese Arbeit auch ohne es erledigen lässt.*

The lock isn't a detail of these designs; it's what they're built around. — So the question isn't "which lock is **cheapest**" — it's what the lock is actually *doing*, and whether that job can be done without it.

blind (für)

T0

blind to (imperfections/races)

*Zwei pro Slot: A (publish, Producer → Consumer) und B (vacate, Consumer → Producer der nächsten Runde). - compare_exchange (CAS) — atomares Read-Modify-Write, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so beansprucht genau ein Thread ein Ticket. - Relaxed / Acquire / Release — Relaxed ist atomar ohne Ordering; Release auf einem Store ist ein Boden unter allem, was davor kommt; Acquire auf einem Load ist ein Dach über allem, was danach kommt; ein Acquire-Load, der einen Release-Store beobachtet, verbindet die beiden. - happens-before — die threadübergreifende Garantie, die ein Release/Acquire-Paar aufbaut; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - False Sharing — zwei unabhängige Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores invalidiert; behoben durch CachePadded. - loom — ein Model Checker, der einen kleinen Test unter jeder Verschränkung ausführt; **blind** für Übergaben, die über einen CAS laufen. - Miri — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB meldet — Data Races, uninitialisierte Reads, Leaks.*

Two per slot: A (publish, producer → consumer) and B (vacate, consumer → next lap's producer). - compare_exchange (CAS) — atomic read-modify-write that succeeds only if the value is what you expected; how exactly one thread claims a ticket. - Relaxed / Acquire / Release — Relaxed is atomic with no ordering; Release on a store floors what comes before it; Acquire on a load roofs what comes after it; an Acquire load that observes a Release store joins the two. - happens-before — the cross-thread guarantee a Release/Acquire pair builds; without one, a plain write on one thread and a plain read on another are a data race. - false sharing — two independent values on one cache line, each write invalidating the other core's copy; fixed by CachePadded. - loom — a model checker that runs a small test under every interleaving; **blind** to hand-offs carried by a CAS. - Miri — an interpreter that runs Rust against the memory model and reports UB — data races, uninitialized reads, leaks.

blockierend

T1

blocking (holding thread in wait state)

*Die Garantie, die wir wollen: Zu jedem Zeitpunkt macht *irgendein* Thread Fortschritt, und ein Thread, den das OS mitten in einer Operation unterbricht — und der dabei nichts hält — kann den Rest nicht einfrieren. - Viele-zu-viele, und nicht-**blockierend**. N Producer, M Consumer, keine festen Rollen.*

The guarantee we want is that at any moment *some* thread is making progress, and a thread the OS preempts mid-operation — holding nothing — cannot freeze the rest. - Many-to-many, and non-**blocking**. N producers, M consumers, no fixed roles.

echt

T1

genuine, real (real contention)

*Das sind zwei verschiedene Aufgaben, und ein Lock wirft sie in einen Topf. — Schau hin, wo der **echte** Konflikt sitzt.*

Those are two different jobs, and a lock conflates them. — Look at where the **genuine** conflict lives.

eigen

T1

own, distinct (each thread has its own slot)

*Jeder braucht einen **eigenen** Slot, und es ist egal, wer welchen bekommt. — Zwei Consumer: dasselbe.*

Each needs a **distinct** slot, and it doesn't matter who gets which. — Two consumers, the same.

eigenständig

T3

standalone (independent fence)

*## Warum kein Fence — der Unterschied zum SeqLock — Der [SeqLock] (../seqlock/de/03_memory_ordering.md) brauchte zusätzlich zu seinen Orderings zwei **eigenständige** fences.*

Why no fence — the difference from a SeqLock — The [SeqLock] (../seqlock/en/03_memory_ordering.md) needed two **standalone** fences on top of its orderings.

einfach

T0

simple, plain (non-atomic access)

*Es ist eine Design-Untersuchung, kein Tutorial. — Jede Entscheidung ist erzwungen — durch einen Use Case, durch das Scheitern der **einfacheren** Variante oder durch ein Verifikationswerkzeug.*

It's a design investigation, not a tutorial. — Every decision is forced — by a use case, by the **simpler** version failing, or by a verification tool.

einseitig

T3

one-sided (Acquire or Release barrier)

*## Boden und Dach — Beide Orderings sind **einseitig**, und es lohnt sich, genau festzuhalten, welche Seite.*

Floor and roof — The two orderings are **one-sided**, and it's worth being exact about which side.

einzel

T0

individual, single

*Nicht wait-free: Ein **einzelner** Thread kann es erneut versuchen müssen. - Ticket — der Wert, den ein Thread von **tail** (Producer) oder **head** (Consumer) zieht; **ticket & (capacity - 1)** ist sein Slot. - Slot / Zelle — ein Element des Puffers: ein payload und seine Sequenznummer. - **seq** — der Zähler pro Slot, der sagt, für welches Ticket der Slot bereit ist.*

Not wait-free: an **individual** thread may retry. - ticket — the value a thread takes from **tail** (producer) or **head** (consumer); **ticket & (capacity - 1)** is its slot. - slot / cell — one element of the buffer: a payload and its sequence number. - seq — the per-slot counter that says which ticket the slot is ready for.

endlich

T4

finite (finite state machine paths)

*Das Modell hat nur noch **endlich** viele Pfade — und ist grün. loom sagt dir auf seine Weise, dass der ***Fortschritt*** deiner Schleife auf einem Ordering ruhte, das du nicht geschrieben hattest.*

The model becomes **finite**, and green. loom is telling you, in its own way, that your loop's ***progress*** was resting on an ordering you hadn't written.

erfolgreich

T2

successful (successful CAS claim)

*Nur der Producer, der Ticket `pos` hält, kann `seq` von `pos` auf `pos + 1` bewegen, und wenn mein CAS auf `tail` von `pos` aus gelingt, dann bin ich dieser Producer. Ein **erfolgreicher** Claim beweist im Nachhinein, dass der Gate-Read aktuell war.*

Only the producer holding ticket `pos` can move `seq` from `pos` to `pos + 1`, and if my CAS on `tail` succeeds from `pos`, that producer is me. A **successful** claim retroactively proves the gate read was current.

falsch

T2

wrong, incorrect

*Der Consumer hat nichts **falsch** gemacht. — Er hat dem einzigen Signal vertraut, das er hatte, und das Signal wurde beim Claim gesetzt, nicht beim Write.*

The consumer did nothing **wrong**. — It trusted the only signal it had, and the signal was raised at claim time, not at write time.

fertig

T0

finished, done

*Verschiedene Operationen, verschiedene Atomics. - Übergabe (hand-off) — ein Thread ist mit ein paar Bytes **fertig** und signalisiert es, ein anderer holt sie ab: eine Sendeseite und eine Empfangsseite.*

Different operations, different atomics. - hand-off — one thread **finishing** with some bytes and signaling, another picking them up: a sending side and a receiving side.

fest

T1

fixed (fixed array, no allocation)

*Die Queue parkt nie einen Thread. — Blockieren ist eine Policy, die der Aufrufer vielleicht will, und sie darf nicht **fest** ins Primitiv eingebaut sein.*

The queue never parks a thread. — Blocking is a policy the caller may want, and it must not be baked into the primitive.

frei

T2

free, unallocated

*Der Fix stammt von Dmitry Vyukov, und er besteht aus einem Integer pro Slot: `seq`. — Er sagt nicht „**frei**“ oder „voll“.*

The fix is Dmitry Vyukov's, and it's one integer per slot: `seq`. — It doesn't say "free" or "full".

frisch

T4

fresh, up-to-date

*Das ist ein Urteil, nur ein indirektes. — Die Gate-Schleife aus Teil 2 — `diff > 0`, Zähler neu laden, erneut versuchen — geht davon aus, dass das Neuladen irgendwann einen **frischen** Wert sieht.*

This is a verdict, just an indirect one. — The gate loop from Part 2 — `diff > 0`, reload the counter, retry — assumes the reload eventually sees a **fresh** value.

gelegentlich

T3

occasionally

*Auf dem M2 ist das Symptom ein Consumer, der **gelegentlich** den vorherigen Inhalt des Slots liest; in Rusts Memory-Modell ist es undefiniertes Verhalten, noch bevor es ein falscher Wert ist.*

On the M2 the symptom is a consumer that **occasionally** reads the slot's previous contents; in Rust's memory model it's undefined behavior before it's a wrong value.

genau

T1

exact, precise

*Zwölf Feed-Handler, einer pro Core, jeder zieht Orders aus einer anderen Exchange-Session und schiebt sie an eine einzige Stelle: den Eingang der Matching Engine. Auf der anderen Seite poppt ein Pool von Matching-Threads aus **genau** diesem Eingang.*

Twelve feed handlers, one per core, each pulling orders off a different exchange session and pushing them into one place: the matching engine's intake. On the other side, a pool of matching threads pops from that same intake.

generisch

T0

generic (type parameter <T>)

*## Umfang — Ein **generischer**, beschränkter MpmcRing<T> — der Vyukov-Algorithmus, so wie man ihn in eine Concurrency-Crate legen würde.*

*## Scope — A **generic** bounded MpmcRing<T> — the Vyukov algorithm, as you'd put it in a concurrency crate.*

gerade

T2

just now, currently

*Die Menge der tatsächlich geschriebenen Slots ist {1}. — Kein einzelner Zähler kann „Slot 1 ist geschrieben, Slot 0 nicht“ ausdrücken, denn Claims werden der Reihe nach vergeben, und Writes werden in der Reihenfolge fertig, die dem Scheduler **gerade** gefällt.*

The set of slots actually written is {1}. — No single counter can express "slot 1 is written, slot 0 isn't", because claims are handed out in order and writes complete in whatever order the scheduler likes.

getrennt

T0

separate (padded cache lines)

*Die Arbeitsteilung zwischen den drei Instrumenten, ein Drop, das Miris Leak-Checker absegnet, und der Benchmark: `head` und `tail` mit Padding auf **getrennte** cache lines zu legen, bringt die SPSC-Übergabe von ~51 ns auf ~9 ns.*

The division of labor between the three instruments, a Drop that Miri's leak checker approves, and the benchmark: padding head and tail onto **separate** cache lines takes the SPSC hand-off from ~51 ns to ~9 ns.

gleich

T2

same, identical

*## Der Vertrag: `try` heißt versuchen — Die letzte Frage ist, was zu tun ist, wenn etwas nicht bereit ist, und es gibt zwei Fälle, die **gleich** aussehen und es nicht sind.*

*## The contract: `try` means try — The last question is what to do when something isn't ready, and there are two cases that look the **same** and aren't.*

gleichzeitig

T1

simultaneous, at the same time

*Wäre Durchsatz die einzige Anforderung, wäre das ein Benchmark und kein Designproblem. Es ist ein Designproblem, weil die Queue drei Dinge **gleichzeitig** einhalten muss:*

If throughput were the only requirement this would be a benchmark, not a design problem. It's a design problem because the queue has to hold three things **at once**:

heiß

heißer · am heißesten T1

hot (hot execution path)

*Mitten im **heißesten** Pfad des Systems sitzt eine Queue, und das Erste, wonach jeder greift, ist `Mutex<VecDeque<T>>`.*

In the middle of the **hottest** path in the system sits a queue, and the first thing anyone reaches for is `Mutex<VecDeque<T>>`.

indirekt

T4

indirect (indirect verdict)

*Das ist ein Urteil, nur ein **indirektes**. — Die Gate-Schleife aus Teil 2 — `diff > 0`, Zähler neu laden, erneut versuchen — geht davon aus, dass das Neuladen irgendwann einen frischen Wert sieht.*

This is a verdict, just an **indirect** one. — The gate loop from Part 2 — `diff > 0`, reload the counter, retry — assumes the reload eventually sees a fresh value.

kaputt

T0

broken

*Mit den Orderings drin werden beide grün — und das Grün von loom ist weniger wert, als es aussieht: Es modelliert einen Compare-and-Swap stärker als die Hardware und winkt eine **kaputte** Übergabe durch, die über einen läuft — gezeigt, indem man eine einzige Zeile ändert.*

With the orderings in, both go green — and loom's green is worth less than it looks: it models a compare-and-swap more strongly than the hardware, and passes a **broken** hand-off that runs through one, shown by changing one line.

klein

kleiner · am kleinsten T0

small (small payload)

Zwei pro Slot: A (publish, Producer → Consumer) und B (vacate, Consumer → Producer der nächsten Runde). - compare_exchange (CAS) — atomares Read-Modify-Write, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so beansprucht genau ein Thread ein Ticket. - Relaxed / Acquire / Release — Relaxed ist atomar ohne Ordering; Release auf einem Store ist ein Boden unter allem, was davor kommt; Acquire auf einem Load ist ein Dach über allem, was danach kommt; ein Acquire-Load, der einen Release-Store beobachtet, verbindet die beiden. - happens-before — die threadübergreifende Garantie, die ein Release/Acquire-Paar aufbaut; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - False Sharing — zwei unabhängige Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores invalidiert; behoben durch CachePadded. - loom — ein Model Checker, der einen **kleinen** Test unter jeder Verschränkung ausführt; blind für Übergaben, die über einen CAS laufen. - Miri — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB meldet — Data Races, uninitialisierte Reads, Leaks.

Two per slot: A (publish, producer → consumer) and B (vacate, consumer → next lap's producer). - compare_exchange (CAS) — atomic read-modify-write that succeeds only if the value is what you expected; how exactly one thread claims a ticket. - Relaxed / Acquire / Release — Relaxed is atomic with no ordering; Release on a store floors what comes before it; Acquire on a load roofs what comes after it; an Acquire load that observes a Release store joins the two. - happens-before — the cross-thread guarantee a Release/Acquire pair builds; without one, a plain write on one thread and a plain read on another are a data race. - false sharing — two independent values on one cache line, each write invalidating the other core's copy; fixed by CachePadded. - loom — a model checker that runs a **small** test under every interleaving; blind to hand-offs carried by a CAS. - Miri — an interpreter that runs Rust against the memory model and reports UB — data races, uninitialized reads, leaks.

kompiliert

T4

compiled

Miri: der Race, beim Namen genannt — Miri interpretiert das **kompilierte** Programm und verfolgt jedes Byte: wer es geschrieben hat, auf welchem Thread, mit welchem Ordering und ob der Read danach ein happens-before zum Write hat.

Miri: the race, by name — Miri interprets the **compiled** program and tracks every byte: who wrote it, on which thread, with what ordering, and whether the read that follows has a happens-before to the write.

konstruktionsbedingt

T2

by construction, structurally guaranteed

Nach dem Lesen speichert er `seq = pos + capacity`: bereit für den *nächsten Producer*, der auf diesen Slot abgebildet wird und **konstruktionsbedingt** Ticket `pos + capacity` hält.

After reading it stores `seq = pos + capacity`: ready for the *next producer* that maps to this slot, who holds ticket `pos + capacity` by **construction**.

korrekt

T0

correct (logically sound)

*Nach Teil 2 aufzuhören lässt dich mit einem Ring zurück, der logisch **korrekt** ist; die Teile 3 und 4 sind der Ort, an dem er auf die Hardware und die Verifizierer trifft und an dem die Fehler wohnen, die Tests bestehen.*

Stopping after Part 2 leaves you with a ring that's logically **correct**; Parts 3 and 4 are where it meets the hardware and the verifiers, and where the failures that pass tests live.

langsam

langsamer · am
langsamsten

T2

slow

*Es überlebt eine Runde. — Ein Ring mit zwei Slots und ein **langsamer** Producer, der Ticket 0 hält:*

It survives one lap. — A ring of two slots, and a **slow** producer holding ticket 0:

leer

T1

empty (queue empty)

*Und eine `try_-API`: `try_push` auf einem vollen Ring gibt den Wert zurück; `try_pop` auf einem **leeren** Ring liefert `None`.*

And a `try_` API: `try_push` on a full ring hands the value back; `try_pop` on an **empty** ring returns `None`.

legal

T4

legal, valid under memory model

*Unter `ReLaxed` sagt nichts, dass es das muss; ein veralteter Wert ist für immer **legal**, also ist der Pfad, auf dem die Schleife nie terminiert, ein echter Pfad, und loom folgt ihm, bis sein Branch-Budget aufgebraucht ist.*

Under `ReLaxed`, nothing says it must; a stale value is **legal** forever, so the path where the loop never terminates is a real path, and loom follows it until its branch budget runs out.

lock-free

T0

lock-free (system-wide progress guarantee)

*Diese Serie entwirft die Queue, die das nicht tut: einen Ring — beschränkt, alloktationsfrei, **lock-free** —, den sich N Producer und M Consumer teilen und bei dem nur noch die Übergabe selbst etwas kostet.*

This series designs the queue that doesn't: a bounded, allocation-free, **lock-free** ring shared by N producers and M consumers, where the only cost is the hand-off itself.

logisch

T0

logical (algorithmic correctness)

*Nach Teil 2 aufzuhören lässt dich mit einem Ring zurück, der **logisch** korrekt ist; die Teile 3 und 4 sind der Ort, an dem er auf die Hardware und die Verifizierer trifft und an dem die Fehler wohnen, die Tests bestehen.*

Stopping after Part 2 leaves you with a ring that's **logically** correct; Parts 3 and 4 are where it meets the hardware and the verifiers, and where the failures that pass tests live.

lügend

T0

lying, deceptive (green passing status)

*[Teil 4 — Der Beweis: zwei Werkzeuge und ein **lügendes** Grün.] (04_proving_it.md) Miri nennt den Race beim Namen; loom, mit dem `ReLaxed`-Ring gefüttert, explodiert, statt zu antworten.*

[Part 4 — Proving it: two tools, and a **lying** green.](04_proving_it.md) Miri names the race; loom, given the `ReLaxed` ring, explodes instead of answering.

minimal

T4

minimal

*Jetzt die Probe aufs Exempel. — Eine **minimale** Übergabe, alles Relaxed: Der Producer beansprucht einen Zähler und schreibt dann den Slot; der Consumer sieht den Zähler weiterrücken, beansprucht seine Seite und liest den Slot.*

Now the probe. — A **minimal** hand-off, everything Relaxed: the producer claims a counter and then writes the slot; the consumer sees the counter move, claims its side, and reads the slot.

mitten

T1

in the middle of

***Mitten** im heißesten Pfad des Systems sitzt eine Queue, und das Erste, wonach jeder greift, ist **Mutex<VecDeque<T>>**.*

In the **middle** of the hottest path in the system sits a queue, and the first thing anyone reaches for is **Mutex<VecDeque<T>>**.

möglich

T1

possible

*Gib den Producern eine **Möglichkeit**, die Reihen unter sich zu verteilen, ohne je einen Consumer zu berühren; gib den Consumern das Spiegelbild davon; mach die Übergabe des payloads zu einer Sache pro Slot, zwischen den zwei Threads, die sich diesen Slot teilen.*

Give producers a **way** to allocate turns among themselves that never touches a consumer; give consumers the mirror of it; let the payload hand-off be a per-slot matter between the two threads that share that slot.

naheliegend

T0

obvious, intuitive

*[Teil 2 — Ein Zähler kann nicht „geschrieben“ sagen.](02_the_sequence.md) Der **naheliegende** Ring zerreißt in dem Moment, in dem ein Consumer neben einem Producer läuft, weil **tail** beim ***Claim*** weiterrückt, vor dem **Write**.*

[Part 2 — One counter can't say "written".](02_the_sequence.md) The **obvious** ring tears the instant a consumer runs beside a producer, because the tail moves at ***claim*** time, before the write.

nebeneinander

T4

side by side, adjacent

*Von verschiedenen Threads angefasst — und direkt **nebeneinander**. — **head** und **tail** sind zwei benachbarte **AtomicUsize** — sechzehn Bytes, eine cache line.*

Touched by different threads, and sitting **next to each other**. — head and tail are two adjacent **AtomicUsize**s — sixteen bytes, one cache line.

neu

T2

new

*CAS verloren. Ein anderer Thread hat Ticket **pos** genommen. — Das nächste Ticket ist frei, niemand muss etwas tun, damit ich es bekomme, also läuft die Schleife mit dem **neuen tail** weiter.*

Lost the CAS. Another thread took ticket **pos**. — The next ticket is free, nobody has to do anything for me to get it, so the loop continues with the **new tail**.

nicht-atomar

T3

non-atomic (plain memory access)

*Ein Release auf diesem CAS hat **nichts** unter sich. — Der Write liegt immer noch über dem Boden und kann ungehindert hindurchsinken.*

A Release on that CAS has nothing under it. — The write is still above the floor, free to sink past.

nicht-blockierend

T0

non-blocking (lock-free / try API)

*## Die Teile — [Teil 1 — Das Problem, und warum die naheliegenden Queues nicht passen.](01_the_problem.md) Drei Randbedingungen — beschränkt und allokatonsfrei, lock-free, Viele-zu-viele und **nicht-blockierend** — und die Tour durch die Queues, von denen jede eine davon verletzt.*

The parts — [Part 1 — The problem, and why the obvious queues don't fit.](01_the_problem.md) Three constraints — bounded and allocation-free, lock-free, many-to-many and **non-blocking** — and the tour of queues that each break one.

niedrig

niedriger · am
niedrigsten

T1

low (low bits in mask)

*Dieses & ist ein Modulo, aber nur, wenn capacity eine Zweierpotenz ist — dann ist capacity - 1 eine Folge **niedriger** 1-Bits, und die Maske behält genau diese.*

That & is a modulo, but only when capacity is a power of two — then capacity - 1 is a run of **low** 1 bits and the mask keeps exactly them.

normal

T3

normal, plain (unsynchronized access)

*Der Read bleibt **normal**. — Der vacate-Store seq.store(pos + capacity) ist der Boden unter diesem Read — die Sendeseite von Übergabe B — Release.*

The read is **plain**. — The vacate store seq.store(pos + capacity) is the floor under that read — hand-off B's sending side — Release.

paarweise

T4

in pairs (cache lines fetched in pairs)

*Das ist False Sharing, und der Fix ist, jeden Zähler auf seine eigene cache line zu legen: CachePadded, 128 Bytes auf Apple Silicon, wo der Prefetcher cache lines **paarweise** holt.*

That's false sharing, and the fix is to put each counter on its own line: CachePadded, 128 bytes on Apple silicon, where the prefetcher pulls lines **in pairs**.

passend

T0

fitting, suitable

*## Die Teile — [Teil 1 — Das Problem, und warum die naheliegenden Queues nicht **passen**.](01_the_problem.md) Drei Randbedingungen — beschränkt und allokatonsfrei, lock-free, Viele-zu-viele und nicht-blockierend — und die Tour durch die Queues, von denen jede eine davon verletzt.*

The parts — [Part 1 — The problem, and why the obvious queues don't fit.](01_the_problem.md) Three constraints — bounded and allocation-free, lock-free, many-to-many and non-blocking — and the tour of queues that each break one.

physisch

T1

physical (hardware address or slot)

*Keiner der beiden Zähler nimmt je ab, also laufen sie fast sofort über die Array-Länge hinaus, und ein Ticket wird auf einen **physischen** Slot abgebildet, indem man es zurückfaltet: $\text{slot} = \text{ticket} \& (\text{capacity} - 1)$.*

Neither counter ever decreases, so they run past the array length almost immediately, and a ticket maps to a **physical** slot by folding it back: $\text{slot} = \text{ticket} \& (\text{capacity} - 1)$.

positiv

T3

positive (diff > 0)

*Kein Ordering auf diesen Ops konnte die Daten erreichen. — Ein Fence — eine zweiseitige Wand, die du selbst **positionierst**, kein an eine Op geklebtes Gate — war das einzige Werkzeug.*

No ordering on those ops could reach the data. — A fence — a two-sided wall you position yourself, not a gate glued to an op — was the only tool.

prozessweit

T1

process-wide (lock affecting all threads)

*Ein Allocator-Aufruf bedeutet an einem guten Tag hunderte Nanosekunden und an einem schlechten ein **prozessweites** Lock; so oder so passt er nicht in eine Übergabe, die in unter einer Mikrosekunde fertig sein muss. - Lock-free. Kein einzelnes Speicherwort, das sich jeder Thread erst holen muss.*

An allocator call is hundreds of nanoseconds on a good day and a **process-wide** lock on a bad one; either way it doesn't fit inside a hand-off that has to complete in under a microsecond. - Lock-free. No single word every thread must acquire.

schlecht

schlechter · am schlechtesten

T1

bad — an einem schlechten Tag = on a bad day

*Ein Allocator-Aufruf bedeutet an einem guten Tag hunderte Nanosekunden und an einem **schlechten** ein prozessweites Lock; so oder so passt er nicht in eine Übergabe, die in unter einer Mikrosekunde fertig sein muss. - Lock-free. Kein einzelnes Speicherwort, das sich jeder Thread erst holen muss.*

An allocator call is hundreds of nanoseconds on a good day and a process-wide lock on a **bad** one; either way it doesn't fit inside a hand-off that has to complete in under a microsecond. - Lock-free. No single word every thread must acquire.

schnell

schneller · am schnellsten

T0

fast

*# Einen lock-freien MPMC-Ring entwerfen — Am schreibenden Ende von allem, was **schnell** ist — eine Matching Engine, die Orders von einem Dutzend Feed-Threads entgegennimmt, ein Write-Ahead-Log, das viele Writer in einen einzigen Commit bündelt —, sitzt eine Queue, in die viele Threads pushen und aus der viele Threads poppen, millionenfach pro Sekunde.*

Designing a lock-free MPMC ring — At the write end of anything **fast** — a matching engine taking orders off a dozen feed threads, a write-ahead log fanning writers into one commit — there's a queue that many threads push and many threads pop, millions of times a second.

schwach

schwächer · am
schwächsten

T0

weak (weak memory model e.g. ARM)

*Gebaut und gemessen auf **aarch64** (Apple M2), denn ein **schwaches** Memory-Modell ist der Ort, an dem sich die Fehler aus Teil 3 zeigen; auf x86 würde sich der Data Race hinter einer stärkeren Hardware-Garantie verstecken.*

Built and measured on **aarch64** (Apple M2), because a **weak** memory model is where the Part 3 failures show; on x86 the data race would hide behind a stronger hardware guarantee.

selbst

T0

itself

*Diese Serie entwirft die Queue, die das nicht tut: einen Ring — beschränkt, allokatonsfrei, lock-free —, den sich N Producer und M Consumer teilen und bei dem nur noch die Übergabe **selbst** etwas kostet.*

This series designs the queue that doesn't: a bounded, allocation-free, lock-free ring shared by N producers and M consumers, where the only cost is the hand-off **itself**.

sichtbar

T0

visible (across memory bus)

*Er zählt nur hoch: **pos** (Producer darf schreiben), **pos + 1** (Consumer darf lesen), **pos + capacity** (der Producer der nächsten Runde darf schreiben). - **claim** vs. **publish** — ein Ticket auf dem Zähler ziehen vs. den payload über **seq sichtbar** machen.*

It only increases: **pos** (producer may write), **pos + 1** (consumer may read), **pos + capacity** (next lap's producer may write). - **claim** vs **publish** — taking a ticket on the counter vs making the payload **visible** through **seq**.

spät

später · am spätesten

T2

late, later

*Eine Konsequenz, die man aussprechen sollte: **try_pop** gibt **None** zurück, wenn das Element **am Head** nicht veröffentlicht ist, selbst wenn **spätere** Slots es sind.*

One consequence worth stating: **try_pop** returns **None** when the item **at the head** isn't published, even if **later** slots are.

stark

stärker · am stärksten

T0

strong (strong hardware memory model e.g. x86)

*Gebaut und gemessen auf **aarch64** (Apple M2), denn ein schwaches Memory-Modell ist der Ort, an dem sich die Fehler aus Teil 3 zeigen; auf x86 würde sich der Data Race hinter einer **stärkeren** Hardware-Garantie verstecken.*

Built and measured on **aarch64** (Apple M2), because a weak memory model is where the Part 3 failures show; on x86 the data race would hide behind a **stronger** hardware guarantee.

strukturell

T2

structural (inherent flaw, not transient race)

*## Zwei Producer machen es **strukturell** — Mit zwei Producern ist die Lücke kein Wettlauf mehr, sondern eine Tatsache über Zähler:*

Two producers make it **structural** — With two producers the gap stops being a race and becomes a fact about counters:

unabhängig

T0

independent

Zwei pro Slot: A (publish, Producer → Consumer) und B (vacate, Consumer → Producer der nächsten Runde). - `compare_exchange` (CAS) — atomares Read-Modify-Write, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so beansprucht genau ein Thread ein Ticket. - `Relaxed / Acquire / Release` — `Relaxed` ist atomar ohne Ordering; `Release` auf einem Store ist ein Boden unter allem, was davor kommt; `Acquire` auf einem Load ist ein Dach über allem, was danach kommt; ein `Acquire`-Load, der einen `Release`-Store beobachtet, verbindet die beiden. - `happens-before` — die threadübergreifende Garantie, die ein `Release`/`Acquire`-Paar aufbaut; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - `False Sharing` — zwei **unabhängige** Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores invalidiert; behoben durch `CachePadded`. - `loom` — ein Model Checker, der einen kleinen Test unter jeder Verschränkung ausführt; blind für Übergaben, die über einen CAS laufen. - `Miri` — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB meldet — Data Races, uninitialisierte Reads, Leaks.

Two per slot: A (publish, producer → consumer) and B (vacate, consumer → next lap's producer). - `compare_exchange` (CAS) — atomic read-modify-write that succeeds only if the value is what you expected; how exactly one thread claims a ticket. - `Relaxed / Acquire / Release` — `Relaxed` is atomic with no ordering; `Release` on a store floors what comes before it; `Acquire` on a load roofs what comes after it; an `Acquire` load that observes a `Release` store joins the two. - `happens-before` — the cross-thread guarantee a `Release`/`Acquire` pair builds; without one, a plain write on one thread and a plain read on another are a data race. - `false sharing` — two **independent** values on one cache line, each write invalidating the other core's copy; fixed by `CachePadded`. - `loom` — a model checker that runs a small test under every interleaving; blind to hand-offs carried by a CAS. - `Miri` — an interpreter that runs Rust against the memory model and reports UB — data races, uninitialized reads, leaks.

unbeschränkt

T1

unbounded (infinite queue)

Ein **unbeschränkter** Channel — `std::sync::mpsc` oder irgendeine Queue aus verketteten Knoten — umgeht das eine Lock, indem er pro Nachricht einen Knoten alloziert.

An **unbounded** channel — `std::sync::mpsc`, or any linked-node queue — dodges the single lock by allocating a node per message.

undefiniert

T3

undefined (undefined behavior)

Auf dem M2 ist das Symptom ein Consumer, der gelegentlich den vorherigen Inhalt des Slots liest; in Rusts Memory-Modell ist es **undefiniertes** Verhalten, noch bevor es ein falscher Wert ist.

On the M2 the symptom is a consumer that occasionally reads the slot's previous contents; in Rust's memory model it's **undefined** behavior before it's a wrong value.

ungehindert

T3

unhindered, freely

*Ein Release auf diesem CAS hat nichts unter sich. — Der Write liegt immer noch über dem Boden und kann **ungehindert** hindurchsinken.*

A Release on that CAS has nothing under it. — The write is still above the floor, **free** to sink past.

uninitialisiert

T0

uninitialized (uninitialized memory read)

*Zwei pro Slot: A (publish, Producer → Consumer) und B (vacate, Consumer → Producer der nächsten Runde). - compare_exchange (CAS) — atomares Read-Modify-Write, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so beansprucht genau ein Thread ein Ticket. - Relaxed / Acquire / Release — Relaxed ist atomar ohne Ordering; Release auf einem Store ist ein Boden unter allem, was davor kommt; Acquire auf einem Load ist ein Dach über allem, was danach kommt; ein Acquire-Load, der einen Release-Store beobachtet, verbindet die beiden. - happens-before — die threadübergreifende Garantie, die ein Release/Acquire-Paar aufbaut; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - False Sharing — zwei unabhängige Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores invalidiert; behoben durch CachePadded. - loom — ein Model Checker, der einen kleinen Test unter jeder Verschränkung ausführt; blind für Übergaben, die über einen CAS laufen. - Miri — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB meldet — Data Races, **uninitialisierte** Reads, Leaks.*

Two per slot: A (publish, producer → consumer) and B (vacate, consumer → next lap's producer). - compare_exchange (CAS) — atomic read-modify-write that succeeds only if the value is what you expected; how exactly one thread claims a ticket. - Relaxed / Acquire / Release — Relaxed is atomic with no ordering; Release on a store floors what comes before it; Acquire on a load roofs what comes after it; an Acquire load that observes a Release store joins the two. - happens-before — the cross-thread guarantee a Release/Acquire pair builds; without one, a plain write on one thread and a plain read on another are a data race. - false sharing — two independent values on one cache line, each write invalidating the other core's copy; fixed by CachePadded. - loom — a model checker that runs a small test under every interleaving; blind to hand-offs carried by a CAS. - Miri — an interpreter that runs Rust against the memory model and reports UB — data races, **uninitialized** reads, leaks.

unteilbar

T3

indivisible (atomic operation)

*Atomarität ist nicht Ordering. — compare_exchange garantiert, dass das Read-Modify-Write **unteilbar** ist — keine zwei Threads nehmen Ticket pos.*

Atomicity is not ordering. — compare_exchange guarantees the read-modify-write is **indivisible** — no two threads take ticket pos.

veraltet

T3

stale, outdated (cached value)

*tail.load — ein Anhaltspunkt. — Das Gate validiert ihn erneut, und ein **veralteter** Wert kostet einen zusätzlichen Schleifendurchlauf. Relaxed. 2.*

tail.load — a hint. — The gate re-validates it, and a **stale** value costs one extra loop. Relaxed. 2.

verknüpft

T1

linked (linked node queue)

- Beschränkt und allokatonsfrei. Ein fester Puffer, einmal dimensioniert. — Keine Box pro Nachricht, kein verketteter Knoten, der beim Push alloziert und beim Pop freigegeben wird.

- Bounded and allocation-free. A fixed buffer, sized once. — No Box per message, no linked node allocated on push and freed on pop.

verloren

T2

lost (lost CAS / lost update)

CAS verloren. Ein anderer Thread hat Ticket pos genommen. — Das nächste Ticket ist frei, niemand muss etwas tun, damit ich es bekomme, also läuft die Schleife mit dem neuen tail weiter.

Lost the CAS. Another thread took ticket pos. — The next ticket is free, nobody has to do anything for me to get it, so the loop continues with the new tail.

verschieden

T0

different, distinct

Verschiedene Operationen, verschiedene Atomics. - Übergabe (hand-off) — ein Thread ist mit ein paar Bytes fertig und signalisiert es, ein anderer holt sie ab: eine Sendeseite und eine Empfangsseite.

Different operations, different atomics. - hand-off — one thread finishing with some bytes and signaling, another picking them up: a sending side and a receiving side.

voll

T1

full (queue full)

Und eine try_-API: try_push auf einem vollen Ring gibt den Wert zurück; try_pop auf einem leeren Ring liefert None.

And a try_ API: try_push on a full ring hands the value back; try_pop on an empty ring returns None.

vorig

T0

previous (previous lap or previous part)

Der Reihe nach. — Jeder Teil beginnt dort, wo der vorige aufgehört hat, und schließt mit der Frage, die der nächste beantwortet.

In order. — Each part opens where the previous one stopped and closes on the question the next one answers.

vorzeichenbehaftet

T2

signed (signed integer difference)

Das Drei-Wege-Gate — Der Vergleich ist eine vorzeichenbehaftete Differenz, seq.wrapping_sub(pos) as isize, damit er aussagekräftig bleibt, falls die Zähler je überlaufen.

The three-way gate — The comparison is a signed difference, seq.wrapping_sub(pos) as isize, so it stays meaningful if the counters ever wrap.

wait-free

T0

wait-free (per-thread bounded progress)

Nicht **wait-free**: Ein einzelner Thread kann es erneut versuchen müssen. - Ticket — der Wert, den ein Thread von **tail** (Producer) oder **head** (Consumer) zieht; **ticket** & **(capacity - 1)** ist sein Slot. - Slot / Zelle — ein Element des Puffers: ein payload und seine Sequenznummer. - **seq** — der Zähler pro Slot, der sagt, für welches Ticket der Slot bereit ist.

Not **wait-free**: an individual thread may retry. - ticket — the value a thread takes from **tail** (producer) or **head** (consumer); **ticket** & **(capacity - 1)** is its slot. - slot / cell — one element of the buffer: a payload and its sequence number. - **seq** — the per-slot counter that says which ticket the slot is ready for.

wertvoll

T0

valuable — weniger wert = worth less

Mit den Orderings drin werden beide grün — und das Grün von loom ist weniger **wert**, als es aussieht: Es modelliert einen Compare-and-Swap stärker als die Hardware und winkt eine kaputte Übergabe durch, die über einen läuft — gezeigt, indem man eine einzige Zeile ändert.

With the orderings in, both go green — and loom's green is **worth** less than it looks: it models a compare-and-swap more strongly than the hardware, and passes a broken hand-off that runs through one, shown by changing one line.

wirklich

T1

real, actually

Dann zerreißt es, sobald zwei Threads es zum ersten Mal **wirklich** anfassen, aus einem Grund, der kein Bug im Code ist, sondern ein Loch in der Idee.

Then it tears the first time two threads touch it for **real**, for a reason that isn't a bug in the code but a hole in the idea.

zweiseitig

T3

two-sided (fence barrier)

Kein Ordering auf diesen Ops konnte die Daten erreichen. — Ein Fence — eine **zweiseitige** Wand, die du selbst positionierst, kein an eine Op geklebtes Gate — war das einzige Werkzeug.

No ordering on those ops could reach the data. — A fence — a **two-sided** wall you position yourself, not a gate glued to an op — was the only tool.

Redewendungen & feste Wendungen idioms and set phrases

11

an der Reihe sein

T1

to be one's turn (claim turn in queue)

Schau hin, wo der echte Konflikt sitzt. — Zwei Producer, die beide pushen wollen, streiten nicht um *Daten* — sie streiten darum, wer ***an der Reihe*** ist.

Look at where the genuine conflict lives. — Two producers both wanting to push don't conflict over *data* — they conflict over ***a turn***.

auf der anderen Seite

T1

on the other hand / on the receiving side

*Zwölf Feed-Handler, einer pro Core, jeder zieht Orders aus einer anderen Exchange-Session und schiebt sie an eine einzige Stelle: den Eingang der Matching Engine. **Auf der anderen Seite** poppt ein Pool von Matching-Threads aus genau diesem Eingang.*

Twelve feed handlers, one per core, each pulling orders off a different exchange session and pushing them into one place: the matching engine's intake. **On the other side**, a pool of matching threads pops from that same intake.

beim Namen nennen

T0

to call a spade a spade / name the bug directly

*[Teil 4 — Der Beweis: zwei Werkzeuge und ein lügendes Grün.] (04_proving_it.md) Miri nennt den Race **beim Namen**; loom, mit dem ReLaxed-Ring gefüttert, explodiert, statt zu antworten.*

*[Part 4 — Proving it: two tools, and a lying green.](04_proving_it.md) Miri **names** the race; loom, given the ReLaxed ring, explodes instead of answering.*

hin und her

T1

back and forth (cache bounce)

*Aber jeder push und jeder pop nimmt dasselbe Lock, also springt das Lock-Wort bei jeder einzelnen Operation wie ein Pingpongball zwischen den Cores **hin und her** — das MESI-Protokoll invalidiert eine cache line in jedem anderen Core, millionenfach pro Sekunde, selbst wenn logisch gar keine Contention herrscht.*

But every push and every pop takes the same lock, so the lock word **ping-pongs** between cores on every single operation — the MESI protocol invalidating one cache line in every other core, millions of times a second, even when nobody is logically contending.

in einen Topf werfen

T1

to conflate two separate jobs into one

*Das sind zwei verschiedene Aufgaben, und ein Lock wirft sie **in einen Topf**. — Schau hin, wo der echte Konflikt sitzt.*

Those are two different jobs, and a lock **conflates them**. — Look at where the genuine conflict lives.

mit Absicht

T3

deliberately, on purpose

*## Der CAS ist **mit Absicht** ReLaxed — Bei (3) zucken die Leute zusammen.
The CAS is ReLaxed **on purpose** — The one people flinch at is (3).*

ohne weiteres

T1

readily, without further ado

*Ein Mutex *pro Slot* verkleinert das Lock, **ohne es** loszuwerden, und bringt ein Problem mit, das das Design vorher nicht hatte: entscheiden, welchen Slot jeder Thread ansteuern soll.*

A mutex *per slot* shrinks the lock without removing it, and adds a problem the design didn't have before: deciding which slot each thread should head for.

die Probe aufs Exempel

T4

the acid test / proving the thesis

*Jetzt die **Probe aufs Exempel**. — Eine minimale Übergabe, alles **Re**laxed: Der Producer beansprucht einen Zähler und schreibt dann den Slot; der Consumer sieht den Zähler weiterrücken, beansprucht seine Seite und liest den Slot.*

Now the **probe**. — A minimal hand-off, everything **Re**laxed: the producer claims a counter and then writes the slot; the consumer sees the counter move, claims its side, and reads the slot.

der Reihe nach

T0

in order, sequentially

***Der Reihe nach**. — Jeder Teil beginnt dort, wo der vorige aufgehört hat, und schließt mit der Frage, die der nächste beantwortet.*

In order. — Each part opens where the previous one stopped and closes on the question the next one answers.

unter Dach und Fach

T3

under roof and floor / safely bounded

*Darüber, wann der payload für irgendwen sichtbar wird, sagt es nichts, und das muss es auch nicht: Der Zugriff hat bereits ein **Dach und einen Boden** — das **Acquire** in (2), das ihm vorausgeht, und das **Release** in (5), das ihm folgt.*

It says nothing about when the payload becomes visible to anyone, and it doesn't need to: the access is already **roofed by the Acquire in (2), which precedes it, and floored by the Release in (5), which follows it.**

von oben nach unten

T3

top to bottom (code walk-through)

*## Jedes Ordering platzieren — Geh `try_push` **von oben nach unten** durch.*

*## Placing every ordering — Walk `try_push` **top to bottom**.*

Textstruktur & Signalwörter

discourse markers

52

aber

T1

but, however

*Dieses **&** ist ein Modulo, **aber** nur, wenn `capacity` eine Zweierpotenz ist — dann ist `capacity - 1` eine Folge niedriger 1-Bits, und die Maske behält genau diese.*

That **&** is a modulo, **but** only when `capacity` is a power of two — then `capacity - 1` is a run of low 1 bits and the mask keeps exactly them.

also

T1

so, therefore

*Das Lock ist kein Detail dieser Designs; sie sind um das Lock herum gebaut. — Die Frage ist **also** nicht „welches Lock ist am billigsten“ — sondern was das Lock eigentlich *tut*, und ob sich diese Arbeit auch ohne es erledigen lässt.*

The lock isn't a detail of these designs; it's what they're built around. — So the question isn't "which lock is cheapest" — it's what the lock is actually *doing*, and whether that job can be done without it.

anders als

T0

unlike — anders als ein SeqLock = unlike a SeqLock

*Boden und Dach, ein Gate nach dem anderen, warum der CAS Relaxed bleibt, und warum dieser Ring — **anders als** ein SeqLock — keinen Fence braucht.*

Floor and roof, one gate at a time, why the CAS stays Relaxed, and why this ring — **unlike** a SeqLock — needs no fence.

auch

T4

also, too

*Teil 3 hat jedes Ordering per Argument platziert. — Der All-Relaxed-Ring hatte **auch** ein Argument — die Tests liefen durch —, also ist ein Argument kein Beweis.*

Part 3 placed every ordering by argument. — The all-Relaxed ring had an argument too — the tests passed — so an argument isn't proof.

auf diese Weise

T2

in this way, the same way

*Ein Consumer liest head, prüft `head != tail`, beansprucht das Ticket **auf dieselbe Weise** und liest den Slot.*

A consumer reads head, checks `head != tail`, claims the ticket **the same way**, and reads the slot.

aus diesem Grund

T4

for this reason

*Data Race, die zwei Threads, der Typ an der Adresse: `MaybeUninit<usize>`. — Diese Präzision ist der **Grund**, warum Miri zuerst drankommt.*

Data race, the two threads, the type at the address: `MaybeUninit<usize>`. — That precision is why Miri comes first.

bereits

T3

already

*Darüber, wann der payload für irgendwen sichtbar wird, sagt es nichts, und das muss es auch nicht: Der Zugriff hat **bereits** ein Dach und einen Boden — das Acquire in (2), das ihm vorausgeht, und das Release in (5), das ihm folgt.*

It says nothing about when the payload becomes visible to anyone, and it doesn't need to: the access is **already** roofed by the Acquire in (2), which precedes it, and floored by the Release in (5), which follows it.

da

T2

since, because / there

*C0 findet `seq = 0`, nicht 1, und weiß, dass sein Element nicht **da** ist. — Keine Runde wird je für eine andere gehalten.*

C0 finds `seq = 0`, not 1, and knows its item isn't **there**. — No lap is ever mistaken for another.

damit

T1

so that, in order that

***Damit** liegt der Durchsatz in der Hand des Allocators, und der aus `std` ist ohnehin schon dem Namen nach single-consumer.*

That hands throughput to the allocator, and `std`'s is single-consumer by name anyway.

daraus

T1

from it, out of that

*Ein einzelnes Lock lässt jeden Thread jedes Mal auf beide Aufgaben warten: Ein Producer blockiert einen Consumer, der einen völlig anderen Slot will, weil sie sich das Lock-Wort teilen. Das ganze Design ergibt sich **daraus**, genau das zu verweigern.*

A single lock makes every thread wait for both jobs, every time: a producer blocks a consumer that wants a completely different slot, because they share the lock word. The whole design falls out of refusing that.

darüber hinaus

T2

beyond that

*Der Ring ist voll. — `Err(value)` zurückgeben. — `diff > 0` — irgendein Producer hat Ticket `pos` schon genommen und `seq` **darüber hinausgeschoben**.*

The ring is full. — Return `Err(value)`. — `diff > 0` — some producer already took ticket `pos` and moved `seq` **past it**.

davor

T0

before it, prior

*Die Orderings werden hergeleitet, nicht geraten: die zwei Stellen finden, an denen Daten den Thread wechseln, und dann bemerken, dass der payload nur über `seq` veröffentlicht werden kann, nie über den Zähler — ein `Release` legt einen Boden unter alles, was ***davor*** kommt, und der Claim kommt vor dem Write.*

The orderings are derived, not guessed: find the two places data crosses threads, then notice the payload can only be published through `seq`, never through the counter — a `Release` floors what comes ***before*** it, and the claim comes before the write.

denn

T0

because, for

*Gebaut und gemessen auf `aarch64` (Apple M2), **denn** ein schwaches Memory-Modell ist der Ort, an dem sich die Fehler aus Teil 3 zeigen; auf `x86` würde sich der Data Race hinter einer stärkeren Hardware-Garantie verstecken.*

Built and measured on `aarch64` (Apple M2), **because** a weak memory model is where the Part 3 failures show; on `x86` the data race would hide behind a stronger hardware guarantee.

dennoch

T4

nevertheless, yet

*Der Consumer schreibt `head`, und das invalidiert die des Producers. — Keiner der beiden Threads liest auf dem Hot Path je den Zähler des anderen, und **trotzdem** zahlen sie für die Writes des jeweils anderen.*

The consumer writes `head`, which invalidates the producer's. — Neither thread ever reads the other's counter on the hot path, and they pay for each other's writes **anyway**.

direkt

T4

directly, right next to

*Von verschiedenen Threads angefasst — und **direkt nebeneinander**. — `head` und `tail` sind zwei benachbarte `AtomicUsize` — sechzehn Bytes, eine cache line.*

Touched by different threads, and **sitting next to** each other. — `head` and `tail` are two adjacent `AtomicUsize`s — sixteen bytes, one cache line.

durchaus

T1

definitely, certainly

*## Drei Randbedingungen, nicht eine — Wäre **Durchsatz** die einzige Anforderung, wäre das ein Benchmark und kein Designproblem.*

Three constraints, not one — If throughput were the only requirement this would be a benchmark, not a design problem.

eigentlich

T1

actually, in reality

Das Lock ist kein Detail dieser Designs; sie sind um das Lock herum gebaut.

*— Die Frage ist also nicht „welches Lock ist am billigsten“ — sondern was das Lock **eigentlich** *tut*, und ob sich diese Arbeit auch ohne es erledigen lässt.*

*The lock isn't a detail of these designs; it's what they're built around. — So the question isn't "which lock is cheapest" — it's what the lock is **actually** *doing*, and whether that job can be done without it.*

einmal

T1

once — einmal dimensioniert = sized once

*- Beschränkt und allokatonsfrei. Ein fester Puffer, **einmal** dimensioniert. — Keine Box pro Nachricht, kein verketteter Knoten, der beim Push alloziert und beim Pop freigegeben wird.*

*- Bounded and allocation-free. A fixed buffer, sized **once**. — No Box per message, no linked node allocated on push and freed on pop.*

entweder ... oder

T2

either ... or

*written = false hieß „konsumiert“ ***oder*** „beansprucht, aber noch nicht geschrieben“, und P2 kann die beiden nicht auseinanderhalten.*

*written = false meant "consumed" ***or*** "claimed but not yet written", and P2 can't tell which.*

erneut

T0

again, anew (retry)

*Ein Drei-Wege-Gate, die **publish**- und **vacate**-Regeln und der Vertrag darüber, wann **try_push** **erneut** versucht und wann es zurückkehrt.*

*A three-way gate, the publish and vacate rules, and the contract for when **try_push** **retries** and when it returns.*

erst

T1

only, not until

*Ein Allocator-Aufruf bedeutet an einem guten Tag hunderte Nanosekunden und an einem schlechten ein prozessweites Lock; so oder so passt er nicht in eine Übergabe, die in unter einer Mikrosekunde fertig sein muss. - Lock-free. Kein einzelnes Speicherwort, das sich jeder Thread **erst** holen muss.*

An allocator call is hundreds of nanoseconds on a good day and a process-wide lock on a bad one; either way it doesn't fit inside a hand-off that has to complete in under a microsecond. - Lock-free. No single word every thread must acquire.

etwa

T4

approximately (~)

- ein Thread, erst push, dann pop — ~7,3 ns. - ein Producer-Thread, ein Consumer-Thread — ~51 ns.

- one thread, push then pop — ~7.3 ns. - one producer thread, one consumer thread — ~51 ns.

fast

T1

almost, nearly

*Keiner der beiden Zähler nimmt je ab, also laufen sie **fast** sofort über die Array-Länge hinaus, und ein Ticket wird auf einen physischen Slot abgebildet, indem man es zurückfaltet: `slot = ticket & (capacity - 1)`.*

Neither counter ever decreases, so they run past the array length **almost** immediately, and a ticket maps to a physical slot by folding it back: `slot = ticket & (capacity - 1)`.

gar nicht

T1

not at all

*Aber jeder **push** und jeder **pop** nimmt dasselbe Lock, also springt das Lock-Wort bei jeder einzelnen Operation wie ein Pingpongball zwischen den Cores hin und her — das MESI-Protokoll invalidiert eine cache line in jedem anderen Core, millionenfach pro Sekunde, selbst wenn logisch **gar keine** Contention herrscht.*

But every push and every pop takes the same lock, so the lock word pings between cores on every single operation — the MESI protocol invalidating one cache line in every other core, millions of times a second, **even when nobody** is logically contending.

genauso

T3

just as, equally

*Bytes zu überschreiben, die ein anderer Thread vielleicht noch liest, ist **genauso** ein Data Race.*

Writing over bytes another thread may still be reading is a race **too**.

hier

T2

here

*# Teil 2 — Ein Zähler kann nicht „geschrieben“ sagen — **Hier** ist der Ring aus Teil 1 mit dem naheliegenden Push und Pop.*

Part 2 — One counter can't say "written" — **Here** is the ring from Part 1 with the obvious push and pop.

hinterher

T4

afterward, behind

*Der payload-Zugriff läuft über `loom::cell::UnsafeCell`, damit loom ihn sieht, **hinter** einem `cfg(loom)`-Shim, der looms Atomics einwechselt.*

Payload access goes through `loom::cell::UnsafeCell` so loom can see it, **behind** a `cfg(loom)` shim that swaps in loom's atomics.

im Kreis

T2

in a circle, cyclically

*Der Slot reicht einen Staffelstab **im Kreis** weiter, und auf dem Staffelstab steht die Ticketnummer.*

The slot passes a baton **around a cycle**, and the baton has the ticket number written on it.

immer noch

T3

still (above the boundary)

*Ein **Release** auf diesem CAS hat nichts unter sich. — Der Write liegt **immer noch** über dem Boden und kann ungehindert hindurchsinken.*

A **Release** on that CAS has nothing under it. — The write is **still** above the floor, free to sink past.

immer nur

T2

strictly monotonic (only ever goes up)

*Er sagt nicht „frei“ oder „voll“. — Er sagt, für welches Ticket der Slot bereit ist, und diese Zahl geht **immer nur** nach oben.*

It doesn't say "free" or "full". — It says which ticket the slot is ready for, and that number **only ever** goes up.

indem

T1

by (doing something)

*Ein unbeschränkter Channel — `std::sync::mpsc` oder irgendeine Queue aus verketteten Knoten — umgeht das eine Lock, **indem** er pro Nachricht einen Knoten alloziert.*

An unbounded channel — `std::sync::mpsc`, or any linked-node queue — dodges the single lock **by** allocating a node per message.

jedenfalls

T2

at any rate, in any case

*Kann er nicht, **jedenfalls** nicht auf die eine Art, auf die es ankommt. — Nur der Producer, der Ticket `pos` hält, kann `seq` von `pos` auf `pos + 1` bewegen, und wenn mein CAS auf `tail` von `pos` aus gelingt, dann bin ich dieser Producer.*

It can't, in the one way that matters. — Only the producer holding ticket `pos` can move `seq` from `pos` to `pos + 1`, and if my CAS on `tail` succeeds from `pos`, that producer is me.

jemals

T2

ever

*C0 findet `seq = 0`, nicht 1, und weiß, dass sein Element nicht da ist. — Keine Runde wird **je** für eine andere gehalten.*

C0 finds `seq = 0`, not 1, and knows its item isn't there. — No lap is **ever** mistaken for another.

kaum

T4

barely, hardly (no more than 2 ns)

*Die Single-Producer/Single-Consumer-Übergabe geht von ~51 ns auf ~8,8 ns — 5,8×, und liegt jetzt **keine zwei** Nanosekunden über der Ein-Thread-Untergrenze.*

The single-producer/single-consumer hand-off goes from ~51 ns to ~8.8 ns — 5.8×, and now **within two** nanoseconds of the single-thread floor.

keineswegs

T1

by no means

*- Beschränkt und allokatonsfrei. Ein fester Puffer, einmal dimensioniert. — **Keine** Box pro Nachricht, kein verketteter Knoten, der beim Push alloziert und beim Pop freigegeben wird.*

- Bounded and allocation-free. A fixed buffer, sized once. — No Box per message, no linked node allocated on push and freed on pop.

nach wie vor

T3

still, as before

> Der Claim kann die Daten nicht veröffentlichen, denn zum Zeitpunkt des Claims > existieren die Daten noch nicht. Die Veröffentlichung gehört dem letzten Store nach > dem Write — `seq`.

> The claim can't publish the data, because at claim time the data doesn't exist > yet. Publication belongs to the last store after the write — `seq`.

nicht mehr

T4

no longer

*Setz die Orderings aus Teil 3 ein, und die Schedules, in denen der Wert ewig veraltet bleibt, sind **nicht mehr** legal.*

Put the Part 3 orderings in and the stale-forever schedules stop being legal.

nie

T0

never

*Die Orderings werden hergeleitet, nicht geraten: die zwei Stellen finden, an denen Daten den Thread wechseln, und dann bemerken, dass der payload nur über seq veröffentlicht werden kann, **nie** über den Zähler — ein Release legt einen Boden unter alles, was *davor* kommt, und der Claim kommt vor dem Write.*

The orderings are derived, not guessed: find the two places data crosses threads, then notice the payload can only be published through seq, **never** through the counter — a Release floors what comes *before* it, and the claim comes before the write.

noch bevor

T3

even before (it is a wrong value)

*Auf dem M2 ist das Symptom ein Consumer, der gelegentlich den vorherigen Inhalt des Slots liest; in Rusts Memory-Modell ist es undefiniertes Verhalten, **noch bevor** es ein falscher Wert ist.*

On the M2 the symptom is a consumer that occasionally reads the slot's previous contents; in Rust's memory model it's undefined behavior **before** it's a wrong value.

nun mal

T4

as it happens, simply

*Was Miri nicht tut, ist aufzählen. — Es führt die Verschränkungen aus, die es **nun mal** ausführt, mit ein paar eingestreuten Unterbrechungen.*

What Miri does not do is enumerate. — It runs the interleavings it **happens** to run, with some preemption thrown in.

nur noch

T0

only (the only remaining cost)

*Diese Serie entwirft die Queue, die das nicht tut: einen Ring — beschränkt, allokatonsfrei, lock-free —, den sich N Producer und M Consumer teilen und bei dem **nur noch** die Übergabe selbst etwas kostet.*

This series designs the queue that doesn't: a bounded, allocation-free, lock-free ring shared by N producers and M consumers, where the **only** cost is the hand-off itself.

ohnehin

adv T1

anyway, in any case

*Damit liegt der Durchsatz in der Hand des Allocators, und der aus std ist **ohnehin** schon dem Namen nach single-consumer.*

That hands throughput to the allocator, and std's is single-consumer by name **anyway**.

rund um

T1

around (built around)

*Drei der vier opfern dasselbe. — Das Lock ist kein Detail dieser Designs; sie sind um das Lock **herum** gebaut.*

Three of the four surrender the same thing. — The lock isn't a detail of these designs; it's what they're **built around**.

schließlich

T0

after all, eventually

*Der Reihe nach. — Jeder Teil beginnt dort, wo der vorige aufgehört hat, und **schließt** mit der Frage, die der nächste beantwortet.*

In order. — Each part opens where the previous one stopped and closes on the question the next one answers.

so oder so

T1

either way

*Ein Allocator-Aufruf bedeutet an einem guten Tag hunderte Nanosekunden und an einem schlechten ein prozessweites Lock; **so oder so** passt er nicht in eine Übergabe, die in unter einer Mikrosekunde fertig sein muss. - Lock-free. Kein einzelnes Speicherwort, das sich jeder Thread erst holen muss.*

An allocator call is hundreds of nanoseconds on a good day and a process-wide lock on a bad one; **either way** it doesn't fit inside a hand-off that has to complete in under a microsecond. - Lock-free. No single word every thread must acquire.

sobald

T1

as soon as, the moment

*Dann zerreißt es, **sobald** zwei Threads es zum ersten Mal wirklich anfassen, aus einem Grund, der kein Bug im Code ist, sondern ein Loch in der Idee.*

Then it tears **the first time** two threads touch it for real, for a reason that isn't a bug in the code but a hole in the idea.

sowohl ... als auch

T3

both ... and

*Der Slot wurde schon in (2) geprüft; der payload wird in (5) veröffentlicht. — Auf dieser Op wechseln keine Daten den Thread. Relaxed, **Erfolg wie Fehlschlag**. 4.*

The slot was already checked by (2); the payload will be published by (5). — No data crosses on this op. Relaxed, **success and failure**. 4.

statt

T0

instead of

*[Teil 4 — Der Beweis: zwei Werkzeuge und ein lügendes Grün.] (04_proving_it.md) Miri nennt den Race beim Namen; loom, mit dem Relaxed-Ring gefüttert, explodiert, **statt** zu antworten.*

[Part 4 — Proving it: two tools, and a lying green.](04_proving_it.md) Miri names the race; loom, given the Relaxed ring, explodes **instead of** answering.

stattdessen

T4

instead

*## loom: eine Explosion **statt** einer Antwort — loom führt einen kleinen Test unter *jeder* Verschränkung aus, die das Memory-Modell erlaubt, innerhalb einer Schranke.*

loom: an explosion **instead** of an answer — loom runs a small test under *every* interleaving the memory model permits, within a bound.

trotzdem

T4

nonetheless, anyway

*Der Consumer schreibt head, und das invalidiert die des Producers. — Keiner der beiden Threads liest auf dem Hot Path je den Zähler des anderen, und **trotzdem** zahlen sie für die Writes des jeweils anderen.*

The consumer writes head, which invalidates the producer's. — Neither thread ever reads the other's counter on the hot path, and they pay for each other's writes **anyway**.

wieder

T1

again

*Und in dem Moment, in dem der Halter des Locks vom OS verdrängt wird, wartet jeder andere Thread darauf, dass genau dieser eine Thread **wieder** eingeplant wird.*

And the instant the holder is preempted, every other thread waits for that one thread to be scheduled **again**.

zuerst

T4

first, initially

*Data Race, die zwei Threads, der Typ an der Adresse: **MaybeUninit**<usize>. — Diese Präzision ist der Grund, warum Miri **zuerst** drankommt.*

Data race, the two threads, the type at the address: **MaybeUninit**<usize>. — That precision is why Miri comes **first**.

Englische Lehnwörter

English loanwords and the gender German gives them

84

der **Allocator**

-en T1

memory allocator

*Ein **Allocator**-Aufruf bedeutet an einem guten Tag hunderte Nanosekunden und an einem schlechten ein prozessweites Lock; so oder so passt er nicht in eine Übergabe, die in unter einer Mikrosekunde fertig sein muss. - Lock-free. Kein einzelnes Speicherwort, das sich jeder Thread erst holen muss.*

An **allocator** call is hundreds of nanoseconds on a good day and a process-wide lock on a bad one; either way it doesn't fit inside a hand-off that has to complete in under a microsecond. - Lock-free. No single word every thread must acquire.

das **Array**

-s T1

array — fixed-size buffer

*Ein festes **Array** von Slots. — Ein Zähler, **tail**, den Producer atomar lesen und hochzählen, um eine Reihe zu beanspruchen — der Wert, den du dabei genommen hast, ist dein *Ticket*.*

A fixed **array** of slots. — One counter, **tail**, that producers atomically read-and-bump to claim a turn — the value you took is your *ticket*.

das **Assert**

-s T1

assertion (in code)

*Der Konstruktor prüft die Zweierpotenz also per **Assert** und speichert **mask = capacity - 1** ein einziges Mal.*

So the constructor asserts the power of two and stores **mask = capacity - 1** once.

das **Atomic**

-s T0

atomic variable

*Verschiedene Operationen, verschiedene **Atomics**. - Übergabe (hand-off) — ein Thread ist mit ein paar Bytes fertig und signalisiert es, ein anderer holt sie ab: eine Sendeseite und eine Empfangsseite.*

Different operations, different **atomics**. - hand-off — one thread finishing with some bytes and signaling, another picking them up: a sending side and a receiving side.

der **Benchmark**

-s T0

benchmark test

Die Arbeitsteilung zwischen den drei Instrumenten, ein Drop, das Miris Leak-Checker absegnet, und der **Benchmark**: `head` und `tail` mit Padding auf getrennte cache lines zu legen, bringt die SPSC-Übergabe von ~51 ns auf ~9 ns.

The division of labor between the three instruments, a Drop that Miri's leak checker approves, and the **benchmark**: padding `head` and `tail` onto separate cache lines takes the SPSC hand-off from ~51 ns to ~9 ns.

das **Bit**

-s T1

bit (binary digit)

Dieses `&` ist ein Modulo, aber nur, wenn `capacity` eine Zweierpotenz ist — dann ist `capacity - 1` eine Folge niedriger 1-Bits, und die Maske behält genau diese.

That `&` is a modulo, but only when `capacity` is a power of two — then `capacity - 1` is a run of low 1 bits and the mask keeps exactly them.

der **Boolean**

-s T0

boolean value

Ein **Boolean** pro Slot kann das nicht reparieren — ein Boolean hat kein Gedächtnis, und Slots werden über Runden hinweg wiederverwendet.

A per-slot **boolean** can't fix it — a boolean has no memory, and slots are reused across laps.

die **Box**

-en T1

Box (heap allocation pointer)

- Beschränkt und allokatonsfrei. Ein fester Puffer, einmal dimensioniert. — Keine ``Box`` pro Nachricht, kein verketteter Knoten, der beim Push alloziert und beim Pop freigegeben wird.

- Bounded and allocation-free. A fixed buffer, sized once. — No ``Box`` per message, no linked node allocated on push and freed on pop.

der **Branch**

-es T4

branch (execution branch in model checking)

Unter `ReLaxed` sagt nichts, dass es das muss; ein veralteter Wert ist für immer legal, also ist der Pfad, auf dem die Schleife nie terminiert, ein echter Pfad, und loom folgt ihm, bis sein **Branch**-Budget aufgebraucht ist.

Under `ReLaxed`, nothing says it must; a stale value is legal forever, so the path where the loop never terminates is a real path, and loom follows it until its **branch** budget runs out.

das **Budget**

-s T1

budget (latency or branch limit)

Jedes Mal dasselbe Objekt: viele Threads pushen, viele Threads poppen, auf dem Hot Path, bei einer Rate, bei der der Overhead der Queue selbst das **Budget** ist.

Same object each time: many threads push, many threads pop, on the hot path, at a rate where the queue's own overhead is the **budget**.

der **Bug**

-s T1

software bug

Dann zerreißt es, sobald zwei Threads es zum ersten Mal wirklich anfassen, aus einem Grund, der kein **Bug** im Code ist, sondern ein Loch in der Idee.

Then it tears the first time two threads touch it for real, for a reason that isn't a **bug** in the code but a hole in the idea.

das **Byte**

-s T0

byte (8 bits)

*Verschiedene Operationen, verschiedene Atomics. - Übergabe (hand-off) — ein Thread ist mit ein paar **Bytes** fertig und signalisiert es, ein anderer holt sie ab: eine Sendeseite und eine Empfangsseite.*

Different operations, different atomics. - hand-off — one thread finishing with some **bytes** and signaling, another picking them up: a sending side and a receiving side.

die **Cache Line**

-s T0

cache line (CPU cache coherency unit)

*Die Arbeitsteilung zwischen den drei Instrumenten, ein Drop, das Miris Leak-Checker absegnet, und der Benchmark: `head` und `tail` mit Padding auf getrennte **cache lines** zu legen, bringt die SPSC-Übergabe von ~51 ns auf ~9 ns.*

The division of labor between the three instruments, a Drop that Miri's leak checker approves, and the benchmark: padding `head` and `tail` onto separate **cache lines** takes the SPSC hand-off from ~51 ns to ~9 ns.

das **CachePadded**

T0

CachePadded wrapper (crossbeam false-sharing prevention)

*Kein Lock-free-Hintergrund wird vorausgesetzt; `compare_exchange`, `Acquire/Release`, `UnsafeCell`, `MaybeUninit`, `loom`, `Miri` und **'CachePadded'** werden dort eingeführt, wo das Design mit ihnen kollidiert.*

No lock-free background is assumed; `compare_exchange`, `Acquire/Release`, `UnsafeCell`, `MaybeUninit`, `loom`, `Miri` and **'CachePadded'** are introduced where the design collides with them.

der **Channel**

-s T1

channel (message passing queue)

*Ein unbeschränkter **Channel** — `std::sync::mpsc` oder irgendeine Queue aus verketteten Knoten — umgeht das eine Lock, indem er pro Nachricht einen Knoten alloziert.*

An unbounded **channel** — `std::sync::mpsc`, or any linked-node queue — dodges the single lock by allocating a node per message.

der **Claim**

-s T0

claim (ticket acquisition)

*[Teil 2 — Ein Zähler kann nicht „geschrieben“ sagen.](02_the_sequence.md) Der naheliegende Ring zerreißt in dem Moment, in dem ein Consumer neben einem Producer läuft, weil `tail` beim ***Claim*** weiterrückt, vor dem Write.*

[Part 2 — One counter can't say "written".](02_the_sequence.md) The obvious ring tears the instant a consumer runs beside a producer, because the tail moves at ***claim*** time, before the write.

der **Code**

-s T1

code, source code

*## Die naheliegenden Queues, je eine Randbedingung verletzt — `Mutex<VecDeque<T>>` ist das, was der meiste **Code** ausliefert, und es funktioniert.*

The obvious queues, one constraint each — `Mutex<VecDeque<T>>` is what most **code** ships, and it works.

der Commit

-s T0

commit (WAL group transaction)

*# Einen lock-freien MPMC-Ring entwerfen — Am schreibenden Ende von allem, was schnell ist — eine Matching Engine, die Orders von einem Dutzend Feed-Threads entgegennimmt, ein Write-Ahead-Log, das viele Writer in einen einzigen **Commit** bündelt —, sitzt eine Queue, in die viele Threads pushen und aus der viele Threads poppen, millionenfach pro Sekunde.*

Designing a lock-free MPMC ring — At the write end of anything fast — a matching engine taking orders off a dozen feed threads, a write-ahead log fanning writers into one **commit** — there's a queue that many threads push and many threads pop, millions of times a second.

der Compare-and-Swap

-s T0

compare-and-swap (atomic CAS)

*Boden und Dach, ein Gate nach dem anderen, warum der **CAS** Relaxed bleibt, und warum dieser Ring — anders als ein SeqLock — keinen Fence braucht.*

Floor and roof, one gate at a time, why the **CAS** stays Relaxed, and why this ring — unlike a SeqLock — needs no fence.

die Concurrency

T0

concurrency

*## Umfang — Ein generischer, beschränkter `MpmcRing<T>` — der Vyukov-Algorithmus, so wie man ihn in eine **Concurrency**-Crate legen würde.*

Scope — A generic bounded `MpmcRing<T>` — the Vyukov algorithm, as you'd put it in a **concurrency** crate.

der Consumer

- T0

consumer thread

*Diese Serie entwirft die Queue, die das nicht tut: einen Ring — beschränkt, alloktationsfrei, lock-free —, den sich N Producer und M **Consumer** teilen und bei dem nur noch die Übergabe selbst etwas kostet.*

This series designs the queue that doesn't: a bounded, allocation-free, lock-free ring shared by N producers and M **consumers**, where the only cost is the hand-off itself.

die Contention

T1

contention (thread contention on a lock/atomic)

*Aber jeder push und jeder pop nimmt dasselbe Lock, also springt das Lock-Wort bei jeder einzelnen Operation wie ein Pingpongball zwischen den Cores hin und her — das MESI-Protokoll invalidiert eine cache line in jedem anderen Core, millionenfach pro Sekunde, selbst wenn logisch gar keine **Contention** herrscht.*

But every push and every pop takes the same lock, so the lock word pings between cores on every single operation — the MESI protocol invalidating one cache line in every other core, millions of times a second, even when nobody is logically **contending**.

der Core

-s T0

CPU core

*Greif zu einem `Mutex<VecDeque>`, und jeder dieser Threads serialisiert sich auf einem einzigen Lock-Wort, das zwischen den **Cores** hin- und herspringt.*

Reach for a `Mutex<VecDeque>` and every one of those threads serializes on one lock word that bounces between **cores**.

die Crate

-s T0

Rust crate

Umfang — Ein generischer, beschränkter MpmcRing<T> — der Vyukov-Algorithmus, so wie man ihn in eine Concurrency-Crate legen würde.

Scope — A generic bounded MpmcRing<T> — the Vyukov algorithm, as you'd put it in a concurrency crate.

der Data Race

-s T0

data race (concurrent unsynchronized access)

[Teil 3 — Das Memory Ordering richtig hinbekommen.]

(03_memory_ordering.md) Alles ReLaxed, und der Ring ist ein Data Race.

[Part 3 — Getting the memory ordering right.](03_memory_ordering.md)

All-ReLaxed, the ring is a data race.

das Double Free

-s T4

double free (memory error)

Diese Präzision ist der Grund, warum Miri zuerst drankommt. — Es fängt auch die anderen Dinge, die eine Queue mit Speicher falsch machen kann — einen uninitialisierten Read, ein Leak, ein Double Free — und Spin-Schleifen stören es nicht.

That precision is why Miri comes first. — It also catches the other things a queue can do wrong with memory — an uninitialized read, a leak, a double free — and it's fine with spin loops.

das Drop

-s T0

Drop implementation / destructors

Die Arbeitsteilung zwischen den drei Instrumenten, ein Drop, das Miris Leak-Checker absegnet, und der Benchmark: head und tail mit Padding aufgetrennte cache lines zu legen, bringt die SPSC-Übergabe von ~51 ns auf ~9 ns.

The division of labor between the three instruments, a Drop that Miri's leak checker approves, and the benchmark: padding head and tail onto separate cache lines takes the SPSC hand-off from ~51 ns to ~9 ns.

die Engine

-s T0

engine (matching engine)

Einen lock-freien MPMC-Ring entwerfen — Am schreibenden Ende von allem, was schnell ist — eine Matching Engine, die Orders von einem Dutzend Feed-Threads entgegennimmt, ein Write-Ahead-Log, das viele Writer in einen einzigen Commit bündelt —, sitzt eine Queue, in die viele Threads pushen und aus der viele Threads poppen, millionenfach pro Sekunde.

Designing a lock-free MPMC ring — At the write end of anything fast — a matching engine taking orders off a dozen feed threads, a write-ahead log fanning writers into one commit — there's a queue that many threads push and many threads pop, millions of times a second.

die Exchange

-s T1

exchange (trading venue)

Teil 1 — Das Problem, und warum die naheliegenden Queues nicht passen — Zwölf Feed-Handler, einer pro Core, jeder zieht Orders aus einer anderen Exchange-Session und schiebt sie an eine einzige Stelle: den Eingang der Matching Engine.

Part 1 — The problem, and why the obvious queues don't fit — Twelve feed handlers, one per core, each pulling orders off a different exchange session and pushing them into one place: the matching engine's intake.

das False Sharing

T0

false sharing (cache line contention)

Zwei pro Slot: A (publish, Producer → Consumer) und B (vacate, Consumer → Producer der nächsten Runde). - **compare_exchange** (CAS) — atomares Read-Modify-Write, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so beansprucht genau ein Thread ein Ticket. - **Relaxed / Acquire / Release** — **Relaxed** ist atomar ohne Ordering; **Release** auf einem Store ist ein Boden unter allem, was davor kommt; **Acquire** auf einem Load ist ein Dach über allem, was danach kommt; ein **Acquire-Load**, der einen **Release-Store** beobachtet, verbindet die beiden. - **happens-before** — die threadübergreifende Garantie, die ein **Release/Acquire**-Paar aufbaut; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - **False Sharing** — zwei unabhängige Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores invalidiert; behoben durch **CachePadded**. - **loom** — ein Model Checker, der einen kleinen Test unter jeder Verschränkung ausführt; blind für Übergaben, die über einen CAS laufen. - **Miri** — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB meldet — Data Races, uninitialisierte Reads, Leaks.

Two per slot: A (publish, producer → consumer) and B (vacate, consumer → next lap's producer). - **compare_exchange** (CAS) — atomic read-modify-write that succeeds only if the value is what you expected; how exactly one thread claims a ticket. - **Relaxed / Acquire / Release** — **Relaxed** is atomic with no ordering; **Release** on a store floors what comes before it; **Acquire** on a load roofs what comes after it; an **Acquire** load that observes a **Release** store joins the two. - **happens-before** — the cross-thread guarantee a **Release/Acquire** pair builds; without one, a plain write on one thread and a plain read on another are a data race. - **false sharing** — two independent values on one cache line, each write invalidating the other core's copy; fixed by **CachePadded**. - **loom** — a model checker that runs a small test under every interleaving; blind to hand-offs carried by a CAS. - **Miri** — an interpreter that runs Rust against the memory model and reports UB — data races, uninitialized reads, leaks.

der Feed

-s T0

market data feed

Einen lock-freien MPMC-Ring entwerfen — Am schreibenden Ende von allem, was schnell ist — eine Matching Engine, die Orders von einem Dutzend **Feed**-Threads entgegennimmt, ein Write-Ahead-Log, das viele Writer in einen einzigen Commit bündelt —, sitzt eine Queue, in die viele Threads pushen und aus der viele Threads poppen, millionenfach pro Sekunde.

Designing a lock-free MPMC ring — At the write end of anything fast — a matching engine taking orders off a dozen **feed** threads, a write-ahead log fanning writers into one commit — there's a queue that many threads push and many threads pop, millions of times a second.

der Fence

-s T0

standalone memory barrier

Boden und Dach, ein Gate nach dem anderen, warum der CAS **Relaxed** bleibt, und warum dieser Ring — anders als ein SeqLock — keinen **Fence** braucht.

Floor and roof, one gate at a time, why the CAS stays **Relaxed**, and why this ring — unlike a SeqLock — needs no **fence**.

das **FIFO**

T2

first-in first-out ordering

*Consumer springen nicht vor. — Genau das hält den Ring **FIFO**, und das ist der Preis dafür.*

Consumers don't skip ahead. — That's what keeps the ring **FIFO**, and it's the price of it.

das **Gate**

-s T0

gate (memory ordering check / boundary)

*Ein Drei-Wege-**Gate**, die **publish**- und **vacate**-Regeln und der Vertrag darüber, wann **try_push** erneut versucht und wann es zurückkehrt.*

A three-way **gate**, the **publish** and **vacate** rules, and the contract for when **try_push** retries and when it returns.

das **Group Commit**

-s T1

group commit

*Diese Form taucht überall auf, wo schnelle Software ein schreibendes Ende hat. — Ein Write-Ahead-Log bündelt ein Dutzend Writer-Threads in einen **Group Commit**.*

The shape is everywhere fast software has a write end. — A write-ahead log fans a dozen writer threads into one **group commit**.

das **Happens-before**

T0

happens-before relationship

*Zwei pro Slot: A (**publish**, Producer → Consumer) und B (**vacate**, Consumer → Producer der nächsten Runde). - **compare_exchange** (CAS) — atomares Read-Modify-Write, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so beansprucht genau ein Thread ein Ticket. - **Relaxed / Acquire / Release** — **Relaxed** ist atomar ohne Ordering; **Release** auf einem Store ist ein Boden unter allem, was davor kommt; **Acquire** auf einem Load ist ein Dach über allem, was danach kommt; ein **Acquire-Load**, der einen **Release-Store** beobachtet, verbindet die beiden. - **happens-before** — die threadübergreifende Garantie, die ein **Release/Acquire**-Paar aufbaut; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - **False Sharing** — zwei unabhängige Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores invalidiert; behoben durch **CachePadded**. - **loom** — ein Model Checker, der einen kleinen Test unter jeder Verschränkung ausführt; blind für Übergaben, die über einen CAS laufen. - **Miri** — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB meldet — Data Races, uninitialisierte Reads, Leaks.*

Two per slot: A (**publish**, producer → consumer) and B (**vacate**, consumer → next lap's producer). - **compare_exchange** (CAS) — atomic read-modify-write that succeeds only if the value is what you expected; how exactly one thread claims a ticket. - **Relaxed / Acquire / Release** — **Relaxed** is atomic with no ordering; **Release** on a store floors what comes before it; **Acquire** on a load roofs what comes after it; an **Acquire** load that observes a **Release** store joins the two. - **happens-before** — the cross-thread guarantee a **Release/Acquire** pair builds; without one, a plain write on one thread and a plain read on another are a data race. - **false sharing** — two independent values on one cache line, each write invalidating the other core's copy; fixed by **CachePadded**. - **loom** — a model checker that runs a small test under every interleaving; blind to hand-offs carried by a CAS. - **Miri** — an interpreter that runs Rust against the memory model and reports UB — data races, uninitialized reads, leaks.

die Hardware

T0

hardware

*Nach Teil 2 aufzuhören lässt dich mit einem Ring zurück, der logisch korrekt ist; die Teile 3 und 4 sind der Ort, an dem er auf die **Hardware** und die Verifizierer trifft und an dem die Fehler wohnen, die Tests bestehen.*

Stopping after Part 2 leaves you with a ring that's logically correct; Parts 3 and 4 are where it meets the **hardware** and the verifiers, and where the failures that pass tests live.

der Head

-s T0

head pointer / ticket counter

*Die Arbeitsteilung zwischen den drei Instrumenten, ein Drop, das Miris Leak-Checker absegnet, und der Benchmark: **'head und tail'** mit Padding auf getrennte cache lines zu legen, bringt die SPSC-Übergabe von ~51 ns auf ~9 ns.*

The division of labor between the three instruments, a Drop that Miri's leak checker approves, and the benchmark: padding **'head und tail'** onto separate cache lines takes the SPSC hand-off from ~51 ns to ~9 ns.

der Hot Path

-s T1

critical performance path

*Jedes Mal dasselbe Objekt: viele Threads pushen, viele Threads poppen, auf dem **Hot Path**, bei einer Rate, bei der der Overhead der Queue selbst das Budget ist.*

Same object each time: many threads push, many threads pop, on the **hot path**, at a rate where the queue's own overhead is the budget.

das Leak

-s T0

memory leak

*Die Arbeitsteilung zwischen den drei Instrumenten, ein Drop, das Miris **Leak-Checker** absegnet, und der Benchmark: **head und tail** mit Padding auf getrennte cache lines zu legen, bringt die SPSC-Übergabe von ~51 ns auf ~9 ns.*

The division of labor between the three instruments, a Drop that Miri's **leak checker** approves, and the benchmark: padding **head und tail** onto separate cache lines takes the SPSC hand-off from ~51 ns to ~9 ns.

der Leak-Checker

- T0

leak checker

*Die Arbeitsteilung zwischen den drei Instrumenten, ein Drop, das Miris **Leak-Checker** absegnet, und der Benchmark: **head und tail** mit Padding auf getrennte cache lines zu legen, bringt die SPSC-Übergabe von ~51 ns auf ~9 ns.*

The division of labor between the three instruments, a Drop that Miri's **leak checker** approves, and the benchmark: padding **head und tail** onto separate cache lines takes the SPSC hand-off from ~51 ns to ~9 ns.

der Load

-s T0

atomic load

Zwei pro Slot: A (publish, Producer → Consumer) und B (vacate, Consumer → Producer der nächsten Runde). - compare_exchange (CAS) — atomares Read-Modify-Write, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so beansprucht genau ein Thread ein Ticket. - Relaxed / Acquire / Release — Relaxed ist atomar ohne Ordering; Release auf einem Store ist ein Boden unter allem, was davor kommt; Acquire auf einem Load ist ein Dach über allem, was danach kommt; ein Acquire-Load, der einen Release-Store beobachtet, verbindet die beiden. - happens-before — die threadübergreifende Garantie, die ein Release/Acquire-Paar aufbaut; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - False Sharing — zwei unabhängige Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores invalidiert; behoben durch CachePadded. - loom — ein Model Checker, der einen kleinen Test unter jeder Verschränkung ausführt; blind für Übergaben, die über einen CAS laufen. - Miri — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB meldet — Data Races, uninitialisierte Reads, Leaks.

Two per slot: A (publish, producer → consumer) and B (vacate, consumer → next lap's producer). - compare_exchange (CAS) — atomic read-modify-write that succeeds only if the value is what you expected; how exactly one thread claims a ticket. - Relaxed / Acquire / Release — Relaxed is atomic with no ordering; Release on a store floors what comes before it; Acquire on a load roofs what comes after it; an Acquire load that observes a Release store joins the two. - happens-before — the cross-thread guarantee a Release/Acquire pair builds; without one, a plain write on one thread and a plain read on another are a data race. - false sharing — two independent values on one cache line, each write invalidating the other core's copy; fixed by CachePadded. - loom — a model checker that runs a small test under every interleaving; blind to hand-offs carried by a CAS. - Miri — an interpreter that runs Rust against the memory model and reports UB — data races, uninitialized reads, leaks.

das Lock

-s T0

lock, synchronization lock

Greif zu einem Mutex<VecDeque>, und jeder dieser Threads serialisiert sich auf einem einzigen Lock-Wort, das zwischen den Cores hin- und herspringt.

Reach for a Mutex<VecDeque> and every one of those threads serializes on one lock word that bounces between cores.

das loom

T0

loom (permutation testing framework)

Kein Lock-free-Hintergrund wird vorausgesetzt; compare_exchange, Acquire/Release, UnsafeCell, MaybeUninit, loom, Miri und CachePadded werden dort eingeführt, wo das Design mit ihnen kollidiert.

No lock-free background is assumed; compare_exchange, Acquire/Release, UnsafeCell, MaybeUninit, loom, Miri and CachePadded are introduced where the design collides with them.

der **Matcher**

- T1

matcher (order-matching thread)

*Ein paar Millionen Operationen pro Sekunde, von beiden Enden gleichzeitig, und auf keiner der beiden Seiten kann es sich ein Thread leisten stehenzubleiben — ein Feed-Thread, der hängt, verliert Marktdaten; ein **Matcher**, der hängt, weitet den Spread bei jedem Symbol, das hinter ihm in der Queue wartet.*

A few million operations a second, from both ends at once, and no thread on either side can afford to stop — a feed thread that stalls drops market data; a **matcher** that stalls widens the spread on every symbol queued behind it.

die **Matching Engine**

-s T0

matching engine (order matching core)

*# Einen lock-freien MPMC-Ring entwerfen — Am schreibenden Ende von allem, was schnell ist — eine **Matching Engine**, die Orders von einem Dutzend Feed-Threads entgegennimmt, ein Write-Ahead-Log, das viele Writer in einen einzigen Commit bündelt —, sitzt eine Queue, in die viele Threads pushen und aus der viele Threads poppen, millionenfach pro Sekunde.*

Designing a lock-free MPMC ring — At the write end of anything fast — a **matching engine** taking orders off a dozen feed threads, a write-ahead log fanning writers into one commit — there's a queue that many threads push and many threads pop, millions of times a second.

das **Miri**

T0

Miri (Rust interpreter for UB/memory safety)

*Kein Lock-free-Hintergrund wird vorausgesetzt; `compare_exchange`, `Acquire/Release`, `UnsafeCell`, `MaybeUninit`, `loom`, **Miri** und `CachePadded` werden dort eingeführt, wo das Design mit ihnen kollidiert.*

No lock-free background is assumed; `compare_exchange`, `Acquire/Release`, `UnsafeCell`, `MaybeUninit`, `loom`, **Miri** and `CachePadded` are introduced where the design collides with them.

der Model Checker

- T0

model checker

Zwei pro Slot: A (publish, Producer → Consumer) und B (vacate, Consumer → Producer der nächsten Runde). - compare_exchange (CAS) — atomares Read-Modify-Write, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so beansprucht genau ein Thread ein Ticket. - Relaxed / Acquire / Release — Relaxed ist atomar ohne Ordering; Release auf einem Store ist ein Boden unter allem, was davor kommt; Acquire auf einem Load ist ein Dach über allem, was danach kommt; ein Acquire-Load, der einen Release-Store beobachtet, verbindet die beiden. - happens-before — die threadübergreifende Garantie, die ein Release/Acquire-Paar aufbaut; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - False Sharing — zwei unabhängige Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores invalidiert; behoben durch CachePadded. - loom — ein **Model Checker**, der einen kleinen Test unter jeder Verschränkung ausführt; blind für Übergaben, die über einen CAS laufen. - Miri — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB meldet — Data Races, uninitialisierte Reads, Leaks.

Two per slot: A (publish, producer → consumer) and B (vacate, consumer → next lap's producer). - compare_exchange (CAS) — atomic read-modify-write that succeeds only if the value is what you expected; how exactly one thread claims a ticket. - Relaxed / Acquire / Release — Relaxed is atomic with no ordering; Release on a store floors what comes before it; Acquire on a load roofs what comes after it; an Acquire load that observes a Release store joins the two. - happens-before — the cross-thread guarantee a Release/Acquire pair builds; without one, a plain write on one thread and a plain read on another are a data race. - false sharing — two independent values on one cache line, each write invalidating the other core's copy; fixed by CachePadded. - loom — a **model checker** that runs a small test under every interleaving; blind to hand-offs carried by a CAS. - Miri — an interpreter that runs Rust against the memory model and reports UB — data races, uninitialized reads, leaks.

das Modulo

-s T1

modulo operation

Dieses & ist ein **Modulo**, aber nur, wenn capacity eine Zweierpotenz ist — dann ist $\text{capacity} - 1$ eine Folge niedriger 1-Bits, und die Maske behält genau diese.

That & is a **modulo**, but only when capacity is a power of two — then $\text{capacity} - 1$ is a run of low 1 bits and the mask keeps exactly them.

der MPMC-Ring

-e T0

multiple-producer multiple-consumer ring buffer

- **MPMC-Ring** — ein beschränkter Ringpuffer, in den mehrere Producer pushen und aus dem mehrere Consumer poppen, lock-free und alloktationsfrei. - lock-free — irgendein Thread kommt immer voran; kein Thread hält ein Lock, auf das andere warten.

- **MPMC ring** — a bounded circular buffer that multiple producers push and multiple consumers pop, lock-free and allocation-free. - lock-free — some thread always makes progress; no thread holds a lock others wait on.

der **Mutex**

-e T0

mutual exclusion lock

*Greif zu einem **Mutex**<VecDeque>, und jeder dieser Threads serialisiert sich auf einem einzigen Lock-Wort, das zwischen den Cores hin- und herspringt.*

Reach for a **Mutex**<VecDeque> and every one of those threads serializes on one lock word that bounces between cores.

die **Order**

-s T0

trade order

*# Einen lock-freien MPMC-Ring entwerfen — Am schreibenden Ende von allem, was schnell ist — eine Matching Engine, die **Orders** von einem Dutzend Feed-Threads entgegennimmt, ein Write-Ahead-Log, das viele Writer in einen einzigen Commit bündelt —, sitzt eine Queue, in die viele Threads pushen und aus der viele Threads poppen, millionenfach pro Sekunde.*

Designing a lock-free MPMC ring — At the write end of anything fast — a matching engine taking **orders** off a dozen feed threads, a write-ahead log fanning writers into one commit — there's a queue that many threads push and many threads pop, millions of times a second.

das **Ordering**

-s T0

memory ordering

*[Teil 3 — Das Memory **Ordering** richtig hinbekommen.]*

*(03_memory_ordering.md) Alles **ReLaxed**, und der Ring ist ein Data Race.*

[Part 3 — Getting the memory **ordering** right.](03_memory_ordering.md)
All-**ReLaxed**, the ring is a data race.

das **OS**

T1

operating system (OS scheduler)

*Und in dem Moment, in dem der Halter des Locks vom **OS** verdrängt wird, wartet jeder andere Thread darauf, dass genau dieser eine Thread wieder eingeplant wird.*

And the instant the holder is preempted, every other thread waits for that one thread to be scheduled again.

das **Padding**

-s T0

cache padding

*Die Arbeitsteilung zwischen den drei Instrumenten, ein Drop, das Miris Leak-Checker absegnet, und der Benchmark: **head** und **tail** mit **Padding** auf getrennte cache lines zu legen, bringt die SPSC-Übergabe von ~51 ns auf ~9 ns.*

The division of labor between the three instruments, a Drop that Miri's leak checker approves, and the benchmark: **padding** **head** and **tail** onto separate cache lines takes the SPSC hand-off from ~51 ns to ~9 ns.

der **Payload**

-s T0

payload (stored data value)

*Er zählt nur hoch: **pos** (Producer darf schreiben), **pos + 1** (Consumer darf lesen), **pos + capacity** (der Producer der nächsten Runde darf schreiben). - **claim** vs. **publish** — ein Ticket auf dem Zähler ziehen vs. den **payload** über **seq** sichtbar machen.*

It only increases: **pos** (producer may write), **pos + 1** (consumer may read), **pos + capacity** (next lap's producer may write). - **claim** vs **publish** — taking a ticket on the counter vs making the **payload** visible through **seq**.

die Pipeline

-s T1

pipeline (metrics or processing pipeline)

Ein Write-Ahead-Log bündelt ein Dutzend Writer-Threads in einen Group Commit. — Eine Metrics-Pipeline führt Samples von jedem Worker in einem einzigen Aggregator zusammen.

A write-ahead log fans a dozen writer threads into one group commit. — A metrics **pipeline** funnels samples from every worker into one aggregator.

die Policy

-ies T1

policy (caller retry/blocking policy)

Die Queue parkt nie einen Thread. — Blockieren ist eine Policy, die der Aufrufer vielleicht will, und sie darf nicht fest ins Primitiv eingebaut sein.

The queue never parks a thread. — Blocking is a **policy** the caller may want, and it must not be baked into the primitive.

der Pool

-s T1

thread pool

Zwölf Feed-Handler, einer pro Core, jeder zieht Orders aus einer anderen Exchange-Session und schiebt sie an eine einzige Stelle: den Eingang der Matching Engine. Auf der anderen Seite poppt ein Pool von Matching-Threads aus genau diesem Eingang.

Twelve feed handlers, one per core, each pulling orders off a different exchange session and pushing them into one place: the matching engine's intake. On the other side, a **pool** of matching threads pops from that same intake.

der Prefetcher

- T4

CPU hardware prefetcher

Das ist False Sharing, und der Fix ist, jeden Zähler auf seine eigene cache line zu legen: CachePadded, 128 Bytes auf Apple Silicon, wo der Prefetcher cache lines paarweise holt.

That's false sharing, and the fix is to put each counter on its own line: CachePadded, 128 bytes on Apple silicon, where the **prefetcher** pulls lines in pairs.

der Producer

- T0

producer thread

Diese Serie entwirft die Queue, die das nicht tut: einen Ring — beschränkt, allokatonsfrei, lock-free —, den sich N Producer und M Consumer teilen und bei dem nur noch die Übergabe selbst etwas kostet.

This series designs the queue that doesn't: a bounded, allocation-free, lock-free ring shared by N **producers** and M consumers, where the only cost is the hand-off itself.

die Queue

-s T0

queue

Diese Serie entwirft die Queue, die das nicht tut: einen Ring — beschränkt, allokatonsfrei, lock-free —, den sich N Producer und M Consumer teilen und bei dem nur noch die Übergabe selbst etwas kostet.

This series designs the **queue** that doesn't: a bounded, allocation-free, lock-free ring shared by N producers and M consumers, where the only cost is the hand-off itself.

der Race

-s T0

data race

[Teil 3 – Das Memory Ordering richtig hinbekommen.]

(03_memory_ordering.md) Alles **Relaxed**, und der Ring ist ein Data **Race**.

[Part 3 – Getting the memory ordering right.](03_memory_ordering.md)

All-**Relaxed**, the ring is a data **race**.

das Read

-s T0

read operation

Zwei pro Slot: A (**publish**, **Producer** → **Consumer**) und B (**vacate**, **Consumer** → **Producer** der nächsten Runde). - **compare_exchange** (CAS) — atomares **Read-Modify-Write**, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so beansprucht genau ein Thread ein Ticket. - **Relaxed / Acquire / Release** — **Relaxed** ist atomar ohne Ordering; **Release** auf einem Store ist ein Boden unter allem, was davor kommt; **Acquire** auf einem Load ist ein Dach über allem, was danach kommt; ein **Acquire-Load**, der einen **Release-Store** beobachtet, verbindet die beiden. - **happens-before** — die threadübergreifende Garantie, die ein **Release/Acquire**-Paar aufbaut; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - **False Sharing** — zwei unabhängige Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores invalidiert; behoben durch **CachePadded**. - **loom** — ein Model Checker, der einen kleinen Test unter jeder Verschränkung ausführt; blind für Übergaben, die über einen CAS laufen. - **Miri** — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB meldet — Data Races, uninitialisierte Reads, Leaks.

Two per slot: A (**publish**, **producer** → **consumer**) and B (**vacate**, **consumer** → next lap's **producer**). - **compare_exchange** (CAS) — atomic **read-modify-write** that succeeds only if the value is what you expected; how exactly one thread claims a ticket. - **Relaxed / Acquire / Release** — **Relaxed** is atomic with no ordering; **Release** on a store floors what comes before it; **Acquire** on a load roofs what comes after it; an **Acquire load** that observes a **Release store** joins the two. - **happens-before** — the cross-thread guarantee a **Release/Acquire** pair builds; without one, a plain write on one thread and a plain read on another are a data race. - **false sharing** — two independent values on one cache line, each write invalidating the other core's copy; fixed by **CachePadded**. - **loom** — a model checker that runs a small test under every interleaving; blind to hand-offs carried by a CAS. - **Miri** — an interpreter that runs Rust against the memory model and reports UB — data races, uninitialized reads, leaks.

das Read-Modify-Write

-s T0

read-modify-write (atomic RMW)

Zwei pro Slot: A (publish, Producer → Consumer) und B (vacate, Consumer → Producer der nächsten Runde). - `compare_exchange` (CAS) — atomares **Read-Modify-Write**, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so beansprucht genau ein Thread ein Ticket. - `Relaxed / Acquire / Release` — `Relaxed` ist atomar ohne Ordering; `Release` auf einem Store ist ein Boden unter allem, was davor kommt; `Acquire` auf einem Load ist ein Dach über allem, was danach kommt; ein `Acquire-Load`, der einen `Release-Store` beobachtet, verbindet die beiden. - `happens-before` — die threadübergreifende Garantie, die ein `Release/Acquire`-Paar aufbaut; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - `False Sharing` — zwei unabhängige Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores invalidiert; behoben durch `CachePadded`. - `loom` — ein Model Checker, der einen kleinen Test unter jeder Verschränkung ausführt; blind für Übergaben, die über einen CAS laufen. - `Miri` — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB meldet — Data Races, uninitialisierte Reads, Leaks.

Two per slot: A (publish, producer → consumer) and B (vacate, consumer → next lap's producer). - `compare_exchange` (CAS) — atomic **read-modify-write** that succeeds only if the value is what you expected; how exactly one thread claims a ticket. - `Relaxed / Acquire / Release` — `Relaxed` is atomic with no ordering; `Release` on a store floors what comes before it; `Acquire` on a load roofs what comes after it; an `Acquire` load that observes a `Release` store joins the two. - `happens-before` — the cross-thread guarantee a `Release/Acquire` pair builds; without one, a plain write on one thread and a plain read on another are a data race. - `false sharing` — two independent values on one cache line, each write invalidating the other core's copy; fixed by `CachePadded`. - `loom` — a model checker that runs a small test under every interleaving; blind to hand-offs carried by a CAS. - `Miri` — an interpreter that runs Rust against the memory model and reports UB — data races, uninitialized reads, leaks.

der Retry

-s T4

retry attempt

Eine Contention-Achse pro Modell — zwei Producer und ein Consumer oder zwei Consumer und ein Producer, nie alles auf einmal — eine Kapazität von zwei, die Producer gejoint, bevor der Consumer den Ring leert, und jeder **Retry** im Testcode beschränkt.

One contention axis per model — two producers and a consumer, or two consumers and a producer, never everything at once — a capacity of two, producers joined before the consumer drains, and any **retry** in the test body bounded.

das Sample

-s T1

sample, data point

Ein Write-Ahead-Log bündelt ein Dutzend Writer-Threads in einen Group Commit. — Eine Metrics-Pipeline führt **Samples** von jedem Worker in einem einzigen Aggregator zusammen.

A write-ahead log fans a dozen writer threads into one group commit. — A metrics pipeline funnels **samples** from every worker into one aggregator.

der Scheduler

- T2

OS thread scheduler

Die Menge der tatsächlich geschriebenen Slots ist {1}. — Kein einzelner Zähler kann „Slot 1 ist geschrieben, Slot 0 nicht“ ausdrücken, denn Claims werden der Reihe nach vergeben, und Writes werden in der Reihenfolge fertig, die dem Scheduler gerade gefällt.

The set of slots actually written is {1}. — No single counter can express "slot 1 is written, slot 0 isn't", because claims are handed out in order and writes complete in whatever order the scheduler likes.

das Shim

-s T4

shim layer (e.g. cfg(loom) shim)

Der payload-Zugriff läuft über `loom::cell::UnsafeCell`, damit loom ihn sieht, hinter einem `cfg(loom)-Shim`, der looms Atomics einwechselt.

Payload access goes through `loom::cell::UnsafeCell` so loom can see it, behind a `cfg(loom) shim` that swaps in loom's atomics.

der Slot

-s T8

slot, buffer cell

Ein Boolean pro Slot kann das nicht reparieren — ein Boolean hat kein Gedächtnis, und Slots werden über Runden hinweg wiederverwendet.

A per-slot boolean can't fix it — a boolean has no memory, and slots are reused across laps.

der Software

T1

software

Diese Form taucht überall auf, wo schnelle Software ein schreibendes Ende hat. — Ein Write-Ahead-Log bündelt ein Dutzend Writer-Threads in einen Group Commit.

The shape is everywhere fast software has a write end. — A write-ahead log fans a dozen writer threads into one group commit.

der Spread

-s T1

bid-ask spread

Ein paar Millionen Operationen pro Sekunde, von beiden Enden gleichzeitig, und auf keiner der beiden Seiten kann es sich ein Thread leisten stehenzubleiben — ein Feed-Thread, der hängt, verliert Marktdaten; ein Matcher, der hängt, weitet den Spread bei jedem Symbol, das hinter ihm in der Queue wartet.

A few million operations a second, from both ends at once, and no thread on either side can afford to stop — a feed thread that stalls drops market data; a matcher that stalls widens the spread on every symbol queued behind it.

der Store

-s T0

atomic store

Zwei pro Slot: A (publish, Producer → Consumer) und B (vacate, Consumer → Producer der nächsten Runde). - compare_exchange (CAS) — atomares Read-Modify-Write, das nur gelingt, wenn der Wert dem entspricht, was du erwartet hast; so beansprucht genau ein Thread ein Ticket. - Relaxed / Acquire / Release — Relaxed ist atomar ohne Ordering; Release auf einem Store ist ein Boden unter allem, was davor kommt; Acquire auf einem Load ist ein Dach über allem, was danach kommt; ein Acquire-Load, der einen Release-Store beobachtet, verbindet die beiden. - happens-before — die threadübergreifende Garantie, die ein Release/Acquire-Paar aufbaut; ohne sie sind ein einfacher Write auf einem Thread und ein einfacher Read auf einem anderen ein Data Race. - False Sharing — zwei unabhängige Werte auf einer cache line, wobei jeder Write die Kopie des anderen Cores invalidiert; behoben durch CachePadded. - loom — ein Model Checker, der einen kleinen Test unter jeder Verschränkung ausführt; blind für Übergaben, die über einen CAS laufen. - Miri — ein Interpreter, der Rust gegen das Memory-Modell ausführt und UB meldet — Data Races, uninitialisierte Reads, Leaks.

Two per slot: A (publish, producer → consumer) and B (vacate, consumer → next lap's producer). - compare_exchange (CAS) — atomic read-modify-write that succeeds only if the value is what you expected; how exactly one thread claims a ticket. - Relaxed / Acquire / Release — Relaxed is atomic with no ordering; Release on a store floors what comes before it; Acquire on a load roofs what comes after it; an Acquire load that observes a Release store joins the two. - happens-before — the cross-thread guarantee a Release/Acquire pair builds; without one, a plain write on one thread and a plain read on another are a data race. - false sharing — two independent values on one cache line, each write invalidating the other core's copy; fixed by CachePadded. - loom — a model checker that runs a small test under every interleaving; blind to hand-offs carried by a CAS. - Miri — an interpreter that runs Rust against the memory model and reports UB — data races, uninitialized reads, leaks.

das Tail

-s T0

tail pointer / producer counter

[Teil 2 — Ein Zähler kann nicht „geschrieben“ sagen.](02_the_sequence.md) Der naheliegende Ring zerreißt in dem Moment, in dem ein Consumer neben einem Producer läuft, weil `tail` beim `*Claim*` weiterrückt, vor dem Write.

[Part 2 — One counter can't say "written".](02_the_sequence.md) The obvious ring tears the instant a consumer runs beside a producer, because the `tail` moves at `*claim*` time, before the write.

der Test

-s T0

test, test suite

Nach Teil 2 aufzuhören lässt dich mit einem Ring zurück, der logisch korrekt ist; die Teile 3 und 4 sind der Ort, an dem er auf die Hardware und die Verifizierer trifft und an dem die Fehler wohnen, die Tests bestehen.

Stopping after Part 2 leaves you with a ring that's logically correct; Parts 3 and 4 are where it meets the hardware and the verifiers, and where the failures that pass tests live.

der Testcode

-s T4

test code

*Eine Contention-Achse pro Modell — zwei Producer und ein Consumer oder zwei Consumer und ein Producer, nie alles auf einmal — eine Kapazität von zwei, die Producer gejoint, bevor der Consumer den Ring leert, und jeder Retry im **Testcode** beschränkt.*

One contention axis per model — two producers and a consumer, or two consumers and a producer, never everything at once — a capacity of two, producers joined before the consumer drains, and any retry in the **test body** bounded.

der Thread

-s T0

thread (OS thread)

*Greif zu einem `Mutex<VecDeque>`, und jeder dieser **Threads** serialisiert sich auf einem einzigen Lock-Wort, das zwischen den Cores hin- und herspringt.*

Reach for a `Mutex<VecDeque>` and every one of those **threads** serializes on one lock word that bounces between cores.

das Ticket

-s T0

ticket (sequencing token)

*Das Lock erledigt, wie sich herausstellt, zwei Jobs — Reihen vergeben und Daten übergeben —, und die beiden zu trennen ist das ganze Design: ein Array, zwei **Ticket**-Zähler und eine Maske.*

The lock turns out to be doing two jobs, allocating turns and handing off data, and separating them is the whole design: an array, two **ticket** counters, and a mask.

das Tutorial

-s T0

tutorial

*Es ist eine Design-Untersuchung, kein **Tutorial**. — Jede Entscheidung ist erzwungen — durch einen Use Case, durch das Scheitern der einfacheren Variante oder durch ein Verifikationswerkzeug.*

It's a design investigation, not a **tutorial**. — Every decision is forced — by a use case, by the simpler version failing, or by a verification tool.

der Use Case

-s T0

use case

*Es ist eine Design-Untersuchung, kein Tutorial. — Jede Entscheidung ist erzwungen — durch einen **Use Case**, durch das Scheitern der einfacheren Variante oder durch ein Verifikationswerkzeug.*

It's a design investigation, not a tutorial. — Every decision is forced — by a **use case**, by the simpler version failing, or by a verification tool.

der Worker

- T1

worker thread

*Ein Write-Ahead-Log bündelt ein Dutzend Writer-Threads in einen Group Commit. — Eine Metrics-Pipeline führt Samples von jedem **Worker** in einem einzigen Aggregator zusammen.*

A write-ahead log fans a dozen writer threads into one group commit. — A metrics pipeline funnels samples from every **worker** into one aggregator.

der Write

-s T0

write operation

[Teil 2 — Ein Zähler kann nicht „geschrieben“ sagen.](02_the_sequence.md)
Der naheliegende Ring zerreißt in dem Moment, in dem ein Consumer neben einem Producer läuft, weil `tail` beim `*Claim*` weiterrückt, vor dem **Write**.
[Part 2 — One counter can't say "written".](02_the_sequence.md) The obvious ring tears the instant a consumer runs beside a producer, because the tail moves at `*claim*` time, before the **write**.

das Write-Ahead-Log

-s T0

write-ahead log (WAL)

Einen lock-freien MPMC-Ring entwerfen — Am schreibenden Ende von allem, was schnell ist — eine Matching Engine, die Orders von einem Dutzend Feed-Threads entgegennimmt, ein **Write-Ahead-Log**, das viele Writer in einen einzigen Commit bündelt —, sitzt eine Queue, in die viele Threads pushen und aus der viele Threads poppen, millionenfach pro Sekunde.
Designing a lock-free MPMC ring — At the write end of anything fast — a matching engine taking orders off a dozen feed threads, a **write-ahead log** fanning writers into one commit — there's a queue that many threads push and many threads pop, millions of times a second.

der Writer

- T0

writer thread

Einen lock-freien MPMC-Ring entwerfen — Am schreibenden Ende von allem, was schnell ist — eine Matching Engine, die Orders von einem Dutzend Feed-Threads entgegennimmt, ein Write-Ahead-Log, das viele **Writer** in einen einzigen Commit bündelt —, sitzt eine Queue, in die viele Threads pushen und aus der viele Threads poppen, millionenfach pro Sekunde.
Designing a lock-free MPMC ring — At the write end of anything fast — a matching engine taking orders off a dozen feed threads, a write-ahead log fanning **writers** into one commit — there's a queue that many threads push and many threads pop, millions of times a second.

Grammatikmuster the patterns that make the text feel hard

7

Nominalisierter Infinitiv — das Lesen WORTBILDUNG

Jeden Infinitiv großschreiben, und er wird ein neutrales Substantiv = englisches «-ing».

Das **Lesen** ist der Hot Path.

The reading is the hot path.

Das **Zerreißen** lassen wir geschehen und fangen es ab.

We let the tearing happen and catch it.

Das **Schreiben** klammert der Writer mit zwei Bumps.

The writer brackets the writing with two bumps.

Erweitertes Partizipialattribut SATZBAU

Englisch hängt einen Relativsatz an; Deutsch schiebt das Ganze zwischen Artikel und Substantiv. Von hinten lesen: erst das Substantiv, dann nach links auspacken.

ein halb **geschriebener** Wert

a half-written value

die vom Writer gerade **geänderten** Bytes

the bytes the writer is currently changing

der zwischen zwei Bumps **geschriebene** payload

the payload written between two bumps

Konditional ohne «wenn» — Verberststellung SATZBAU

«wenn» weglassen und das Verb an Position eins setzen — kompakter und formeller, überall in den Regeln.

Ist <i>s1 ungerade, liest der Reader gar nicht erst.</i>	If s1 is odd, the reader doesn't even read.
Greift <i>die Kopie auch nur ein Byte von Write N auf, ist s2 ≠ s1.</i>	If the copy grabs even one byte of write N, s2 ≠ s1.

Dativ-Verben FÄLLE

Einige Verben nehmen das Dativobjekt, wo Englisch ein schlichtes Objekt hat.

Wir trauen <i>dem grünen Test nicht.</i>	We don't trust the green test.
Der Code gehört <i>dem Protokoll aus geraden und ungeraden Bumps.</i>	The code obeys the protocol of even and odd bumps.
Der Beweis folgt <i>daraus.</i>	The proof follows from it.

Trennbare Verben im Hauptsatz SATZBAU

Die Vorsilbe springt ans Ende des Satzes; im Nebensatz bleibt sie dran.

Der Reader liest <i>den Zähler zweimal ab.</i>	The reader reads the counter off twice.
<i>..., weil der Writer den Schreibvorgang einklammert.</i>	...because the writer brackets the write.

Komposita — von rechts nach links lesen WORTBILDUNG

Das letzte Element ist der Kopf: es trägt Genus und Kernbedeutung. Alles links davon grenzt es ein.

<i>der Sequenz zähler</i>	sequence + counter → sequence counter
<i>das Maschinen wort</i>	machine + word
<i>die Rand bedingung</i>	edge + condition → constraint
<i>die Cache- Kohärenz</i>	cache + coherence

«man» als allgemeines Subjekt STIL

Wo Englisch «you» oder ein Passiv schreibt, schreibt Deutsch «man» — nie ein Mann, sondern «one, anybody».

Setzt man <i>alle vier Kanten nebeneinander, wählt sich das Werkzeug fast von selbst.</i>	Lay the four edges side by side and the tool almost picks itself.
Man <i>setzt am ungeraden Bump an, nicht am geraden.</i>	One keys on the odd bump, not the even one.

Generiert aus der deutschen Fassung der Serie „Einen lock-freien MPMC-Ring entwerfen“ · Beispielsätze wörtlich aus dem Text, englische Zeile aus der Parallelfassung.